

Introduction to CSS

In the [HTML pack](#), you learned how HTML describes the structure and meaning of a page. HTML gives the browser headings, paragraphs, links, images, forms, and sections to work with.

CSS is the next layer: it tells the browser how that HTML should look, including the font size, text color, background color, spacing, borders, rounded corners, layout, and how the page should adapt to different screen sizes.

HTML and CSS work together, but they do different jobs: HTML says what something is, and CSS says how it should be presented.

html

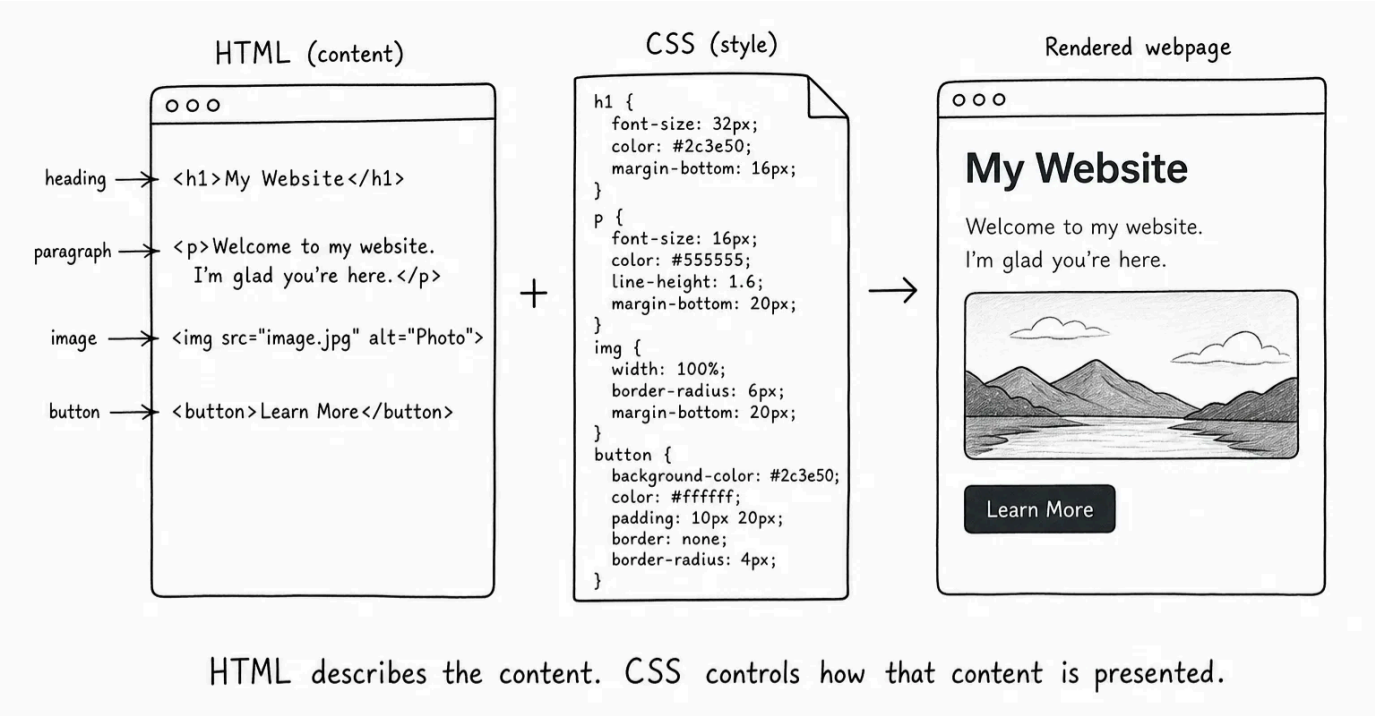
<button>Save changes</button>

That HTML gives the browser a button. CSS can decide whether the button is blue or gray, small or large, square or rounded, flat or shadowed, full-width or only as wide as its text.

CSS

button {
 background-color: #2563eb;
 color: white;
 padding: 12px 16px;
 border: 0;
 border-radius: 6px;
}

CSS lets you take the structure from HTML and turn it into a page that feels designed.



CSS is made of style rules

A CSS file is mostly a list of style rules. Each rule chooses something on the page and gives the browser one or more declarations for how that thing should look.

CSS

```
p {  
  color: #444;  
  font-size: 18px;  
}
```

The rule has three parts you will see everywhere:

- **p** is the **selector**. It chooses which elements the rule applies to.
- **color** and **font-size** are **properties**. They name what you want to change.
- **#444** and **18px** are **values**. They say what each property should

< Previous Lesson

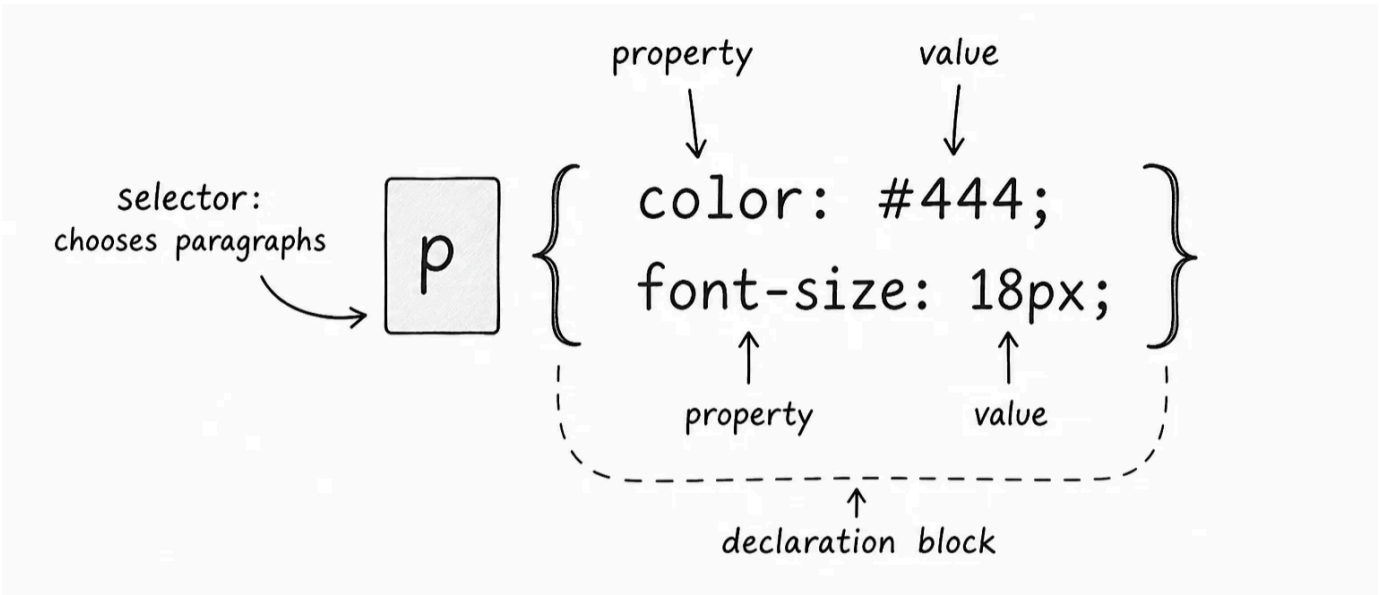
Next Lesson >

The property and value together are called a **declaration**. Each declaration ends with a semicolon, and the declarations sit inside curly braces:

CSS

```
selector {  
  property: value;  
}
```

You do not need to memorize every CSS property before writing CSS. You need to understand that pattern first.



💡 **Read CSS from left to right** - When you see a CSS rule, ask two questions: what does this selector choose, and what declarations does it apply to the matching elements?

Add CSS to HTML

There are three common places you can write CSS.

The smallest option is the **style** attribute. It puts CSS directly on one HTML element:

html

```
<p style="color: #2563eb;">This paragraph is blue.</p>
```

That works, but it mixes the content and the styling in the same line. It also only styles that one element, so it becomes messy if you want several paragraphs to share the same look.

Another option is a **<style>** element inside the **head** :

html

```
<head>
  <style>
    p {
      color: #2563eb;
      font-size: 18px;
    }
  </style>
</head>
```

This keeps the CSS in one place inside the HTML file. It is useful for tiny examples and quick experiments.

For this pack, you will mostly use a separate stylesheet. A stylesheet is a **.css** file linked from your HTML. This is easier to reuse and closer to how real projects organize CSS.

Imagine you have these two files in the same folder:

text

```
index.html
style.css
```

Inside **index.html** , link the stylesheet from the **head** :

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>My styled page</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <h1>Hello, CSS</h1>
    <p>This paragraph will be styled by the CSS file.</p>
  </body>
</html>
```

The `<link>` element tells the browser to load `style.css` and treat it as a stylesheet. The `href` value is the path to the CSS file, just like an image `src` is the path to an image.

Now `style.css` can style the HTML:

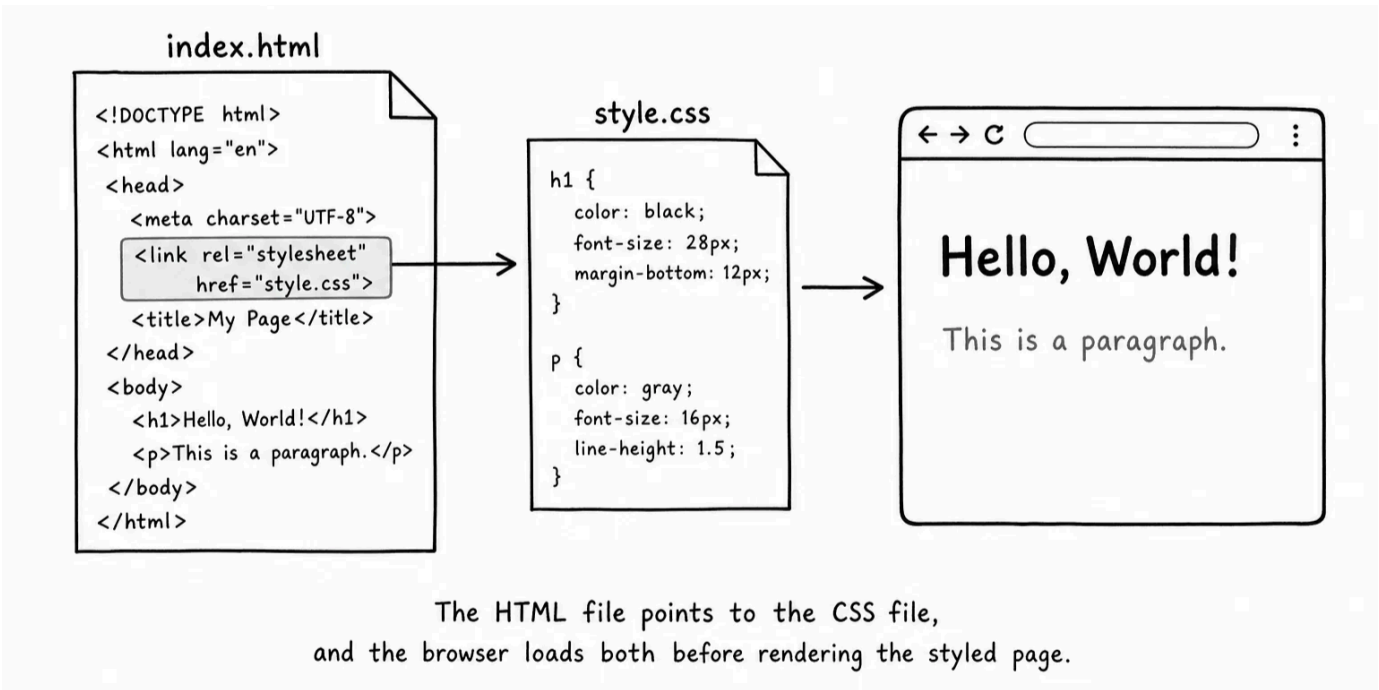
CSS

```
body {
  font-family: system-ui, sans-serif;
  background-color: #f8fafc;
  color: #1f2937;
}

h1 {
  font-size: 40px;
}

p {
  font-size: 18px;
  line-height: 1.6;
}
```

When the browser loads the page, it reads the HTML, finds the stylesheet link, downloads the CSS file, and applies the matching rules to the page.



⚠️ **Check the file path first** - If your CSS does not seem to work at all, check the `href` in your `<link>` element first. A small typo like `styles.css` instead of `style.css` means the browser never loaded the stylesheet.

What CSS can change

CSS can change a lot, but start with the visible stuff: how text reads, how things are spaced, and where they sit.

Text, colors, and backgrounds often show up together:

CSS

```
body {
  color: #111827;
```



```
background-color: #f3f4f6;
}

h1 {
  font-size: 36px;
  font-weight: 700;
  line-height: 1.1;
}
```

Spacing and box styles turn plain content into interface pieces:

```
CSS

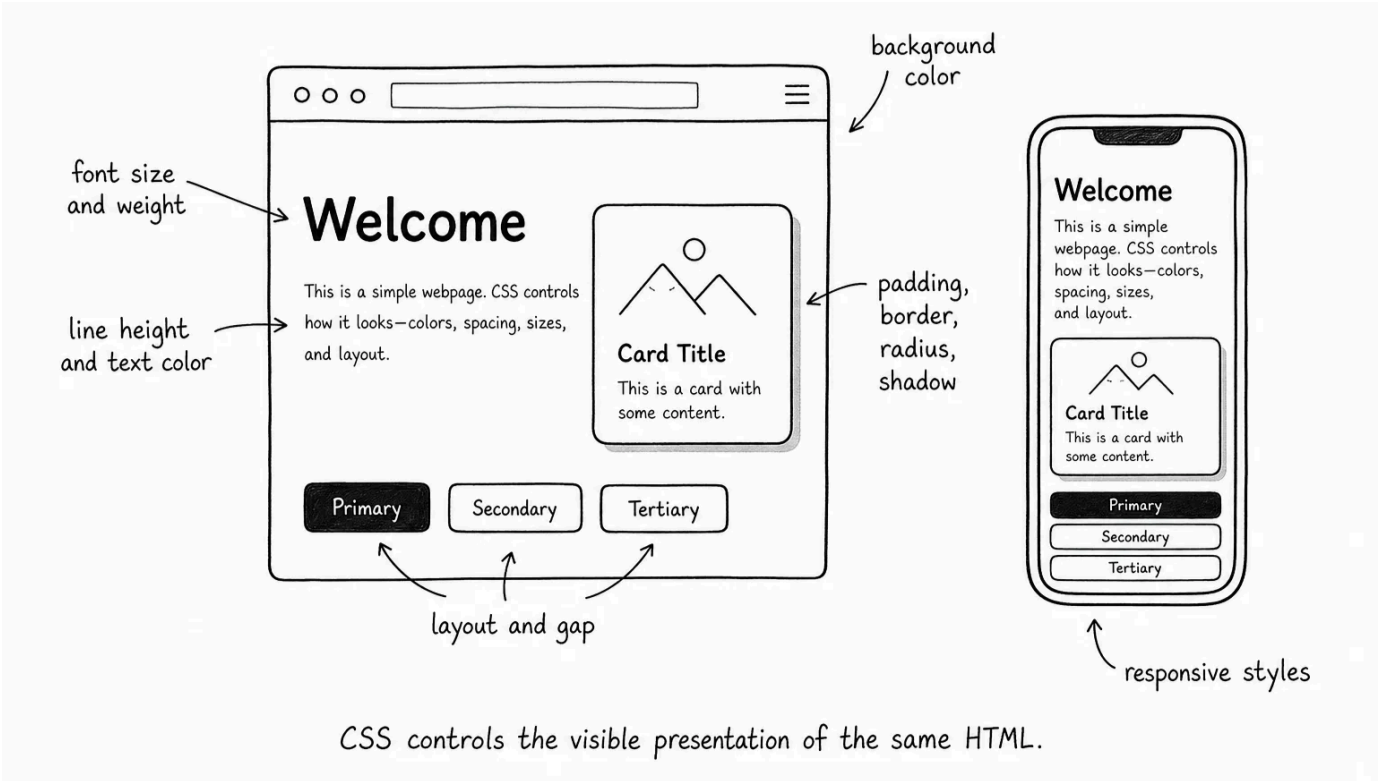
.card {
  padding: 24px;
  margin-bottom: 16px;
  border: 1px solid #d1d5db;
  border-radius: 8px;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.08);
}
```

Layout and responsive CSS control where things sit, and how that changes on different screens:

```
CSS

.actions {
  display: flex;
  gap: 12px;
}

@media (max-width: 600px) {
  h1 {
    font-size: 28px;
  }
}
```



We will cover each of these properties in the future lessons. For now, notice the shape of the language. CSS is a set of instructions about presentation: make this text bigger, give this box space, put these items in a row, change this layout on small screens.

CSS works with the DOM

In the **HTML pack**, you saw that the browser turns HTML into the DOM, a live tree of elements inside the browser. CSS does not style the raw text file directly. It styles elements in that DOM tree.

If your HTML has this paragraph:

html

```
<p class="lead">CSS makes HTML look designed.</p>
```

then CSS can match it by element name:

CSS

```
p {  
  font-size: 18px;  
}
```

CSS can also match the same paragraph by class:

CSS

```
.lead {  
  color: #4b5563;  
}
```

The browser looks through the DOM, finds the elements that match each selector, and applies the declarations from those rules.

That is why HTML attributes like **class** matter so much. A class gives CSS a clear hook:

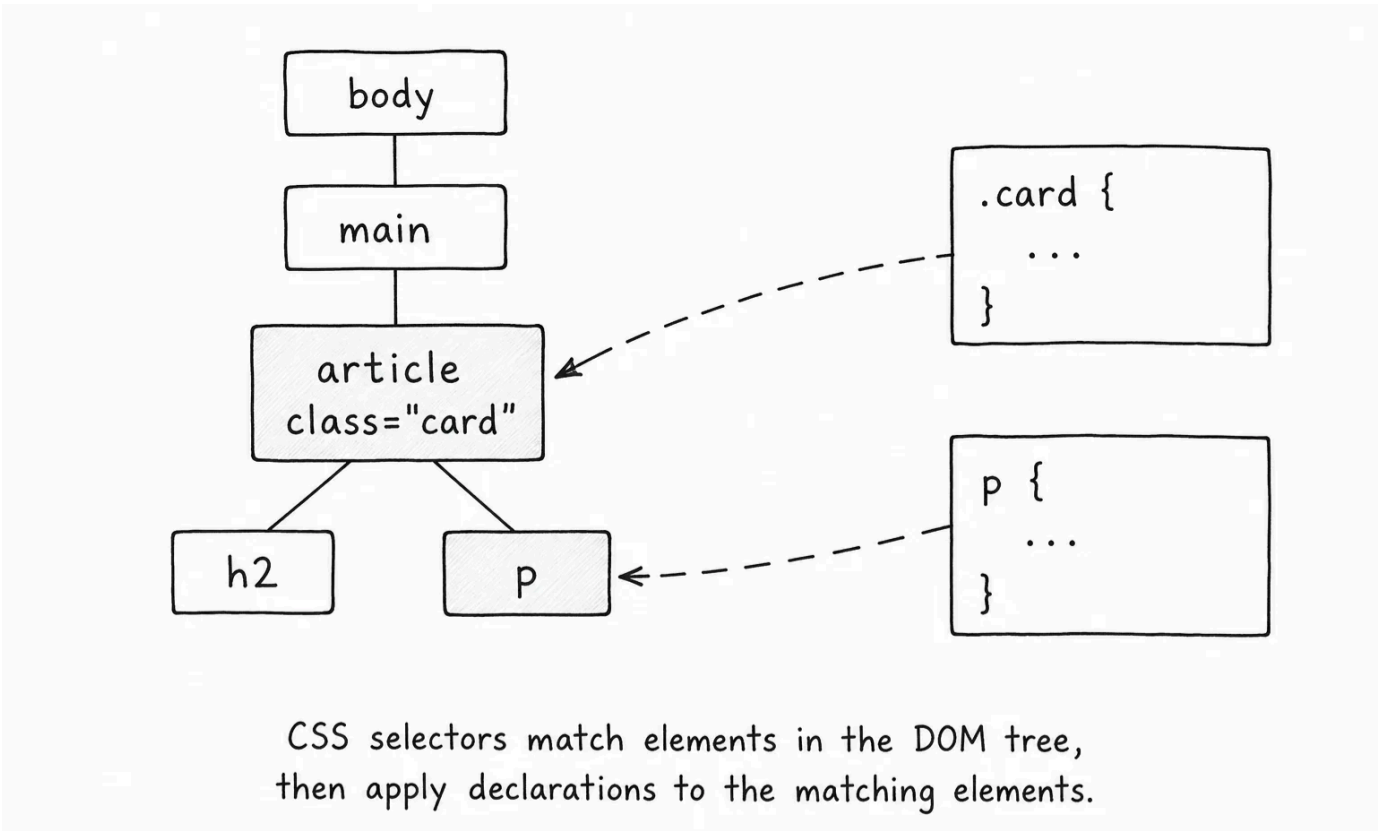
html

```
<article class="card">  
  <h2>Starter plan</h2>  
  <p>Good for small projects.</p>  
</article>
```

CSS

```
.card {  
  background-color: white;  
  padding: 24px;  
  border-radius: 8px;  
}
```

The **article** still means "self-contained content" because of the HTML element. The **card** class says "style this like a card" because of the CSS.

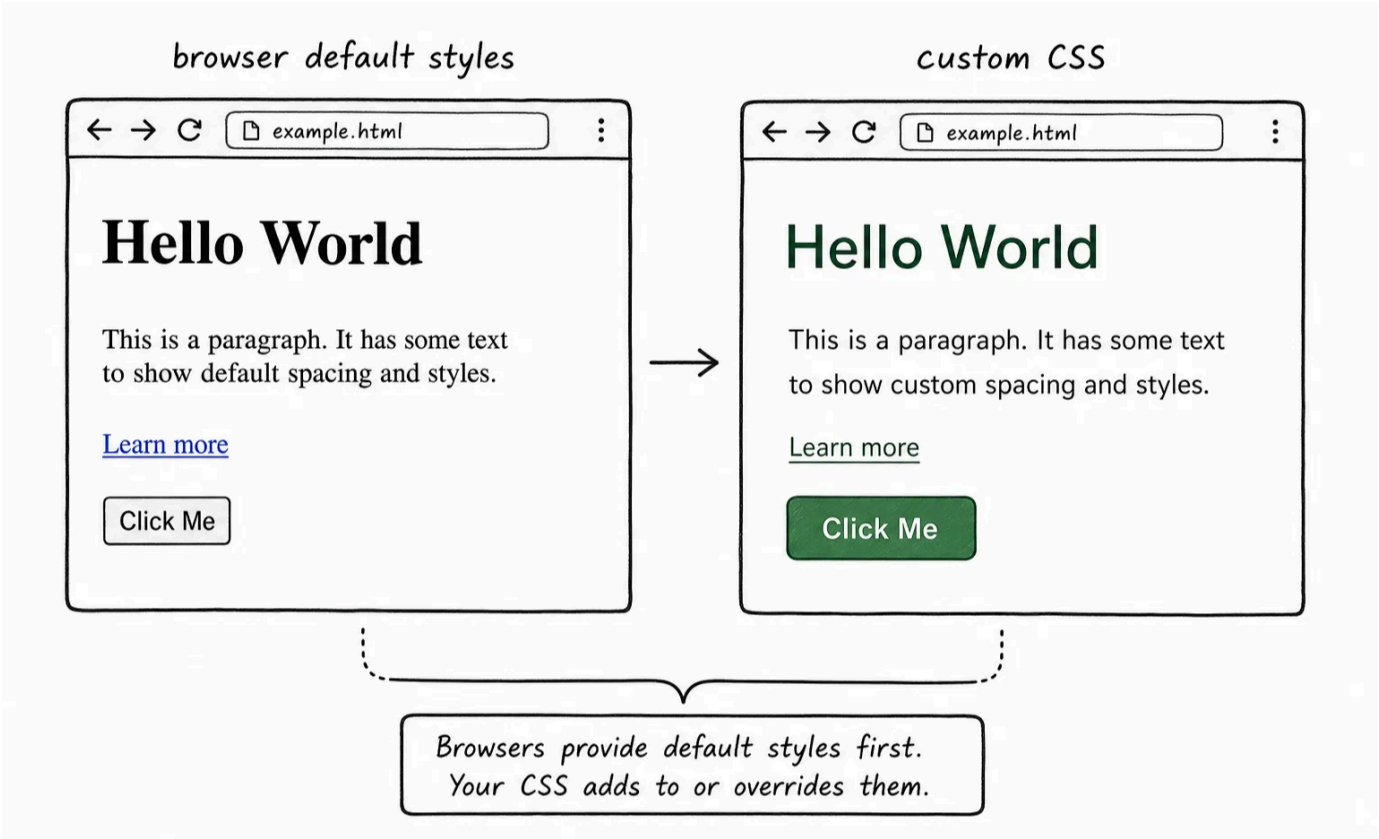


Browsers have default styles

Even before you write any CSS, a page has some style: headings are large and bold, paragraphs have space around them, links are blue and underlined, lists have bullets, and form inputs have borders.

Those styles come from the browser's built-in stylesheet, often called the **user agent stylesheet**. It is the browser's default opinion about how HTML should look.

That is why a plain HTML page is readable without your CSS. The browser is already doing some presentation work, and your CSS adds to or overrides those defaults. You do not need to fight every browser default right away, but you should remember that defaults exist. When something has spacing, color, or a border you did not write, the browser may have supplied it.



💡 **Defaults are useful while learning** - Plain HTML is not supposed to look finished, but it should still be readable. CSS improves the

presentation, but it should not be the only thing making the content understandable.

CSS in real projects

In small projects, you might write one `style.css` file by hand. In larger projects, CSS may appear in different forms:

- A global stylesheet shared by the whole app.
- Component styles that live next to a component.
- Utility classes from a framework like Tailwind CSS.
- CSS variables for shared colors, spacing, and font sizes.
- Styles generated or bundled by a build tool.

The syntax and file structure can change, but the browser still ends up with CSS rules. Those rules still select elements and apply declarations.

This is why CSS basics matter even if you use a framework. A class like `p-4` in Tailwind still means padding. A design token like `--color-primary` still becomes a color value. A component library button still has hover, focus, disabled, spacing, border, and layout rules somewhere underneath.

You do not need to know every CSS architecture on day one. Start with plain CSS, learn how the browser applies it, and the tools will make more sense later.



AI Tutor

Explain why learning plain CSS still matters when a project uses Tailwind, React components, a UI library, or a build tool. Use a button as a simple example, but focus on the mental model: how those tools still become CSS rules that the browser applies to HTML.



Try it

Create a folder with two files: `index.html` and `style.css`.

First, put this in `index.html` without linking the CSS file yet:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>CSS practice</title>
  </head>
  <body>
    <main class="page">
      <h1>CSS practice</h1>
      <p class="lead">This page starts as plain HTML, then CSS changes how it looks.</p>
```

```
<article class="card">
  <h2>Practice card</h2>
  <p>CSS can add space, color, borders, and rounded corners to the same HTML.</p>
  <button>Save changes</button>
</article>
</main>
</body>
</html>
```

Open `index.html` in your browser now. You should see a readable but plain page using the browser's default styles.

Now create `style.css` and add the CSS:

css

```
body {
  font-family: system-ui, sans-serif;
  background-color: #f3f4f6;
  color: #111827;
}

.page {
  max-width: 720px;
  margin: 40px auto;
}

.lead {
  font-size: 20px;
  line-height: 1.5;
  color: #4b5563;
}

.card {
  background-color: white;
  padding: 24px;
  border: 1px solid #d1d5db;
  border-radius: 8px;
}

button {
  background-color: #2563eb;
  color: white;
  padding: 12px 16px;
  border: 0;
  border-radius: 6px;
}
```

Now add this line inside the `head` of `index.html` , just below the `title` :

html

```
<link rel="stylesheet" href="style.css">
```

Reload the page. The HTML did not change, but the page should look very different because the browser is now loading your CSS.

Try these changes one at a time:

- Change the `body` background color.
- Increase the `.card` padding from `24px` to `40px` .
- Change the button `border-radius` from `6px` to `999px` .

- Open DevTools, click the button, and look at the styles applied to it.

The goal is not to memorize the properties yet. The goal is to see the connection: HTML gives you elements, CSS selects those elements, and declarations change how they appear.

< 0 / 8 >

✔ Knew 0 Items

✦ Learnt 0 Items

⏮ Skipped 0 Items

↺ ResetProgress

Test Your Understanding

What does CSS do?

[Click to Reveal the Answer](#)

✔ Already Know that

✦ Didn't Know that

⏮ Skip Question

✓

LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →

Selectors and style rules

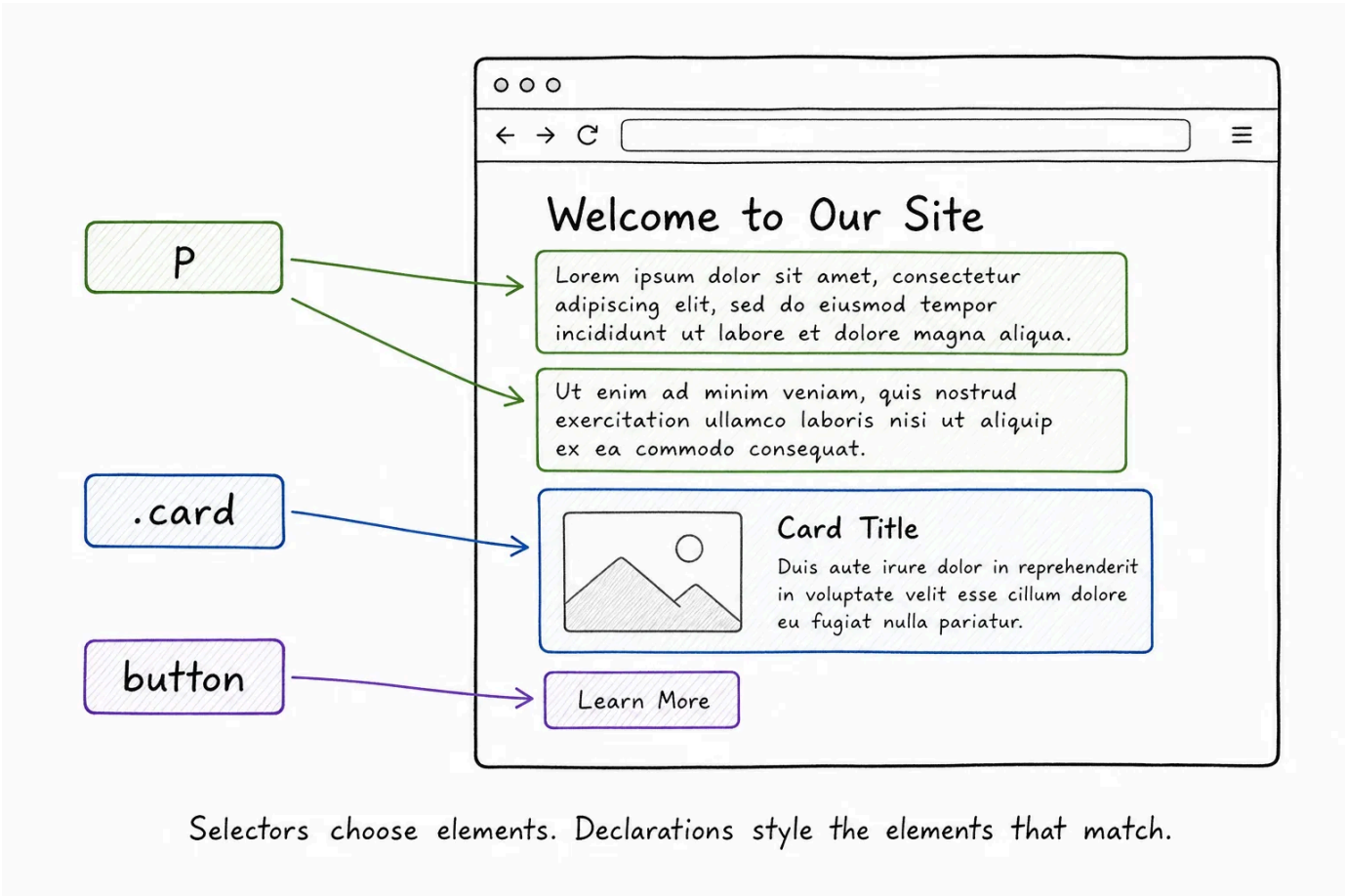
In the previous lesson, you saw that CSS is made of style rules. A rule has a selector, then a set of declarations inside curly braces.

CSS

```
p {
  color: #444;
  font-size: 18px;
}
```

This lesson focuses on the selector part. A selector tells the browser which elements a rule should apply to.

Think of a selector as a way to ask a question about the DOM: "Find every paragraph", "find every element with this class", or "find every link inside this navigation area." The browser checks the page against that selector and applies the declarations to every element that matches.



Element selectors

An element selector targets every element with a given tag name.

CSS

```
p {
  line-height: 1.6;
}
```

That rule applies to every `<p>` element on the page.

Element selectors are useful for broad page defaults:

CSS

```
body {
  font-family: system-ui, sans-serif;
}

h1 {
  font-size: 40px;
}

a {
  color: #047857;
}
```

These rules are saying, "all body text should use this font", "all top-level headings should start at this size", and "all links should use this color."

That is a good use of element selectors. They work well when you really mean every element of that type, or when you are setting a default that more specific component styles can build on later.

They are less useful when you only want to style one particular part of the page. If you style every **button** on the page, then every button gets that style: submit buttons, menu buttons, delete buttons, modal buttons, everything. That is often too broad.

Class selectors

A class selector targets elements by their **class** attribute. In HTML, the class is written without a dot:

html

Previous Lesson

Next Lesson

```
</article>
```

In CSS, you select that class with a dot:

CSS

```
.card {
  padding: 24px;
  border: 1px solid #d1d5db;
  border-radius: 8px;
}
```

The dot means "look for this class name."

Classes are the main styling hook you will use in CSS. They are reusable, readable, and not tied to one specific HTML tag. A **.card** class can go on an

`article` , a `section` , a `div` , or any other element that should look like a card.

html

```
<article class="card">Starter plan</article>
<section class="card">Team plan</section>
<div class="card">Enterprise plan</div>
```

Because the class name is the same, all three elements match `.card` .

 **The dot only belongs in CSS** - Write `class="card"` in HTML, but write `.card` in CSS. Beginners often accidentally write `class=".card"` in HTML. The dot is not part of the class name. It is part of the CSS selector.

Multiple classes on one element

One element can have more than one class. In HTML, separate class names with spaces:

html

```
<button class="button primary-button">Save changes</button>
<button class="button secondary-button">Cancel</button>
```

This lets you split shared styles from variant styles:

css

```
.button {
  padding: 12px 16px;
  border: 0;
  border-radius: 6px;
}

.primary-button {
  background-color: #2563eb;
  color: white;
}

.secondary-button {
  background-color: #e5e7eb;
  color: #111827;
}
```

Both buttons share the `.button` styles. Each button also gets its own variant style.

This pattern shows up everywhere in real interfaces. A card might be both `.card` and `.featured-card` . A message might be both `.alert` and `.error-alert` . A navigation link might be both `.nav-link` and `.active` .

The point is not to create a class for every single property. The point is to give the element meaningful styling hooks.

Combining selectors

You can combine selector parts with no space between them. This means one element must match all the parts.

CSS

```
.card.featured-card {
  background-color: #f9fafb;
}
```

That selector means "an element that has both `card` and `featured-card` in its class list."

Now compare it with a selector that has a space:

CSS

```
.card .featured-card {
  background-color: #f9fafb;
}
```

That selector has a space, so it means "an element with class `featured-card` inside an element with class `card` ." That is a different shape in the DOM.

Small spacing differences can completely change what a selector means.

 **Spaces change selectors** - `.card.featured-card` means one element with both classes. `.card .featured-card` means an element with class `featured-card` somewhere inside an element with class `card` .

ID selectors

An ID selector targets one element by its `id` attribute. In HTML, the ID is written without a `#` :

html

```
<section id="pricing">
  <h2>Pricing</h2>
</section>
```

In CSS, you select it with `#` :

CSS

```
#pricing {  
  background-color: #f9fafb;  
}
```

IDs are meant to be unique. Only one element on a page should use a given ID. That makes IDs useful for things like page anchors:

html

```
<a href="#pricing">Jump to pricing</a>
```

When the user clicks that link, the browser jumps to the element with `id="pricing"` .

For styling, classes are usually the better default because they are reusable, while IDs are unique per page. If you style with IDs everywhere, your CSS becomes harder to reuse and harder to adjust later.

⚠ **Use IDs carefully in CSS** - IDs have their place, especially for unique page anchors and JavaScript hooks. For normal styling, prefer classes unless you truly mean one specific element on the page.

Group selectors

Sometimes several selectors should get the same declarations. Instead of writing the same rule multiple times, separate the selectors with commas:

CSS

```
h1,  
h2,  
h3 {  
  line-height: 1.2;  
}
```

That rule applies to all `h1` , `h2` , and `h3` elements.

The comma matters because it means "match this selector, and also this selector, and also this selector."

Descendant selectors

A descendant selector targets elements inside another element.

CSS

```
nav a {  
  text-decoration: none;  
}
```

This rule applies to links inside a `nav`. It does not apply to every link on the page.

That is useful when the same HTML element should look different in different places:

html

```
<nav class="site-nav">
  <a href="/">Home</a>
  <a href="/pricing">Pricing</a>
</nav>

<main>
  <p>Read the <a href="/docs">full documentation</a>.</p>
</main>
```

css

```
.site-nav a {
  color: #111827;
  text-decoration: none;
}

main a {
  color: #2563eb;
  text-decoration: underline;
}
```

Navigation links and content links are both `<a>` elements, but they live in different parts of the DOM. Selectors let you style them differently.

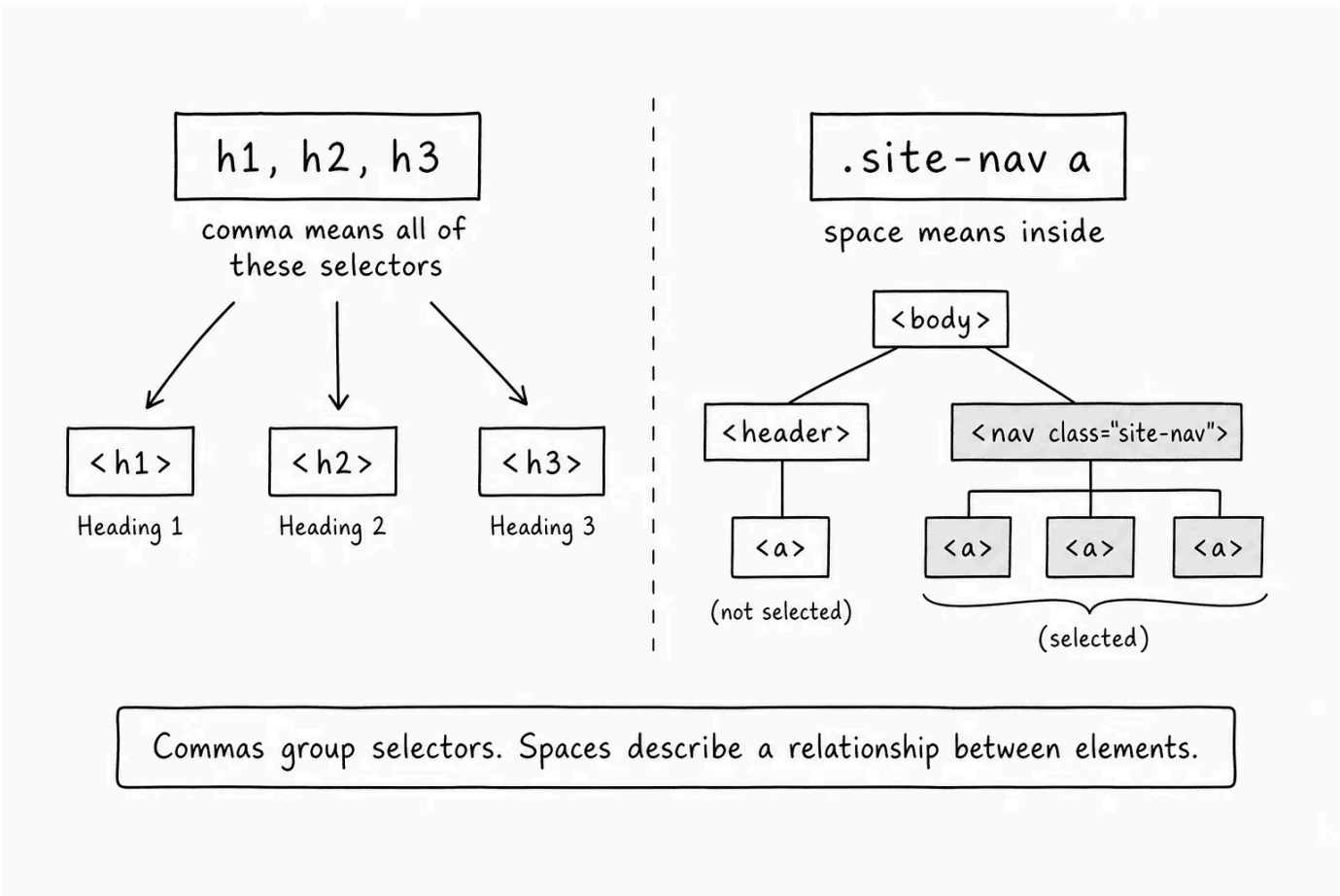
Descendant selectors are practical, but do not make them too long:

css

```
main article section div p a {
  color: red;
}
```

That selector is hard to read and easy to break. If a selector needs to describe the whole page structure to find one thing, a class is probably clearer.

 **Use the nearest useful hook** - If you need to style links in the main navigation, `.site-nav a` is usually clearer than `header nav ul li a`. Use the class that names the region, then select the element inside it.



Name what something is, not what it looks like

Good selectors are not just technically correct. They also make the page easier to understand.

Prefer class names that describe the role of the thing:

html

<p class="error-message">Email is required.</p>

CSS

.error-message {
 color: #b91c1c;
}

Avoid class names that only describe the current look:

html

<p class="red-text">Email is required.</p>

The problem with **red-text** is that the name will become wrong if the design changes from red to orange, or if error messages later need an icon, border, or background. **error-message** describes what the element is in the interface, so the CSS can change without making the HTML feel outdated.

This does not mean every class name has to be perfect. It means you should try to name the thing's job, not only the first style you gave it.

When a selector does not work

When a style does not apply, the first question is simple: did the selector match anything?

When the selector does not match, check these common mistakes:

- The stylesheet is not linked correctly.
- The class name is spelled differently in HTML and CSS.
- The dot is missing from a class selector.
- The `#` is missing from an ID selector.
- A space changed the meaning of the selector.
- The element you expected is not actually in the DOM. Check the Elements tab in DevTools to confirm it is there.

DevTools helps here: inspect the element, look at its tag name, classes, and ID, then compare them with your selector. If the selector does not describe the element you clicked, the rule will not apply.

✦ AI Tutor

Explain how to choose between an element selector, a class selector, an ID selector, and a descendant selector. Use a simple webpage with a header, navigation links, content links, and a card, but focus on the reasoning behind each selector choice.



Try it

For this selector exercise, use a small two-file page: `index.html` and `style.css`.

Start with this HTML:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Selector practice</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <header class="site-header">
      <h1>Selector practice</h1>
      <nav class="site-nav">
        <a href="/">Home</a>
        <a href="#pricing">Pricing</a>
      </nav>
    </header>

    <main>
      <p class="lead">This page uses different selectors for different jobs.</p>

      <section id="pricing" class="card featured-card">
        <h2>Starter plan</h2>
        <p>Good for small projects.</p>
        <a href="/signup" class="button primary-button">Choose plan</a>
      </section>
```



```
<p class="error-message">This is an example error message.</p>
</main>
</body>
</html>
```

Then add these styles:

CSS

```
body {
  font-family: system-ui, sans-serif;
  color: #111827;
  max-width: 720px;
  margin: 40px auto;
}

h1,
h2 {
  line-height: 1.2;
}

.site-nav a {
  color: #111827;
  text-decoration: none;
  margin-right: 12px;
}

.lead {
  font-size: 20px;
  color: #4b5563;
}

.card {
  padding: 24px;
  margin-top: 24px;
  border: 1px solid #d1d5db;
  border-radius: 8px;
}

.featured-card {
  background-color: #f9fafb;
}

.button {
  display: inline-block;
  padding: 10px 14px;
  border-radius: 6px;
  text-decoration: none;
}

.primary-button {
  background-color: #2563eb;
  color: white;
}

.error-message {
  color: #b91c1c;
}
```

Open the page in your browser and try these changes:

- Change `.lead` to `lead` in the CSS and reload. The style stops applying because `lead` looks for a `<lead>` element, not a class.
- Change `.site-nav a` to `a` and reload. Now every link gets the navigation link style.

- Change `.card` to `#pricing` and reload. The styles still apply because both `.card` and `#pricing` happen to match the same element. But `#pricing` would not style a second card if you added one.
- Change `.featured-card` to `.card.featured-card` and reload. The style still applies because the section has both classes.
- Change `.card.featured-card` to `.card .featured-card` and reload. The background stops applying because there is no nested `.featured-card` element inside `.card`.
- Open DevTools, inspect the pricing section, and compare its tag name, ID, and classes with the selectors in your CSS.

The goal is to practice reading a selector as a sentence: `h1, h2` means all `h1` and `h2` elements, `.site-nav a` means links inside `.site-nav`, and `.card.featured-card` means one element with both classes.

< 0 / 12 >

✔ Knew 0 Items

✧ Learnt 0 Items

⏮ Skipped 0 Items

🔄 ResetProgress

Test Your Understanding

What does a CSS selector do?

[Click to Reveal the Answer](#)

✔ Already Know that

✧ Didn't Know that

⏮ Skip Question

✓

LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →

Text, colors, and backgrounds

In the last lesson, you learned how selectors choose which elements a CSS rule applies to. Now you can use those selectors to change the most visible parts of a page: text, colors, and backgrounds.

This lesson uses one small page and styles it in stages, so each rule makes something visible happen in the browser.

Text and color styles answer practical questions:

- Is the page using a readable font?
- Does the heading feel more important than the paragraph?
- Is the message easy to notice?
- Does the text have enough contrast against its background?

Start with the page before styling

To follow along, use two files: `index.html` for the page and `style.css` for the styles. Keep `style.css` empty at first.

Start with the unstyled HTML:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Text and color practice</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <main class="page">
      <h1>Text and color practice</h1>
      <p class="lead">CSS can make text easier to read before you build any complex layout</p>

      <article class="article-preview">
        <h2>Why readable text matters</h2>
        <p>Most pages ask users to read, scan, compare, or decide. Font size, line height, and color make it easier to read, scan, compare, or decide.
        <p class="price"><span class="old-price">$49</span> $29 today</p>
        <a class="read-more" href="/learn-more">Read more</a>
      </article>

      <p class="notice">Email is required before you can continue.</p>
    </main>
  </body>
</html>
```



Open the page in a browser before adding CSS. It still shows the content, but it uses browser defaults for the font, text size, link color, spacing, and page background. That plain version is useful because it gives you a before picture.

As you read the next sections, add the CSS blocks that match this page to `style.css` and reload. Try to notice what changed before reading the explanation.

Add a base style

A simple `body` rule can change the feel of the whole page:

CSS

```
body {
  font-family: system-ui, sans-serif;
  color: #111827;
  background-color: #f9fafb;
}
```

- `font-family` changes the typeface.
- `color` sets the default text color.
- `background-color` sets the page background.

`system-ui` tells the browser to use the operating system's normal interface font. The `sans-serif` part is a fallback. If the browser cannot use the first font, it tries the next one.

You can give the browser a list of fonts:

CSS

```
body {
  font-family: Arial, Helvetica, sans-serif;
}
```

Font names with spaces go in quotes:

CSS

```
body {
  font-family: "Times New Roman", serif;
}
```

You do not need to obsess over fonts while learning CSS.

💡 **Start with readable defaults** - A small `body` rule is often the first useful CSS on a page. Set the font, default text color, and page

background before styling individual pieces.

In the practice page, we will also keep the content from stretching too wide:

CSS

```
.page {  
  max-width: 720px;  
  margin: 40px auto;  
}
```

This is exercise scaffolding, not what the lesson is about. Lessons 5 and 6 teach `max-width` and `margin` properly. For now, these declarations help the sample page look readable while you focus on text and color.

Make headings and paragraphs readable

The most common text properties are `font-size` , `font-weight` , and `line-height` .

CSS

```
h1 {  
  font-size: 40px;  
  font-weight: 700;  
  line-height: 1.1;  
}  
  
h2 {  
  font-size: 28px;  
  line-height: 1.2;  
}  
  
p {  
  font-size: 18px;  
  line-height: 1.6;  
}  
  
.lead {  
  font-size: 20px;  
  color: #4b5563;  
}
```

`font-size` controls how large the text is. Beginners often start with `px` because it is easy to see: `16px` , `18px` , `40px` . You will also see `rem` and `em` later. They scale with font size and user preferences, which matters for accessibility. `px` is fine while you are learning the basics.

`font-weight` controls how thick the text is. Normal text is usually `400` , and bold text is often `700` .

`line-height` controls the space between lines of text. Body text usually needs more line height than a heading, and a paragraph with `line-height: 1.6` , for example, is often easier to read than one where the lines are packed tightly together.

The `.lead` rule makes the intro paragraph slightly larger and softer than the main body text. That tells the reader, "this paragraph introduces the page."

 **Line height helps readability** - If a paragraph feels cramped, try increasing `line-height` before changing lots of other styles. Text can look cleaner just by giving each line more room to breathe.

Set text and background colors

The `color` property sets the foreground text color:

CSS

```
p {
  color: #374151;
}
```

The `background-color` property sets the background behind an element:

CSS

```
body {
  background-color: #f9fafb;
}
```

The names are easy to mix up at first:

- `color` changes the text color.
- `background-color` changes the color behind the element.

You can use both on the same element:

CSS

```
.notice {
  color: #92400e;
  background-color: #fffbeb;
}
```

That kind of pair is useful for messages, notes, warnings, and status labels. The text color and background color should work together so the message is readable.

You can also give a section its own background:

CSS

```
.article-preview {
  background-color: white;
}
```


The white background separates the article from the softer page background. It may still feel unfinished because there is no padding around the content yet, and that is fine for now. Padding belongs to the box model, which is the next lesson.

You can also use image backgrounds:

CSS

```
.hero {  
  background-image: url("hero.jpg");  
}
```

Background images have more details: sizing, repeating, positioning, and contrast with text on top. You will learn those later, so for now focus on solid background colors. They are easier to control, easier to read, and easier to debug.

Align and decorate text when it helps

text-align controls horizontal text alignment inside a block. The common values are **left**, **right**, **center**, and **justify**.

CSS

```
h1 {  
  text-align: center;  
}
```

Try this on the practice page for a moment, then remove it if the page feels better left aligned. Centered text can work for short headings or intro sections, but long centered paragraphs are harder to read.

text-decoration can add or remove lines around text:

CSS

```
.read-more {  
  color: #2563eb;  
  text-decoration: none;  
}  
  
.old-price {  
  color: #6b7280;  
  text-decoration: line-through;  
}
```

Links are underlined by default in browsers. Removing the underline can be fine in navigation or repeated interface links, but make sure the link still clearly looks clickable through color, placement, or another visual cue. The **line-through** value is useful elsewhere, like striking out an old price.

⚠ **Do not hide links completely** - If you remove a link underline and the color looks like normal text, users may not realize it is clickable. Styling should make the page clearer, not more mysterious.

Recognize common color formats

CSS understands several color formats. You do not need to memorize all of them, but you should recognize the common ones.

Named colors are readable in tiny examples:

CSS

```
.error-message {  
  color: red;  
}
```

They are fine for tiny examples, but real projects usually use more precise color values.

Hex colors are more common in real projects:

CSS

```
.error-message {  
  color: #b91c1c;  
}
```

A hex color starts with `#`, followed by numbers and letters. You will see hex values in design tools, codebases, and documentation.

`rgb()` describes a color with red, green, and blue values:

CSS

```
.button {  
  background-color: rgb(37, 99, 235);  
}
```

`hsl()` describes a color with hue, saturation, and lightness:

CSS

```
.button {  
  background-color: hsl(221, 83%, 53%);  
}
```

You will probably use hex most often at first. The useful skill is not memorizing color math. The useful skill is recognizing that all of these are just different ways to give CSS a color.

Some color formats can include transparency too:

CSS

```
.highlight {  
  background-color: rgba(37, 99, 235, 0.12);  
}
```

The **a** in **rgba()** stands for alpha, which is the transparency value. The last value, **0.12**, means the color is mostly transparent: **1** is fully visible, and **0** is fully transparent.

Check contrast before calling it done

Color is not only decoration; it affects whether people can read and use the page.

Imagine a muted style pushed too far:

CSS

```
.too-faint {  
  color: #d1d5db;  
  background-color: #f3f4f6;  
}
```

The text and background are too close in brightness.

This version is easier to read:

CSS

```
.muted {  
  color: #4b5563;  
  background-color: #f3f4f6;  
}
```

The text is darker, so it stands out from the background.

You do not need to become a color expert right away. Start with a simple habit: if you set a text color, check it against the background it sits on.

 **Color should not be the only signal** - Do not rely on color alone to communicate meaning. If an error message is only red, some users may miss the signal. Use clear text too, such as "Email is required."

Try it

You already styled the page in stages. Before moving on, change a few visible details on purpose:

- Change **body** to use **font-family: Georgia, serif;**, then change it back to **system-ui, sans-serif**.

- Add `text-align: center;` to `h1` , reload, then remove it.
- Change the `h1 font-size` from `40px` to `32px` .
- Change paragraph `line-height` from `1.6` to `1.1` , then back to `1.6` . Notice how readability changes.
- Change the `.lead` color to `#9ca3af` . Decide whether it still has enough contrast.
- Change the `.notice` background color to `#fee2e2` and the text color to `#991b1b` .
- Remove the `background-color` from `.article-preview` and reload.

Then add a second message under the notice:

html

<p class="success-message">Profile saved successfully.</p>

Write a new CSS rule for `.success-message` . Give it a text color and a background color that feel different from the warning notice, while still being readable.

The article and notice may still look cramped because their backgrounds have no inner spacing yet. That is not a mistake; the next lesson explains how padding gives a background room around its text.

The goal is to connect CSS properties to visible changes. When you write a rule, reload the page and ask: what changed, and did it make the page easier to read?

< 0 / 10 >

Knew 0 Items

Learnt 0 Items

Skipped 0 Items

ResetProgress

Test Your Understanding

Why open the page before adding CSS?

[Click to Reveal the Answer](#)

- ✔ Already Know that
- ✳ Didn't Know that
- ▶ Skip Question



LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →

The box model

In the last lesson, you added background colors to an article and a notice. The colors worked, but the boxes probably felt cramped because the text touched the edges. That is what this lesson fixes.

Every visible element on a page is drawn as a rectangular box. The box model is the way CSS describes the parts of that box: content, padding, border, and margin. Once you understand those four parts, spacing starts to feel much less random.

Start with a cramped box

If you still have the page from the last lesson, you can keep using it. If not, set up `index.html` and `style.css` for this box model exercise.

Use this HTML as the starting point:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Box model practice</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <main class="page">
      <article class="article-preview">
        <h1>Readable boxes need space</h1>
        <p>Background colors are useful, but they need room around the text to feel comfortable</p>
      </article>

      <p class="notice">Email is required before you can continue.</p>
      <p class="success-message">Profile saved successfully.</p>
    </main>
  </body>
</html>
```

Give the page a little color, but do not add any spacing yet:

CSS

```
body {
  font-family: system-ui, sans-serif;
  color: #111827;
  background-color: #f9fafb;
}

.page {
  max-width: 720px;
  margin: 40px auto;
}
```




```
.article-preview {  
  background-color: white;  
}  
  
.notice {  
  color: #92400e;  
  background-color: #fffbeb;  
}  
  
.success-message {  
  color: #166534;  
  background-color: #f0fdf4;  
}
```

Open the page. The backgrounds show up, but the text sits right against the edges. That is not a color problem; it is a box model problem.

Every element has a box

Even when you do not draw a border, the browser still treats each element as a box.

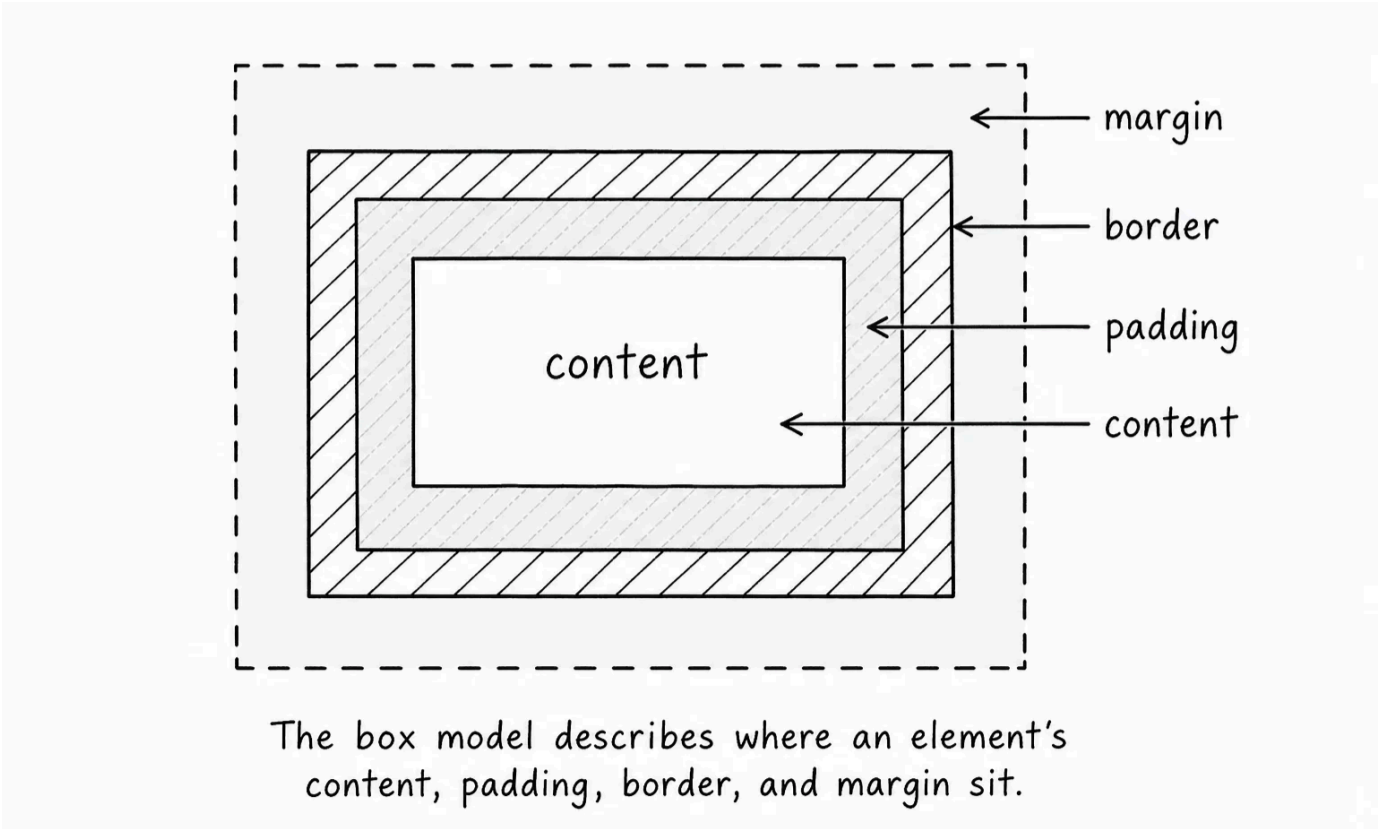
That box has four main parts:

- Content: the actual text, image, button label, or other content.

◀ Previous Lesson

Next Lesson ▶

- Border: a line around the padding and content.
- Margin: space outside the element, between this box and other boxes.



The important idea is simple: padding creates room inside the element, while margin creates room outside the element.

Add padding inside the box

Padding gives the content room inside the element. Add padding to the article and messages:

CSS

```
.article-preview {
  background-color: white;
  padding: 24px;
}

.notice {
  color: #92400e;
  background-color: #fffbeb;
  padding: 16px;
}

.success-message {
  color: #166534;
  background-color: #f0fdf4;
  padding: 16px;
}
```

Reload the page and the colored areas should feel less cramped because the text now has space around it.

There is a detail worth noticing: the element's background color fills the padded area too. That is why adding padding makes the colored notice feel larger, not just roomier. It grows the colored region itself, while margin creates space outside the colored region.

The `padding` property is a shorthand property. One value applies to all sides:

CSS

```
padding: 16px;
```

Two values are also common:

CSS

```
padding: 12px 16px;
```

The first value is top and bottom. The second value is left and right.

You can also target one side with properties like `padding-left` , `padding-right` , `padding-top` , or `padding-bottom` .

You do not need to memorize every shorthand form today. The main habit is this: use padding when the inside of a box needs more room.

Add a border around the box

A border is a line around the element. It can give a box a clearer edge, or it can add emphasis to one side.

The warning notice from the above HTML still relies mostly on color to signal that something needs attention. A left border can emphasize it without

changing the whole layout:

CSS

```
.notice {
  color: #92400e;
  background-color: #ffffeb;
  padding: 16px;
  border-left: 4px solid #f59e0b;
}
```

That colored stripe gives the notice a stronger warning shape.

You can also add a softer border around the article:

CSS

```
.article-preview {
  background-color: white;
  padding: 24px;
  border: 1px solid #e5e7eb;
}
```

The **border** shorthand usually includes three parts:

- Width: **1px**
- Style: **solid**
- Color: **#e5e7eb**

The **border** property targets all sides of the element. The side-specific version works the same way, but only for one side:

CSS

```
.notice {
  border-left: 4px solid #f59e0b;
}
```

Borders are useful when you want a box to have a clear edge. Later, you will learn related visual details like **border-radius** and **box-shadow** .

Add margin outside the box

Margin creates space outside an element.

Right now, the article and messages may sit too close together. Add margin between them:

CSS

```
.article-preview {
  background-color: white;
  padding: 24px;
  border: 1px solid #e5e7eb;
  margin-bottom: 24px;
}
```

```
}

.notice {
  color: #92400e;
  background-color: #ffffbe;
  padding: 16px;
  border-left: 4px solid #f59e0b;
  margin-bottom: 12px;
}
```

Reload the page and the boxes should now have space between them. Margin does not belong to the colored background because it sits outside the element.

That is the main difference to remember:

- Use padding when the content needs space **inside** its own background.
- Use margin when the whole element needs space **away from other elements**.

Margin has shorthand patterns too:

CSS

```
margin: 16px;
margin: 12px 16px;
margin-bottom: 24px;
```

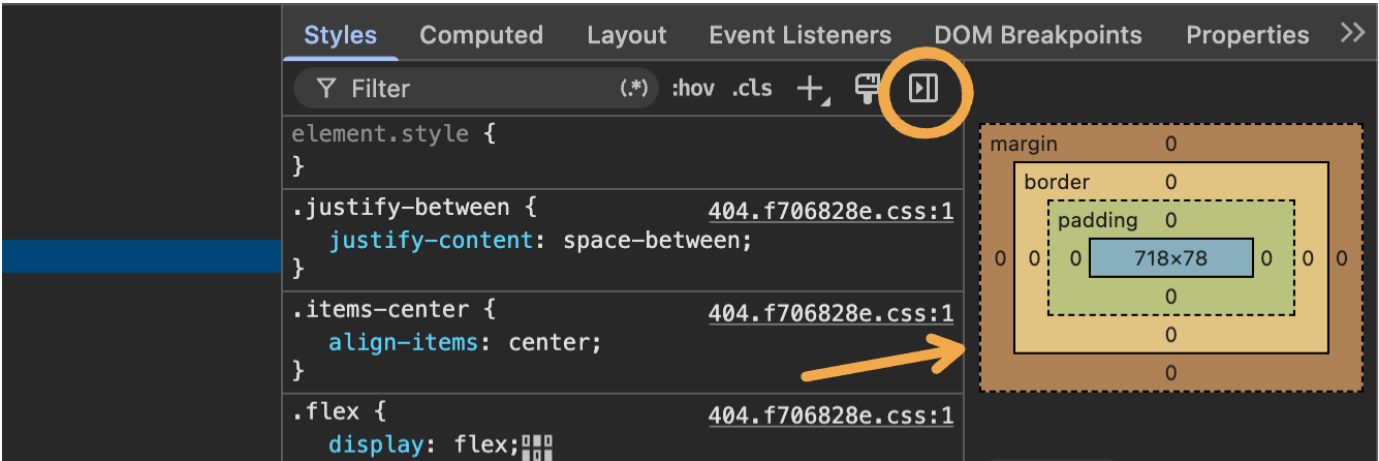
The pattern is similar to padding: one value applies to all sides, two values mean vertical then horizontal, and side-specific properties target one side.

 **Do not use margin for inner space** - If the text is touching the edge of a colored box, use `padding` , not `margin` . Margin moves the whole box away from other boxes, but it does not create space inside the background.

Inspect the box in DevTools

The box model is easier to understand when you inspect it in the browser.

Open DevTools, inspect the `.notice` element, and look for the box model diagram in the Styles or Computed panel. Browser DevTools vary a little, but the idea is the same: you can see the element's content, padding, border, and margin.



You will usually see a diagram with nested rectangles: content in the middle, then padding, border, and margin. Hover each layer if your browser supports it. The matching part of the element will highlight on the page.

Try changing the padding value in DevTools. The browser will update the page immediately.

Then try changing the margin value and notice that a different part of the page moves.

This is one of the best ways to debug spacing.

In the next lesson, you will see a related rule called **box-sizing: border-box**. It makes more sense once you start setting widths.

Try it

You have already made the colored boxes less cramped. Now use DevTools and push the box model a little further:

- Set **.notice** padding to **0**, reload, then set it back to **16px**.
- Change **.article-preview** padding from **24px** to **12px**, then to **32px**.
- Add **border-left: 4px solid #22c55e;** to **.success-message**.
- Change **.notice** **margin-bottom** from **12px** to **32px**.
- Remove the **border** from **.article-preview**, then add it back.
- Inspect **.notice** in DevTools and find its content, padding, border, and margin.

The goal is not to make the page beautiful yet. The goal is to know which part of the box you are changing.

< 0 / 8 >

🕒 Knew

0 Items

🌟 Learnt

0 Items

⏮ Skipped

0 Items

🔄 ResetProgress

Test Your Understanding

What is the CSS box model?

[Click to Reveal the Answer](#)

- ✔ Already Know that
- ✖ Didn't Know that
- ⏩ Skip Question



LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →

Sizing, borders, and rounded corners

In the last lesson, you learned that every element is a box. You added padding inside boxes, borders around boxes, and margin outside boxes. Now you can decide how big a box should be.

Sizing properties help answer practical questions:

- Should this card be a specific width?
- Should it be allowed to shrink on smaller screens?
- Should it have a minimum height?
- Should the box feel sharp, rounded, flat, or lifted?

This lesson turns the box model into practical card styling.

Start with a simple card

Use a small plan card as the practice surface. Put this markup in `index.html`:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Sizing practice</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <main class="page">
      <article class="plan-card">
        <p class="eyebrow">Starter plan</p>
        <h1>Build better habits</h1>
        <p class="summary">A simple plan for learning CSS through small, visible changes</p>
        <p class="price">$12 per month</p>
        <a class="card-action" href="/start">Start learning</a>
      </article>
    </main>
  </body>
</html>
```

Then give it a basic stylesheet:

css

```
body {
  font-family: system-ui, sans-serif;
  color: #111827;
  background-color: #f9fafb;
}
```



```
.page {
  max-width: 720px;
  margin: 40px auto;
}

.plan-card {
  background-color: white;
  padding: 24px;
  border: 1px solid #e5e7eb;
}

.eyebrow {
  color: #2563eb;
  font-weight: 700;
}

.summary {
  color: #4b5563;
  line-height: 1.6;
}

.price {
  font-size: 24px;
  font-weight: 700;
}

.card-action {
  color: #2563eb;
  text-decoration: none;
  font-weight: 700;
}
```

available space. That is not wrong because block elements often take the available width, but many UI pieces, like cards and forms, need a more deliberate size.

Set a width

The `width` property sets how wide an element should be.

Set `width: 320px` on the card:

css

```
.plan-card {
  background-color: white;
  padding: 24px;
  border: 1px solid #e5e7eb;
  width: 320px;
}
```

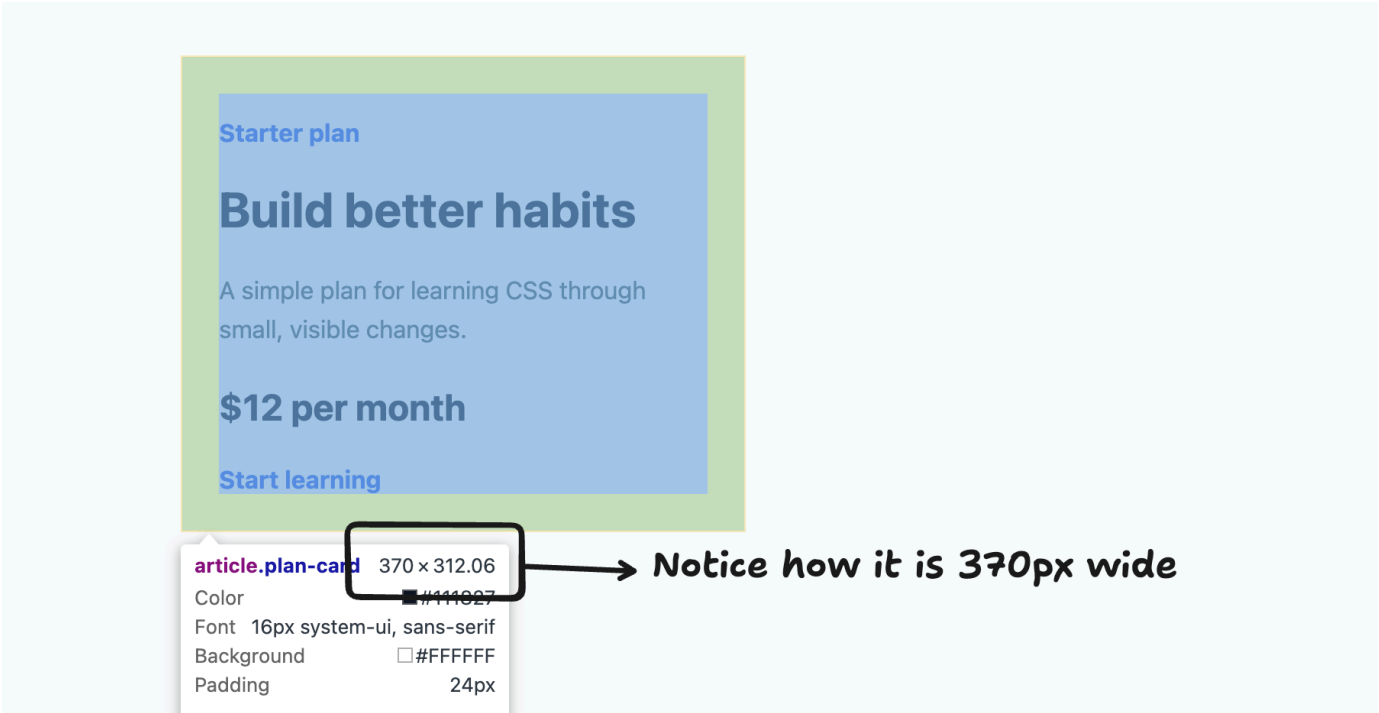
Reload the page. The card should now look narrower, but there is a box model surprise here.

By default, `width: 320px` sets the width of the **content area**. Padding and border are added on top of that. So the visible card can be wider than 320px.

ⓘ Width uses the content box by default - Width is set on the content area by default which means that even though we define the width to

be **320px**, the actual width of the element will be more than that because padding and borders are added on top of that width.

Notice how the actual width in browser is **370px** instead of 320px.

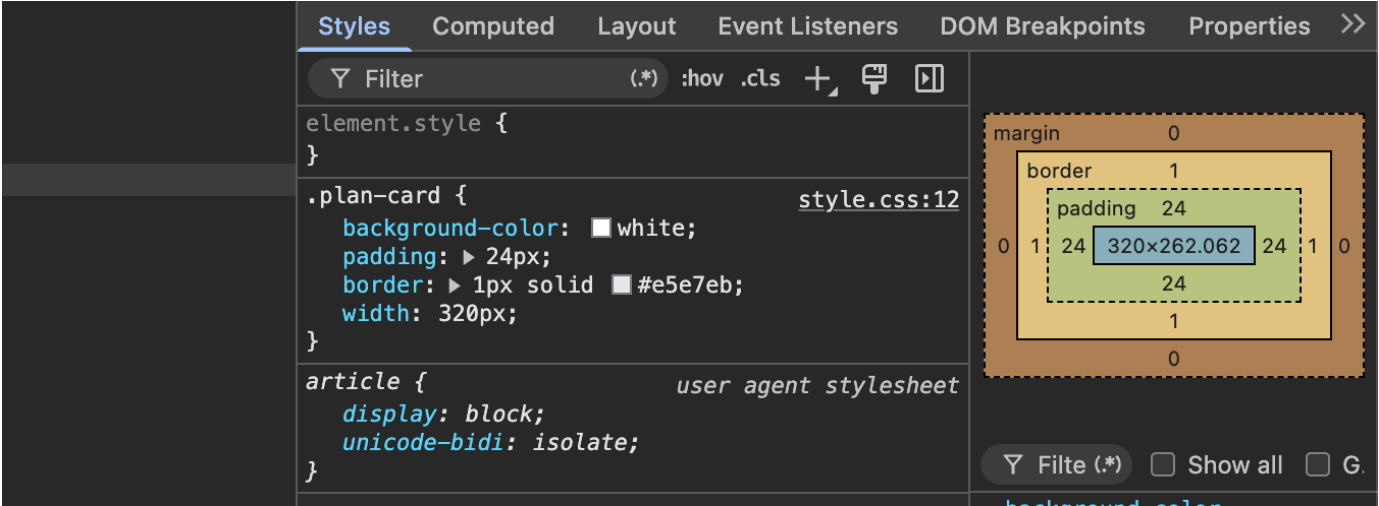


If the card has **24px** of padding on both sides and a **1px** border on both sides, the visible width becomes:

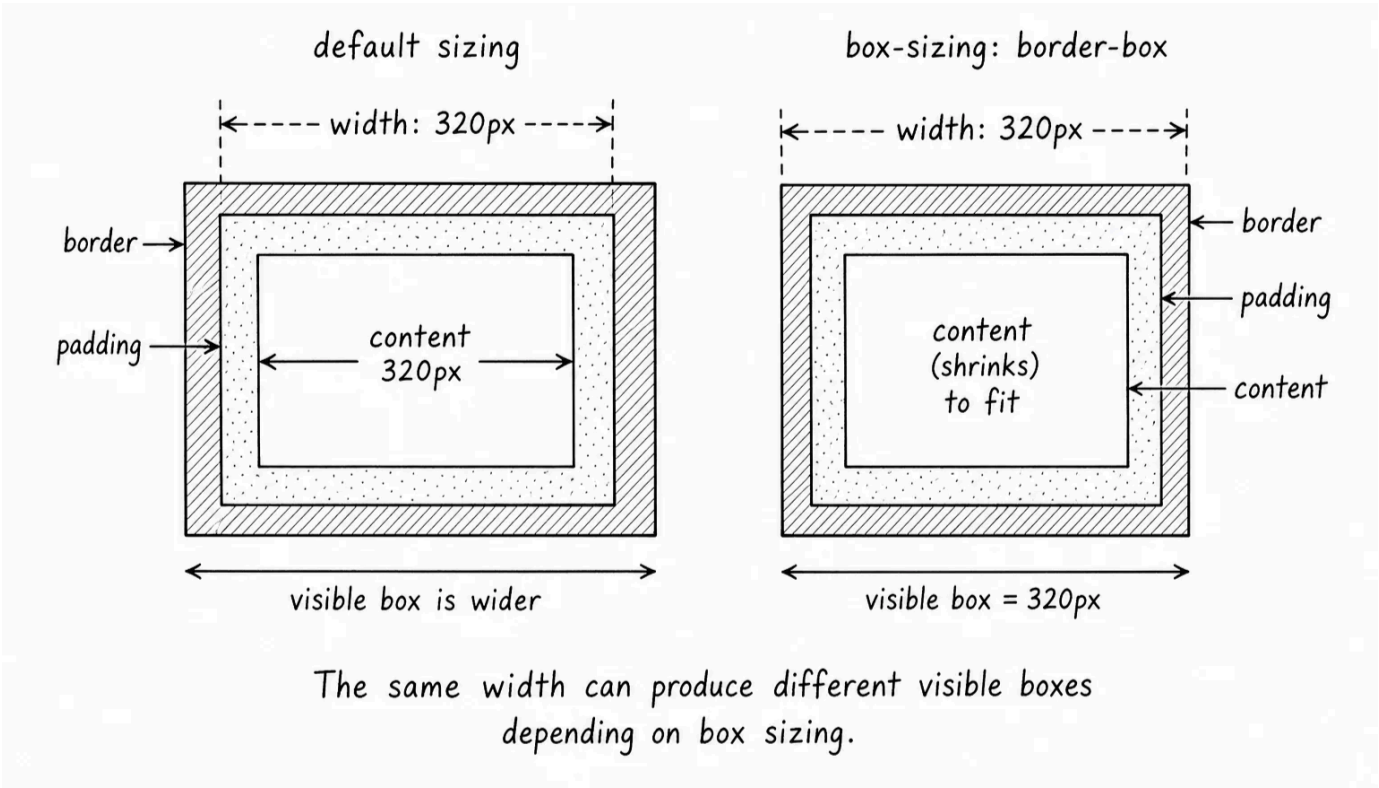
```
text

320px content
+ 24px left padding
+ 24px right padding
+ 1px left border
+ 1px right border
= 370px visible box
```

Inspect the card in DevTools if you want to confirm the numbers. The box model diagram should show the content width, padding, and border separately.



This is where **box-sizing** matters because it controls how the browser applies width. If you want the set width to include the padding and border, use **box-sizing: border-box** .



Use border-box sizing

Most modern projects make sizing easier with this rule:

```
CSS

* {
  box-sizing: border-box;
}
```

The `*` selector matches every element. `box-sizing: border-box` tells the browser to include content, padding, and border inside the set width.

The default behavior has a name too: `box-sizing: content-box`. With `content-box`, width applies to the content only, and padding and border are added outside it. `border-box` flips that: padding and border are pulled inside the width.

Add it near the top of your CSS:

```
CSS

* {
  box-sizing: border-box;
}

body {
  font-family: system-ui, sans-serif;
  color: #111827;
  background-color: #f9fafb;
}
```

Reload the page and inspect `.plan-card` in DevTools. With `border-box`, the visible card width is easier to reason about: the content, padding, and border fit inside the `320px` width.

💡 Use border-box in real projects - `box-sizing: border-box` is a practical default because it makes width and height include padding

and border, which usually matches how people expect boxes to behave.

Prefer max-width for flexible boxes

A fixed `width` can be useful for a small demo, but it can be too rigid in real pages. A card that is exactly `320px` wide may be fine on one screen and awkward on another. A width of `800px`, for example, might look fine on a desktop but break on a smaller screen.

Often, you want the box to be flexible up to a limit. This is where a combination of `width` and `max-width` comes in.

You can also use percentages for `width`: `100%` means the full available width, `80%` means 80% of the available width, and `50%` means half.

If you want the card to work on smaller and larger screens, set `width: 100%` so it can use the available space, then constrain it with `max-width: 420px`:

CSS

```
.plan-card {
  background-color: white;
  padding: 24px;
  border: 1px solid #e5e7eb;
  width: 100%;
  max-width: 420px;
}
```

`width: 100%` lets the card use the available space, and `max-width: 420px` stops it from becoming wider than `420px`. On narrow screens, the card can shrink with the page. On wider screens, it stops at `420px`.

For a block element like `<article>`, `width: 100%` is often technically redundant because block elements already use the available width. You will still see this pattern in real projects because it makes the intent explicit and also works well when the element is not a normal block.

This is a common pattern for cards, forms, articles, and centered content.

You may also see `min-width`, which sets a minimum width. Use it carefully, because a minimum width can make layouts harder on small screens.

Avoid fixed height for text

The `height` property sets an exact height.

That can be useful for things like icons, avatars, thumbnails, and fixed-size controls. But fixed heights can cause trouble for text-heavy boxes.

Avoid this for normal cards:

CSS

```
.plan-card {  
  height: 180px;  
}
```

If the text gets longer, it may overflow or feel squeezed.

Use `min-height` when you want a box to have at least a certain height while still allowing it to grow:

CSS

```
.plan-card {  
  min-height: 260px;  
}
```

`min-height` says, "be at least this tall, but grow if the content needs more room."

You may also see `max-height`, which sets an upper limit. Once content is taller than that limit, overflow becomes part of the problem, which you will learn later.

 **Avoid fixed heights for normal text** - If a box contains text, be careful with `height` because text can wrap, translations can be longer, and users may increase font size. `min-height` is usually safer.

Round the corners

The `border-radius` property rounds the corners of a box.

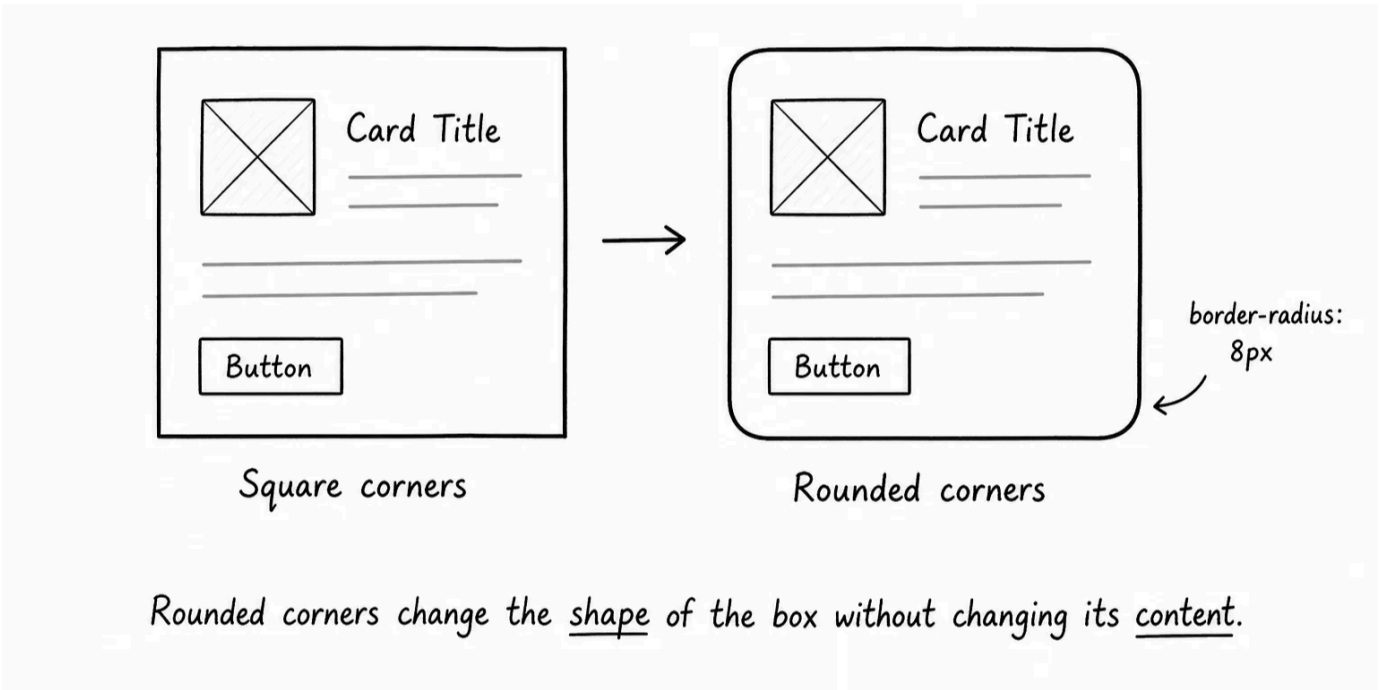
Add the radius to the card:

CSS

```
.plan-card {  
  background-color: white;  
  padding: 24px;  
  border: 1px solid #e5e7eb;  
  width: 100%;  
  max-width: 420px;  
  min-height: 260px;  
  border-radius: 8px;  
}
```

Small radius values make a box feel slightly softer. Large values can create pills or circles, depending on the element shape.

For beginner UI work, values like `4px`, `6px`, `8px`, or `12px` are usually enough.



Add a light shadow

The `box-shadow` property draws a shadow around a box.

Add a subtle shadow to the card:

CSS

```
.plan-card {
  background-color: white;
  padding: 24px;
  border: 1px solid #e5e7eb;
  width: 100%;
  max-width: 420px;
  min-height: 260px;
  border-radius: 8px;
  box-shadow: 0 8px 24px rgba(15, 23, 42, 0.08);
}
```

The shadow value has four parts:

- `0` : horizontal offset
- `8px` : vertical offset
- `24px` : blur
- `rgba(15, 23, 42, 0.08)` : shadow color

Positive vertical offset pushes the shadow down. That is why cards usually have shadow below them: it suggests the card is lifted above the page.

Shadows usually use `rgba()` so the alpha value keeps them subtle. A solid `rgb()` or hex shadow can feel too heavy.

You do not need to memorize shadow math. The practical habit is to keep shadows subtle. A heavy shadow can make a simple UI feel noisy.

Borders, radius, and shadows are visual styling. They should support the content, not compete with it.

✦ AI Tutor

Explain how subtle and heavy box shadows change the feeling of a card. Use the card from this lesson as the example, and focus on when a shadow supports the content versus when it becomes distracting.

>

Try it

The card now has sizing, border, radius, and shadow. Try the following:

- Change `.plan-card` from `width: 100%; max-width: 420px;` back to `width: 320px;`. Inspect the visible width with and without `box-sizing: border-box`.
- Change `max-width` from `420px` to `520px`.
- Change `min-height` from `260px` to `160px`, then add more text to the summary.
- Change the border color from `#e5e7eb` to `#9ca3af`, then change it back.
- Change `border-radius` from `8px` to `0`, then to `16px`.
- Remove `box-shadow`, then add it back.
- Make the shadow stronger by changing `0.08` to `0.2`, then decide which version feels cleaner.

For an extra challenge, add a second card under the first one:

html

```
<article class="plan-card secondary-card">
  <p class="eyebrow">Team plan</p>
  <h1>Practice together</h1>
  <p class="summary">A shared plan for reviewing lessons, comparing solutions, and build
  <p class="price">$29 per month</p>
  <a class="card-action" href="/team">Start as a team</a>
</article>
```

Then write a `.secondary-card` rule that changes only the border color or shadow.

This uses the multi-class pattern from Lesson 2. The card gets all the `.plan-card` styles and adds your `.secondary-card` variant on top.

The goal is to connect sizing and visual polish to the box model. Width sets the box size, `box-sizing` changes how that size is calculated, and radius or shadow changes how the box feels.

Test Your Understanding

What does width do?

[Click to Reveal the Answer](#)

✔ Already Know that

✖ Didn't Know that

⏩ Skip Question



LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →

Spacing with margin, padding, and gap

In the last few lessons, you used padding, margin, width, borders, radius, and shadows. You can now make individual boxes look better, so the next step is spacing the whole page.

Good spacing starts with one practical question: where should the space belong?

- Use `padding` for space inside a box.
- Use `margin` for space outside a box.
- Use `gap` for space between items in a group.

This lesson turns spacing into a repeatable habit.

Start with a cramped page

Use a small lesson page as the practice surface. Put this markup in

`index.html` :

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Spacing practice</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <main class="page">
      <section class="intro">
        <p class="eyebrow">CSS practice</p>
        <h1>Plan your next lesson</h1>
        <p class="summary">Spacing helps related things feel connected and separate thir</p>
      </section>

      <section class="lesson-list">
        <article class="lesson-card">
          <h2>Text and color</h2>
          <p>Review font size, line height, text color, and background color.</p>
        </article>

        <article class="lesson-card">
          <h2>The box model</h2>
          <p>Practice content, padding, border, and margin in DevTools.</p>
        </article>

        <article class="lesson-card">
          <h2>Sizing cards</h2>
          <p>Compare width, max-width, min-height, radius, and shadow.</p>
        </article>
      </section>
    </main>
  </body>
</html>
```




```
<div class="actions">
  <a class="primary-action" href="/start">Start practice</a>
  <a class="secondary-action" href="/review">Review notes</a>
</div>
</main>
</body>
</html>
```

Then add a basic stylesheet:

CSS

```
* {
  box-sizing: border-box;
}

body {
  font-family: system-ui, sans-serif;
  color: #111827;
  background-color: #f9fafb;
}
```

◀ Previous Lesson

Next Lesson ▶

```
.intro,
.lesson-card {
  background-color: white;
  border: 1px solid #e5e7eb;
}

.eyebrow {
  color: #2563eb;
  font-weight: 700;
}

.summary,
.lesson-card p {
  color: #4b5563;
  line-height: 1.6;
}
```

Open the page. The boxes exist, but the content is crowded and the page does not have a clear rhythm yet. That is what spacing fixes.

Choose a small spacing scale

Before adding random values, choose a small set of spacing numbers.

For this lesson, reuse this small spacing scale:

text

4px, 8px, 12px, 16px, 24px, 32px, 48px

You do not need a perfect design system yet. The point is to reuse a few values instead of inventing a new number for every property.

For example, you might use:

- **8px** for small gaps.
- **16px** for comfortable inner spacing.
- **24px** or **32px** for larger section spacing.



AI Tutor

What is a design system in UI design?



Consistent spacing makes a page feel calmer because the distances repeat.



Repeat spacing values - When spacing feels messy, the problem is often too many unrelated numbers. Reusing a small spacing scale usually makes the page feel more intentional.

Add padding inside boxes

Padding creates room inside a box. Let's add padding to the intro and cards:

CSS



```
.intro,
.lesson-card {
  background-color: white;
  border: 1px solid #e5e7eb;
  padding: 24px;
}
```

Reload the page and the text should no longer touch the box edges.

This is the same rule from the box model lesson: if the inside of a box feels cramped, use padding.

Add margin between sections

Margin creates space outside an element.

The intro, lesson list, and actions are separate parts of the page, so they need space between them.

Add margin below the intro and lesson list:

CSS



```
.intro {
  margin-bottom: 32px;
}

.lesson-list {
  margin-bottom: 24px;
}
```

Reload the page and the sections should feel more separated.

Use margin when the whole element needs distance from another element.
Do not use margin to fix text touching the inside of a box.

That distinction matters because the two properties solve different spacing problems:

- Padding fixes cramped content inside a background or border.
- Margin separates one element from another.

 **Do not use empty HTML for spacing** - Avoid adding empty `<p>` , `<br>` , or extra blank elements just to push things apart. Spacing belongs in CSS, not in fake HTML content.

Use gap between grouped items

`gap` creates space between items inside a group.

For `gap` to work, the group needs a layout mode like flex or grid. You will learn both properly later. For now, use one small flex row so you can see what `gap` does.

Add this to the action row:

CSS

```
.actions {
  display: flex;
  gap: 12px;
}
```

Now style the links enough to see the row clearly:

CSS

```
.primary-action,
.secondary-action {
  padding: 10px 14px;
  border-radius: 6px;
  text-decoration: none;
  font-weight: 700;
}

.primary-action {
  color: white;
  background-color: #2563eb;
}

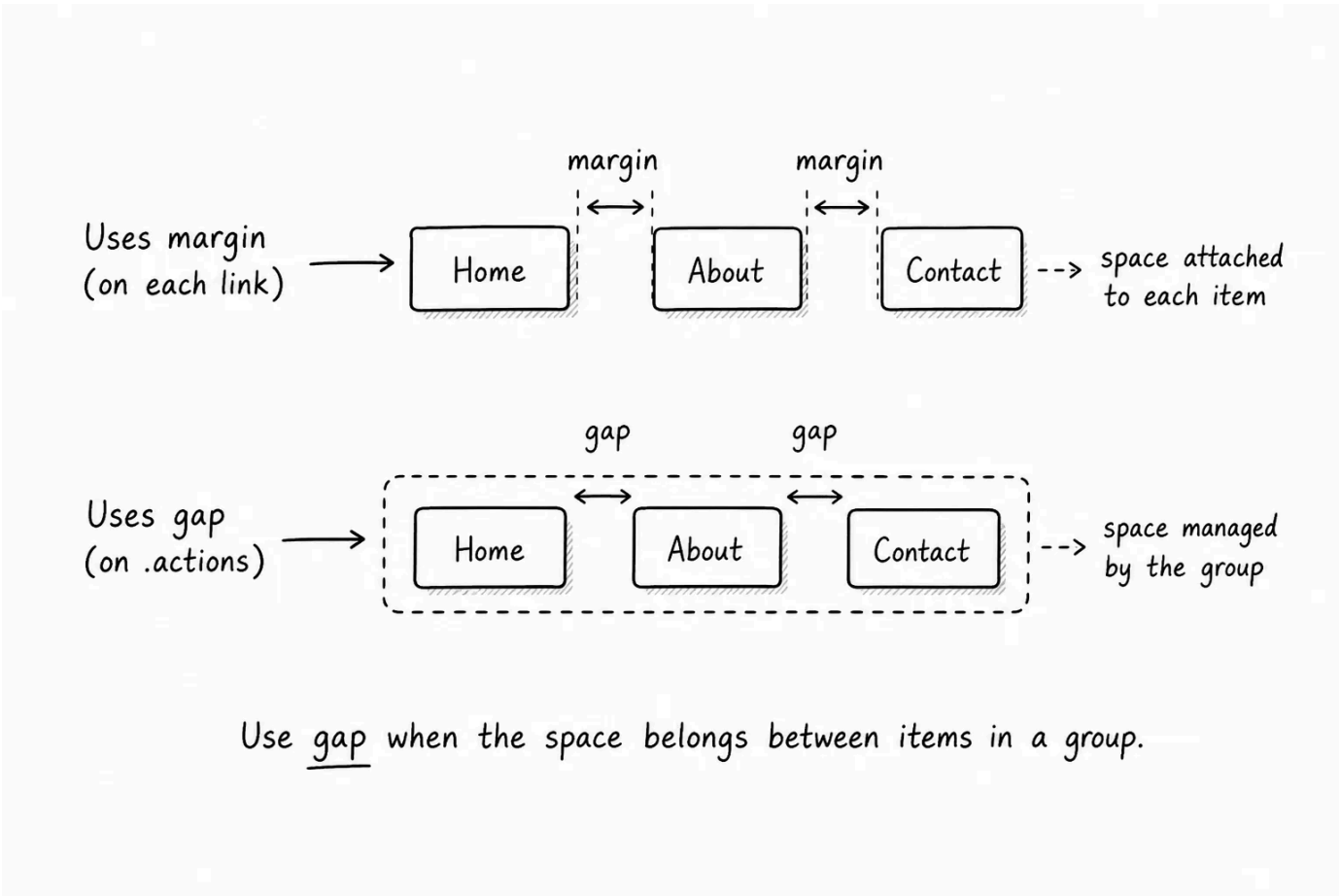
.secondary-action {
  color: #2563eb;
  border: 1px solid #bfdbfe;
}
```

Reload the page and the action links should sit in a row with space between them. The useful part is the parent-owned gap:

CSS

```
.actions {  
  gap: 12px;  
}
```

You do not need to add margin to each link. The parent group owns the space between its children.



Margin versus gap - `margin` creates space around an individual element, `gap` creates space *between* the child elements of an element.

Group the cards with gap

The lesson cards are also a group. Instead of adding `margin-bottom` to every card, let the parent manage the space.

Add the gap to the card list:

CSS

```
.lesson-list {  
  display: flex;  
  flex-direction: column;  
  gap: 16px;  
  margin-bottom: 24px;  
}
```

You will learn flexbox properly later. For now, notice the spacing behavior: the cards get consistent space between them because the parent `.lesson-list` owns the gap.

This is usually cleaner than putting margin on every card:

CSS

```
.lesson-card {  
  margin-bottom: 16px;  
}
```

That can work, but it often creates awkward extra space after the last item.

gap avoids that because it only creates space between items.

Where should the space belong?

When a page feels cramped or messy, ask where the space should live.

If the text is too close to the inside edge of a card, use padding:

CSS

```
.lesson-card {  
  padding: 16px;  
}
```

If two sections are too close together, use margin:

CSS

```
.intro {  
  margin-bottom: 32px;  
}
```

If items in a group need even space between them, use gap:

CSS

```
.lesson-list {  
  display: flex;  
  flex-direction: column;  
  gap: 16px;  
}
```

That one question solves a lot of beginner spacing confusion: where should the space belong?

 AI Tutor

I am learning CSS spacing. Use my .intro, .lesson-list, .lesson-card, and .actions examples to quiz me on whether a spacing problem should use padding, margin, or gap. Ask me where the space belongs before explaining the answer.

>

Try it

Now test where spacing belongs by changing one spacing choice at a time:

- Change **.intro** padding from **24px** to **8px** , then back to **24px** .
- Change **.intro** **margin-bottom** from **32px** to **8px** , then to **48px** .

- Change `.lesson-list gap` from `16px` to `4px` , then to `24px` .
- Remove `display: flex` and `flex-direction: column` from `.lesson-list` and reload. Notice what happens to `gap` .
- Remove `display: flex` from `.actions` and reload. Notice what happens to `gap` .
- Add a fourth `.lesson-card` and check whether the spacing stays consistent.

For an extra challenge, add a small note under the action row:

html

```
<p class="helper-text">You can change these styles and reload the page after each edit.</p>
```

Use margin to separate it from the action row, and use color or font size only if the text needs to feel less important.

The goal is to stop asking, "Which spacing property should I use?" and start asking, "Where should this space belong?"

< 0 / 10 >

✔ Knew 0 Items

✳️ Learnt 0 Items

⏮ Skipped 0 Items

🔄 ResetProgress

Test Your Understanding

What does padding do?

[Click to Reveal the Answer](#)

✔ Already Know that

✳️ Didn't Know that

⏮ Skip Question

✓

LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →

Cascade, specificity, and inheritance

You have now written enough CSS for rules to overlap, which means one element might match several selectors.

Here is a small example. You may have a button that looks like this:

html

```
<button class="button primary-button">click me</button>
```

This button can be matched by the `button` element selector, the `.button` class selector, and the `.primary-button` class selector.

CSS

```
button {
  background-color: gray;
}

.button {
  background-color: #e5e7eb;
}

.primary-button {
  background-color: #2563eb;
}
```

All three rules set `background-color` for the button. How does CSS decide which `background-color` wins?

CSS is not guessing. The browser follows a decision process called the cascade.

Start with overlapping rules

Use a small two-file page for this cascade exercise: `index.html` and `style.css`.

Start with HTML that lets several rules overlap:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Cascade practice</title>
    <link rel="stylesheet" href="style.css">
```



```
</head>
<body>
  <main class="page">
    <h1>Cascade practice</h1>
    <p class="lead" id="intro">This paragraph matches several rules.</p>
    <p>This paragraph helps us compare inherited and direct styles.</p>
    <button class="button primary-button">Save changes</button>
  </main>
</body>
</html>
```

Add the CSS with overlapping rules:

CSS

```
body {
  font-family: system-ui, sans-serif;
  background-color: #f9fafb;
}

.page {
  max-width: 720px;
  margin: 40px auto;
  color: #111827;
}

p {
  color: #444;
}

.lead {
  color: #2563eb;
  font-size: 20px;
}

#intro {
  color: #b91c1c;
}

.button {
```

◀ Previous Lesson

Next Lesson ▶

```
padding: 10px 20px;
border: 0;
border-radius: 6px;
}

.primary-button {
  background-color: #2563eb;
  color: white;
}
```

Open the page. The first paragraph matches `p` , `.lead` , and `#intro` , and all three rules set `color` . The browser has to choose one.

The cascade chooses a value

The cascade only matters when two or more declarations try to set the same property on the same element.

This is not a conflict:

CSS

```
.lead {
  color: #2563eb;
}
```

```
.lead {  
  font-size: 20px;  
}
```

One declaration sets **color**, and the other sets **font-size**, so the paragraph can use both.

This is a conflict:

CSS

```
.lead {  
  color: #2563eb;  
}  
  
#intro {  
  color: #b91c1c;  
}
```

Both declarations set **color** on the same element. The browser needs to decide which value wins.

The cascade looks at three things:

1. **Specificity**: how strong each selector is. A more specific selector wins over a less specific one. In the example above, **#intro** is stronger than **.lead**, which is stronger than **p**, so the lead paragraph is red.
2. **Source order**: when two rules have the same specificity, the one that appears later in the CSS file wins. The button in the practice page has two class selectors, **.button** and **.primary-button**, so source order decides which shared background color shows up.
3. **Importance**: a declaration marked **!important** can beat normal declarations. You can force CSS to use the styles from a less specific selector using the **!important** declaration. For example you can have **color: #2563eb !important;** to make CSS force the color. It is meant for rare overrides, not everyday styling.

The next few sections walk through each one in turn, starting with specificity.

Specificity is selector strength

Specificity means **selector strength**. A more specific selector wins over a less specific selector when both set the same property on the same element.

Specificity exists so you can write broad rules and then override them for specific cases without deleting the broad rule.

For example, **p { color: #444; }** can style normal paragraphs, while **.lead { color: #2563eb; }** can make one kind of paragraph stand out. The

cascade makes that layering predictable.

In the practice page, the lead paragraph matches all of these:

CSS

```
p {
  color: #444;
}

.lead {
  color: #2563eb;
}

#intro {
  color: #b91c1c;
}
```

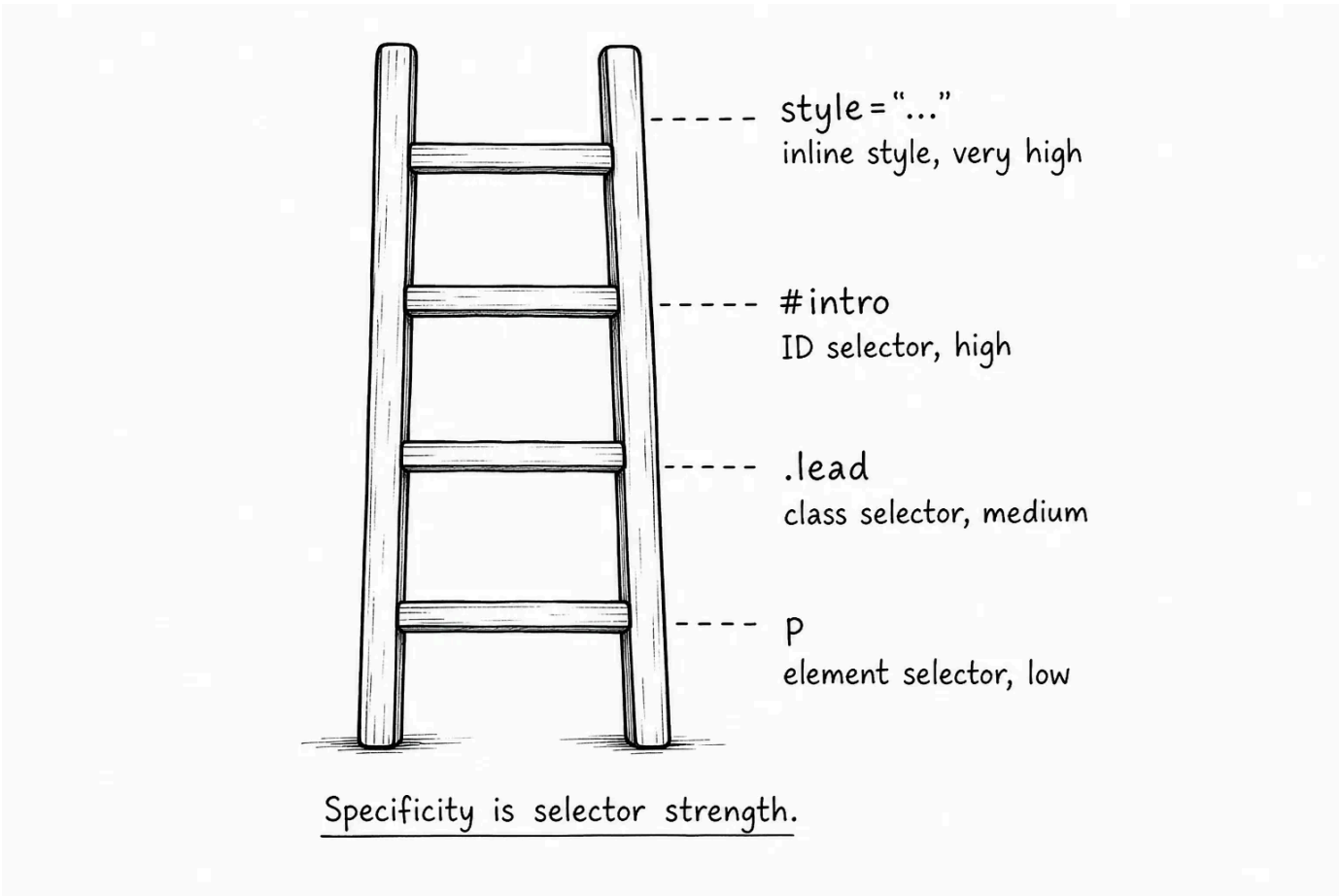
The paragraph is red because `#intro` is the most specific selector in that group.

A useful beginner ranking looks like this:

Selector type	Example	Rough strength
Element selector	<code>p</code>	Low
Class selector	<code>.lead</code>	Medium
ID selector	<code>#intro</code>	High
Inline style	<code>style="color: red;"</code>	Very high

That table is not the full specificity system, but it is enough for early debugging.

We are using `#intro` here only to make specificity easy to see. In real projects, you would usually add another class instead of styling with an ID.



Source order breaks ties

Source order matters when selectors have the same specificity.

Look at the following button rules:

CSS

```
.button {  
  background-color: #e5e7eb;  
  color: #111827;  
}  
  
.primary-button {  
  background-color: #2563eb;  
  color: white;  
}
```

The button has both classes in its HTML:

html

```
<button class="button primary-button">Save changes</button>
```

Both `.button` and `.primary-button` are class selectors, so they have the same specificity. For shared properties like `background-color` and `color`, the later rule wins.

That is why the button is blue with white text.

If you move `.primary-button` above `.button`, the shared `.button` rule comes later and wins those same properties. The button becomes gray again.

Source order does not help if specificity is different. If you move `.lead` below `#intro`, the lead paragraph is still red because `#intro` is more specific.

💡 **Order matters when strength is the same - Order matters when strength is the same** - Source order is not random. If two matching declarations are equally specific, the one that appears later in the CSS wins.

One thing to note about class selectors: as discussed, if you have two selectors with the same specificity that target the same element (e.g. `.button` and `.primary-button`), CSS applies the rule that appears later in the stylesheet.

If you don't want to rely on the order of the rules, you can make a selector more specific. For example, `button.primary-button` or `.parent .primary-button` are more specific than `.primary-button` alone, so their styles would take precedence even if competing rules appear later in the stylesheet.

More specific is not always better

Once you learn specificity, it is tempting to make selectors stronger whenever something does not work.

This selector is strong, but it is tied to the current HTML structure:

CSS

```
main .page p.lead {
  color: #2563eb;
}
```

If the paragraph moves, or the class is reused somewhere else, the selector may stop being helpful.

This is usually easier to understand and reuse:

CSS

```
.lead {
  color: #2563eb;
}
```

Use enough specificity to target the thing clearly. Avoid adding extra element names, IDs, or ancestor chains just to win.

⚠️ **Do not win by making selectors huge** - If you keep making selectors longer just to override earlier CSS, the stylesheet becomes harder to change. First ask whether the class name, source order, or component structure can be simpler.

Inheritance passes some styles down

One important thing that trips beginners up is inheritance. Some CSS properties applied to a parent element can pass down to child elements automatically.

In the practice page, `.page` sets a text color:

CSS

```
.page {  
  color: #111827;  
}
```

That color becomes the default for text inside `.page` unless a more direct rule sets another value.

In the starting CSS, paragraphs also have a direct rule:

CSS

```
p {  
  color: #444;  
}
```

That means the paragraphs use the direct `p` color, not the inherited `.page` color. If you remove the `p` color, the paragraphs fall back to the inherited `.page` color.

Text-related properties often inherit, which is why this kind of rule is so useful:

CSS

```
body {  
  font-family: system-ui, sans-serif;  
}
```

That is why you can set `font-family` once on `body` and most text on the page uses it.

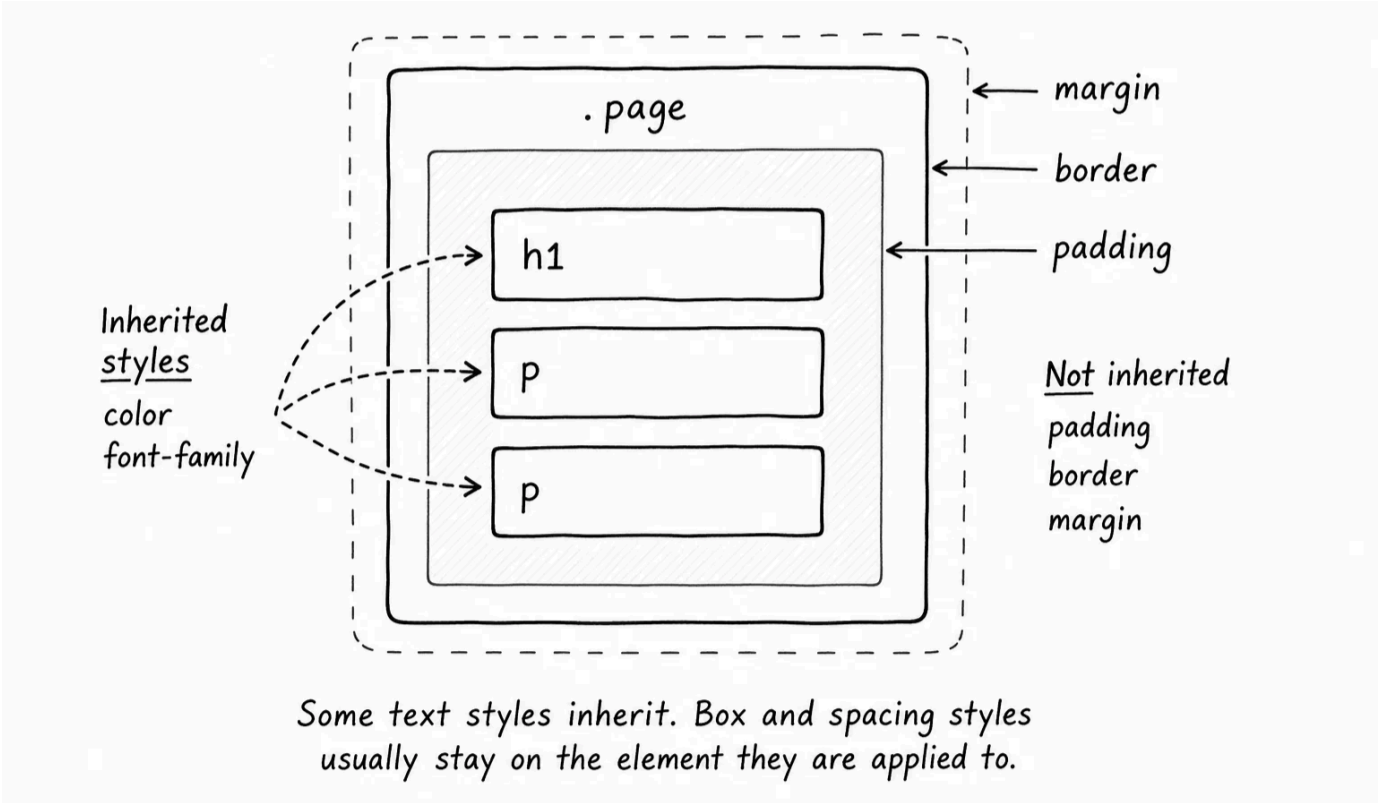
Links are a common inheritance surprise. A link may stay browser-blue even when nearby text inherits a different color, because the browser gives links their own direct color by default. Inheritance is a fallback; direct declarations on the element win.

Box and layout properties usually do not inherit:

CSS

```
.page {  
  padding: 24px;  
  border: 1px solid #e5e7eb;  
}
```

The children inside `.page` do not each get their own padding and border. Those styles belong to the `.page` box.



Direct styles beat inherited styles

Inheritance gives children a default value, but a direct declaration on the child wins.

Look at this CSS:

```
css
.page {
  color: #111827;
}

p {
  color: #444;
}
```

The `p` rule wins for paragraphs because it directly targets the paragraph. The inherited `.page` color is just the fallback.

The same idea applies to `.lead` and `#intro`: they directly target the first paragraph, so they beat inherited text color from `.page`.

Inherited values are defaults for descendants, not locked-in commands.

Crossed-out styles are clues

When a declaration loses, DevTools often shows it crossed out.

Inspect the lead paragraph in the browser. You may see the `p` color and `.lead` color crossed out, while the `#intro` color is active.

That does not mean the `p` selector failed. It matched, but its `color` declaration lost to a more specific rule.

This distinction matters when you debug:

- If a rule does not appear in DevTools, the selector probably did not match.
- If a declaration appears crossed out, the selector matched, but another declaration won.

StylesComputedLayoutEvent Listeners⋮X

<p class="lead" id="intro">

p {

~~color: #444;~~

}

styles.css:10

.lead {

~~color: #2563eb;~~

}

styles.css:18

#intro {

color: #b91c1c;

}

styles.css:27

matched but lost

matched but lost

winning declaration

Crossed-out declarations are usually cascade clues, not broken CSS.

Be careful with important

You can force a declaration to win with `!important` :

CSS

```
.lead {
  color: #047857 !important;
}
```

That tells the browser to treat this declaration as more important than normal declarations.

`!important` can be useful when you are dealing with third-party styles or a very specific override. But it is a bad everyday habit. Once you use it, normal source order and specificity become harder to reason about, and future CSS may need even more `!important` to override it.

⚠

Do not use important as a normal fix - If a style is not applying, inspect the element first. Check whether your selector matches, whether another rule is more specific, and whether source order is the issue. Reach for `!important` only when you understand what you are overriding.

Debug the winning declaration

When a CSS value is not what you expected, debug it in this order:

1. Inspect the element in DevTools.
2. Find the property you care about, such as `color` or `background-color`.
3. Look for all matching rules that set that property.
4. Check whether any declaration is crossed out.
5. Notice whether any declaration uses `!important`.
6. Compare specificity: element, class, ID, inline style.
7. If specificity is the same, check which declaration appears later.
8. Check whether the value is inherited from a parent.

This sounds like a lot, but DevTools puts most of it in one place. The key habit is to inspect the actual element instead of guessing from the stylesheet.

Try it

Now make predictions before each change and use DevTools to check the cascade:

- Inspect the lead paragraph in DevTools. Find which `color` declaration wins.
- Move the `p` rule below the `.lead` rule. Predict what color the lead paragraph will be, then reload. It should stay red because `#intro` is more specific.
- Remove `id="intro"` from the lead paragraph. Predict whether `p` or `.lead` will win, then reload. `.lead` should win because a class is more specific than an element selector.
- Move `.primary-button` above `.button`. Predict the button background, then reload. The button should lose its blue background because the two class selectors have equal specificity and `.button` now comes later.
- Add `color: #047857;` to `.page`, then remove the direct `color` rules from `p`, `.lead`, and `#intro`. Predict where the paragraphs get their color, then reload. They should inherit the color from `.page`.
- Add `!important` to one `color` declaration. Predict whether it will win, inspect the element, then remove it again. Notice how quickly it makes the normal rules harder to read.

The goal is to see that CSS is following rules. When a value surprises you, DevTools can show which declaration won and which declarations lost.

< 0 / 12 >



Knew

0 Items



Learnt

0 Items



Skipped

0 Items



ResetProgress

Test Your Understanding

What is the cascade?

[Click to Reveal the Answer](#)



Already Know that



Didn't Know that



Skip Question



LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →

Debugging CSS with DevTools

In the last lesson, you learned that CSS has rules for deciding which declaration wins.

That knowledge is useful, but you do not have to solve every CSS problem in your head. The browser can show you what it actually did. That is what DevTools is for.

This lesson is about one practical habit: when CSS surprises you, inspect the element before guessing.

Start with a page that has CSS problems

Use a two-file page with a few CSS problems already built in: `index.html` and `style.css` .

Start with this `index.html` file:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>DevTools CSS practice</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <main class="page">
      <section class="intro">
        <p class="eyebrow">CSS practice</p>
        <h1>Debug a styled card</h1>
        <p class="lead">DevTools helps you see which CSS matched, which CSS lost, and wh<
      </section>

      <article class="card featured-card" id="starter-plan">
        <div class="tag-row">
          <span>Popular</span>
          <span>Beginner friendly</span>
        </div>

        <h2>Starter plan</h2>
        <p class="card-text">A small card with a few styles that need debugging.</p>
        <p class="notice">Only a few seats are available this week.</p>
      </article>
    </main>
  </body>
</html>
```

Then add this `style.css` file with the intentional problems:



CSS

```
* {
  box-sizing: border-box;
}

body {
  font-family: system-ui, sans-serif;
  color: #111827;
  background-color: #f9fafb;
}

.page {
  max-width: 760px;
  margin: 40px auto;
}

.intro,
.card {
  background-color: white;
  border: 1px solid #e5e7eb;
  border-radius: 8px;
  padding: 24px;
}

.intro {
  margin-bottom: 24px;
}

.eyebrow {
  color: #2563eb;
  font-weight: 700;
}

.lead,
.card-text {
  color: #4b5563;
  line-height: 1.6;
}
```

< Previous Lesson

Next Lesson >

```
,

.tag-row {
  gap: 8px;
  margin-bottom: 16px;
}

.tag-row span {
  background-color: #eff6ff;
  color: #1d4ed8;
  padding: 6px 10px;
  border-radius: 999px;
  font-size: 14px;
  font-weight: 700;
}

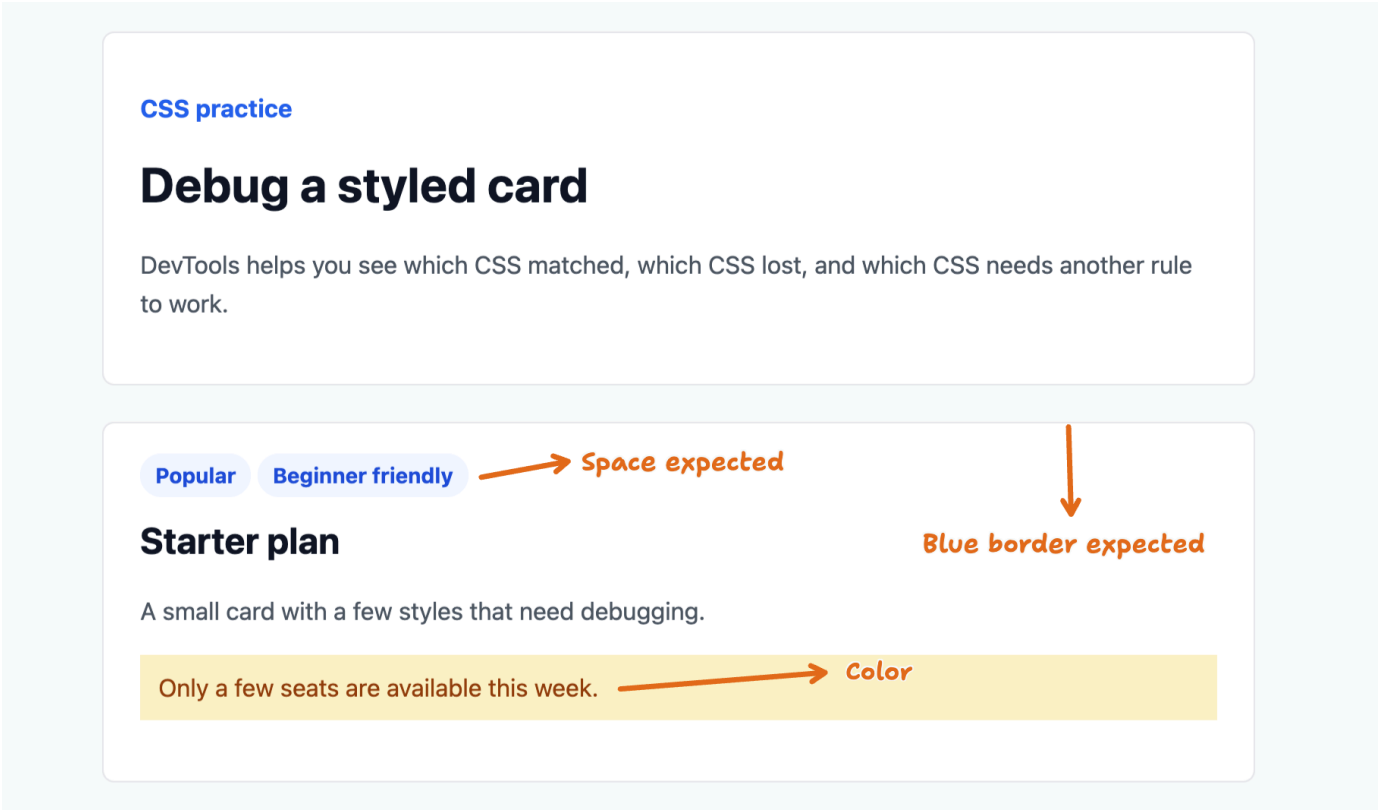
.notice {
  color: #047857;
  background-color: #fef3c7;
  padding: 12px;
  margin-top: 16px;
}

.card .notice {
  color: #92400e;
}
```

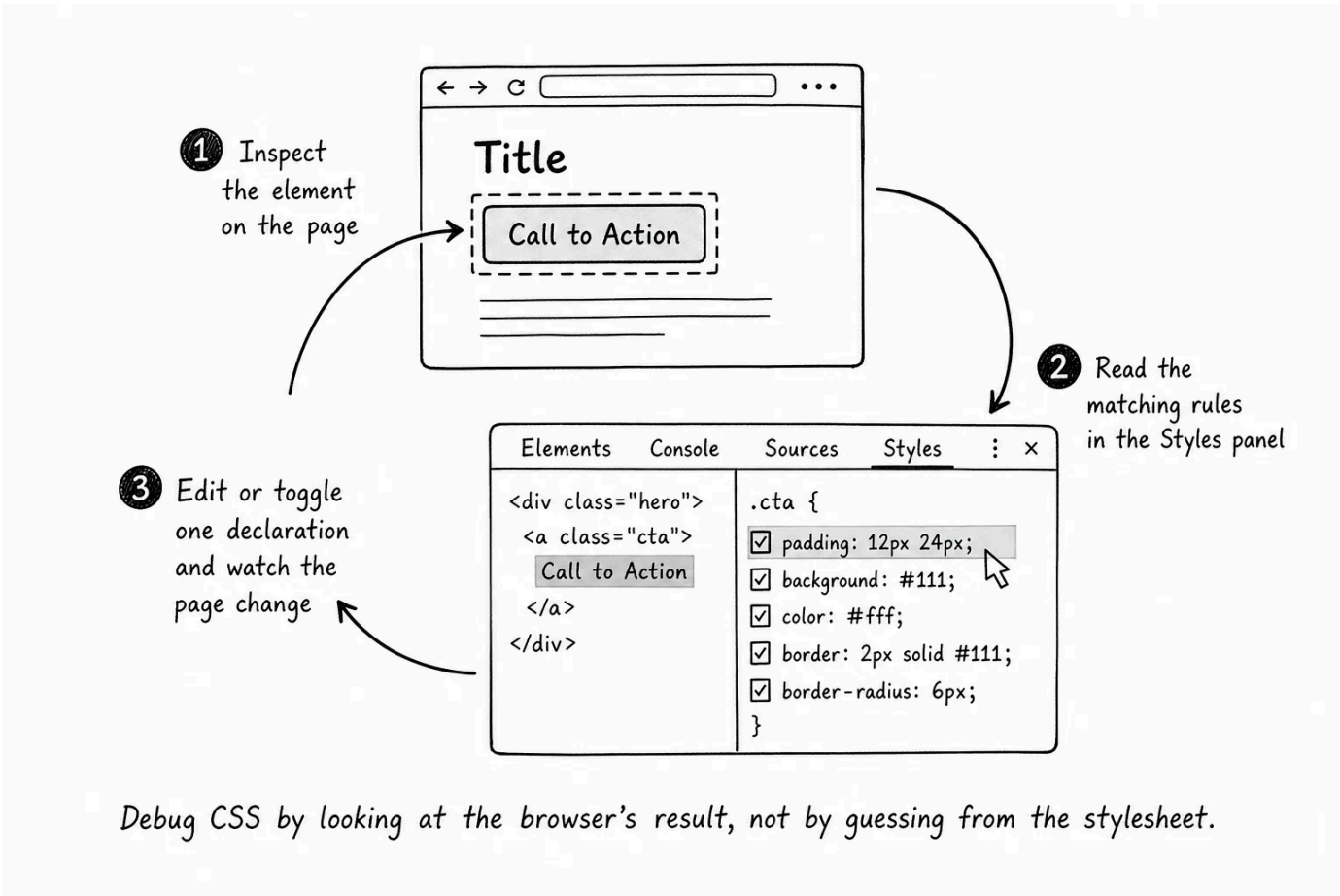
Open the page in a browser.

The CSS file creates three small problems on purpose:

- The featured card border is not blue.
- The notice text color may not be the one you expected from `.notice`.
- The tag pills do not have a visible gap between them.

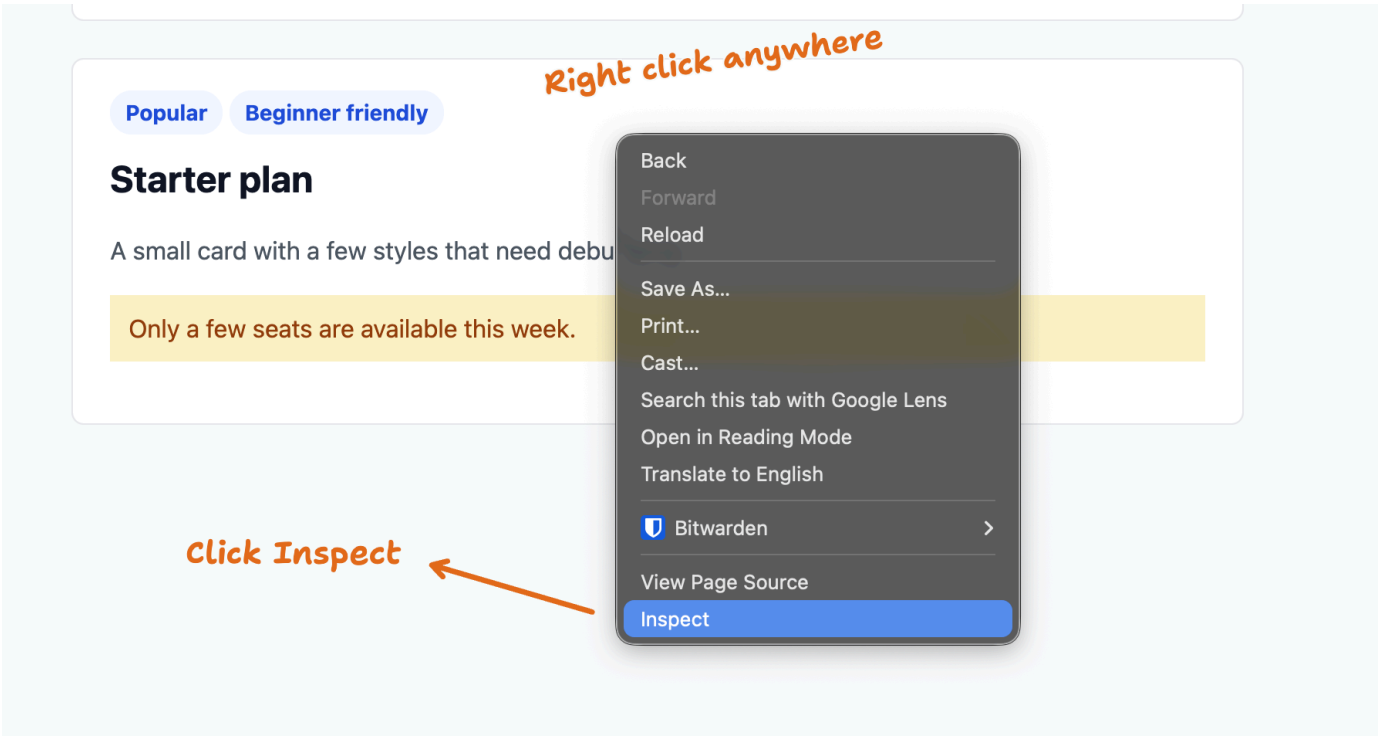


Do not fix them yet; inspect them first.

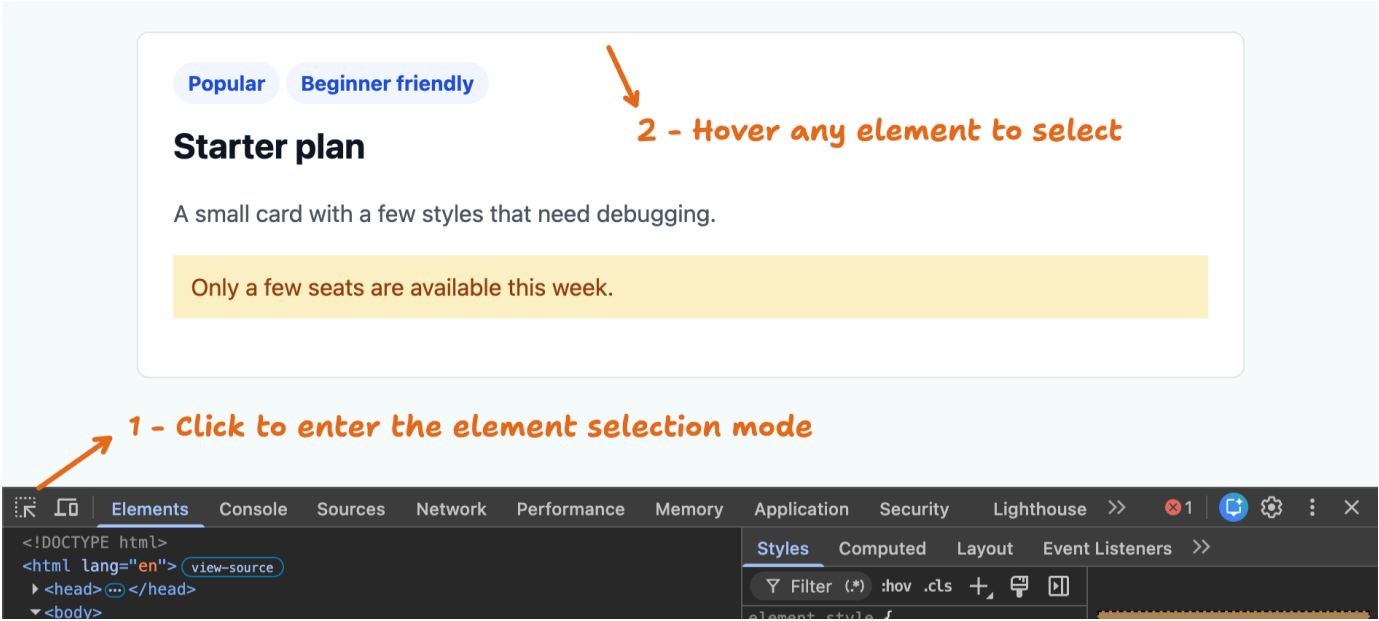


Inspect the element

Open DevTools and inspect the featured card.



In most browsers, you can right-click the card and choose **Inspect** . You can also open DevTools first, then use the element picker to click the card on the page.



For CSS debugging, the first question is simple: am I inspecting the right element?

If you inspect the card, the selected HTML should look like this:

```
html

<article class="card featured-card" id="starter-plan">
```

That selected element is the thing DevTools will explain.

For the first bug, two parts of DevTools matter most: **Elements** shows the selected HTML, and **Styles** shows the CSS rules that match it. Later in the lesson, you will also use **Computed** and the box model diagram.

DevTools may also show browser default styles, often labeled **user agent stylesheet** . These are styles built into the browser. You will see them most often on links, form elements, and headings, and they follow the same cascade rules as your CSS.

💡 **Inspect the exact element** - If a style affects a paragraph, inspect the paragraph. If it affects the card border, inspect the card. CSS debugging gets confusing when you are looking at a parent or child by accident.

Check whether the selector matched

The card is supposed to have a blue border because the HTML has **featured-card**:

html

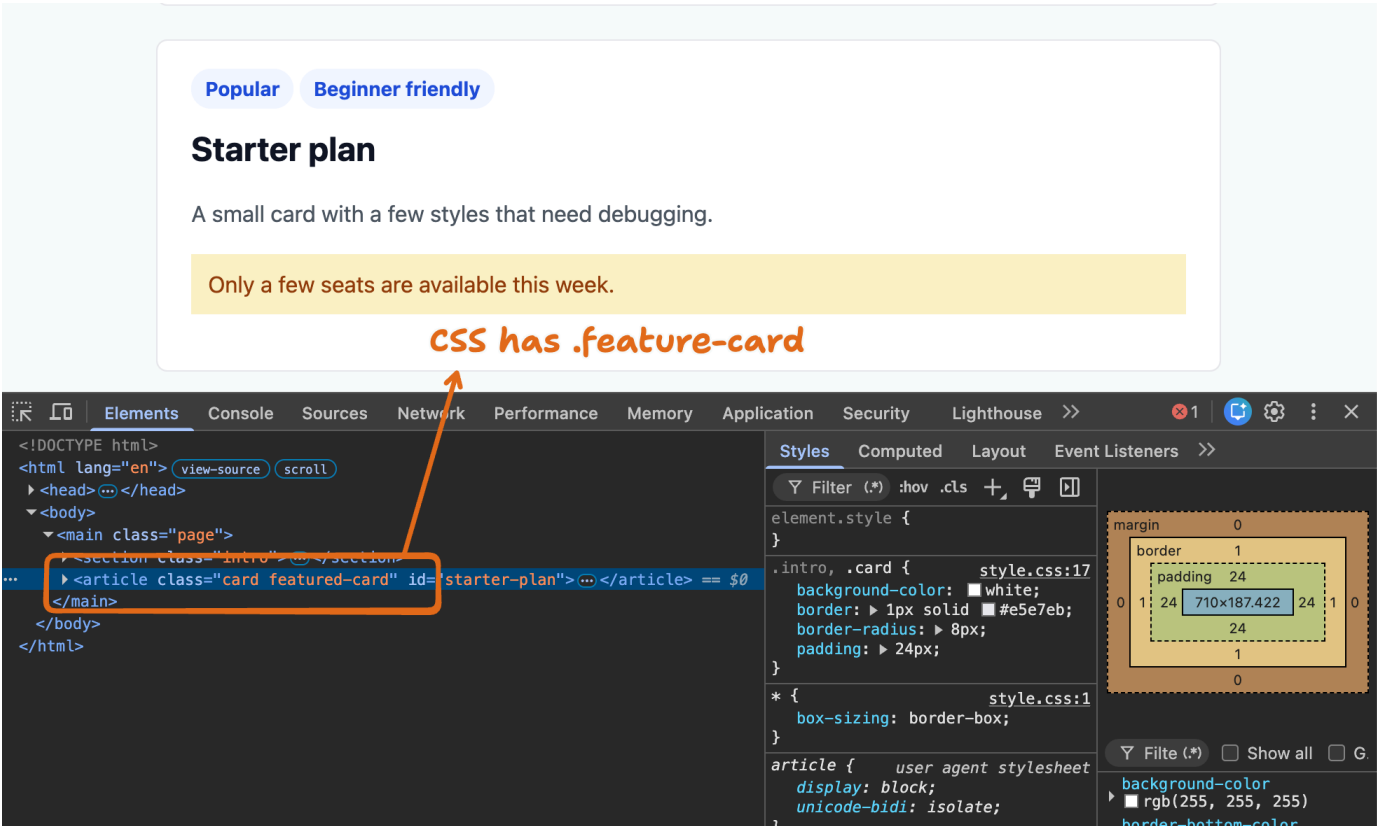
```
<article class="card featured-card" id="starter-plan">
```

But the CSS is looking for a different class:

CSS

```
.feature-card {  
  border-color: #2563eb;  
}
```

Inspect the card and look in the **Styles** panel. You should see the **.card** rule, but you will not see **.feature-card** as a matching rule for this element.



That is the clue: the selector does not match because the class names are different.

- HTML: **featured-card**
- CSS: **.feature-card**

Fix the selector by matching the HTML class name:

CSS

```
.featured-card {  
  border-color: #2563eb;  
}
```

Reload the page and the card border should become blue.

When a rule does not appear in DevTools at all, the selector probably did not match the element.

Invalid CSS is a different clue. If a rule appears but a value is invalid, DevTools may cross out that declaration or show a warning icon. That is not the same as a valid declaration losing in the cascade. The browser usually ignores invalid CSS silently instead of showing an error message.

Check whether the declaration lost

Next, inspect the notice:

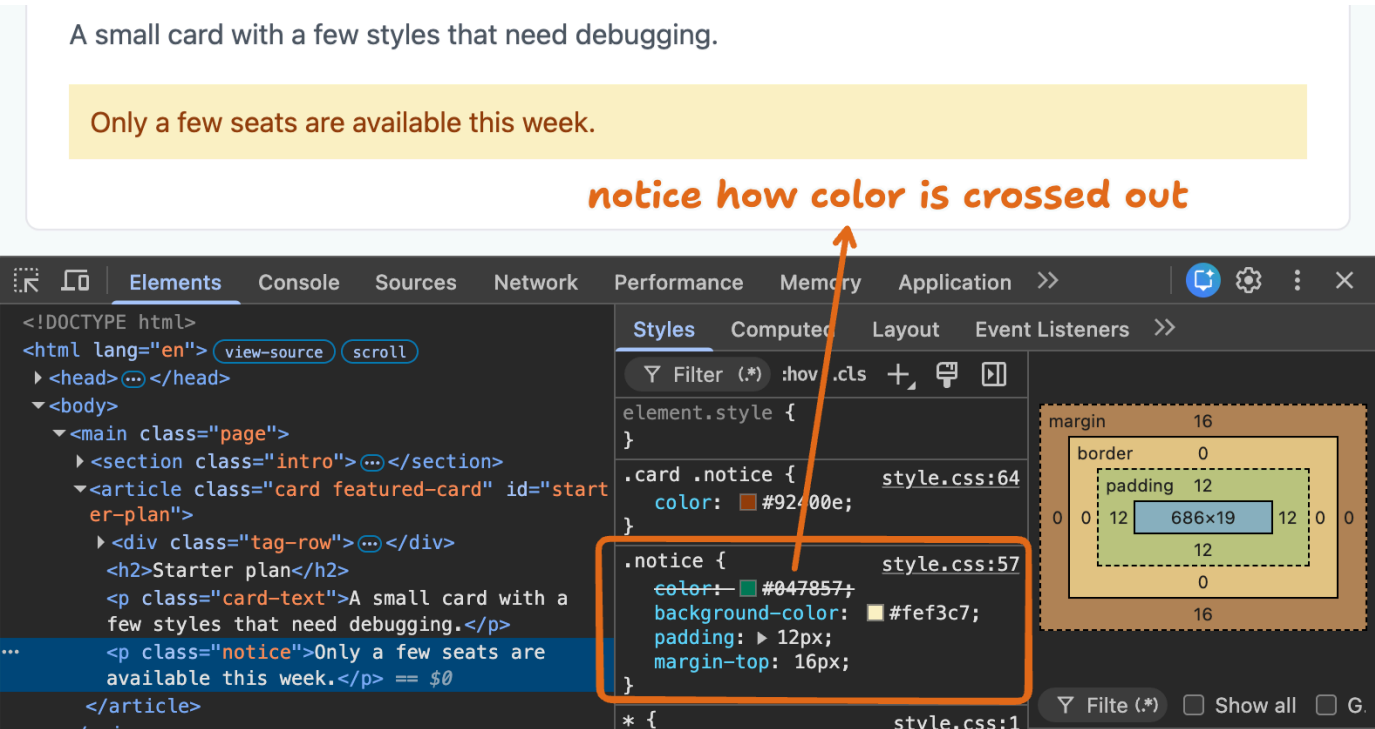
```
html  
  
<p class="notice">Only a few seats are available this week.</p>
```

Two rules set its text color:

```
CSS  
  
.notice {  
  color: #047857;  
}  
  
.card .notice {  
  color: #92400e;  
}
```

The `.notice` selector matches, and the `.card .notice` selector also matches because the notice sits inside `.card`.

In DevTools, you may see the `.notice` color crossed out and the `.card .notice` color active.



That means the selector matched, but the declaration lost.

This is different from the previous bug. `.feature-card` did not match at all. `.notice` matches, but its `color` declaration loses to a more specific selector.

This is the cascade from lesson 7 in action: both selectors match, but the more specific selector wins.

Check whether the property needs setup

The tag row has this CSS:

```
CSS

.tag-row {
  gap: 8px;
  margin-bottom: 16px;
}
```

The `gap` declaration is valid, but it does not create visible space yet.

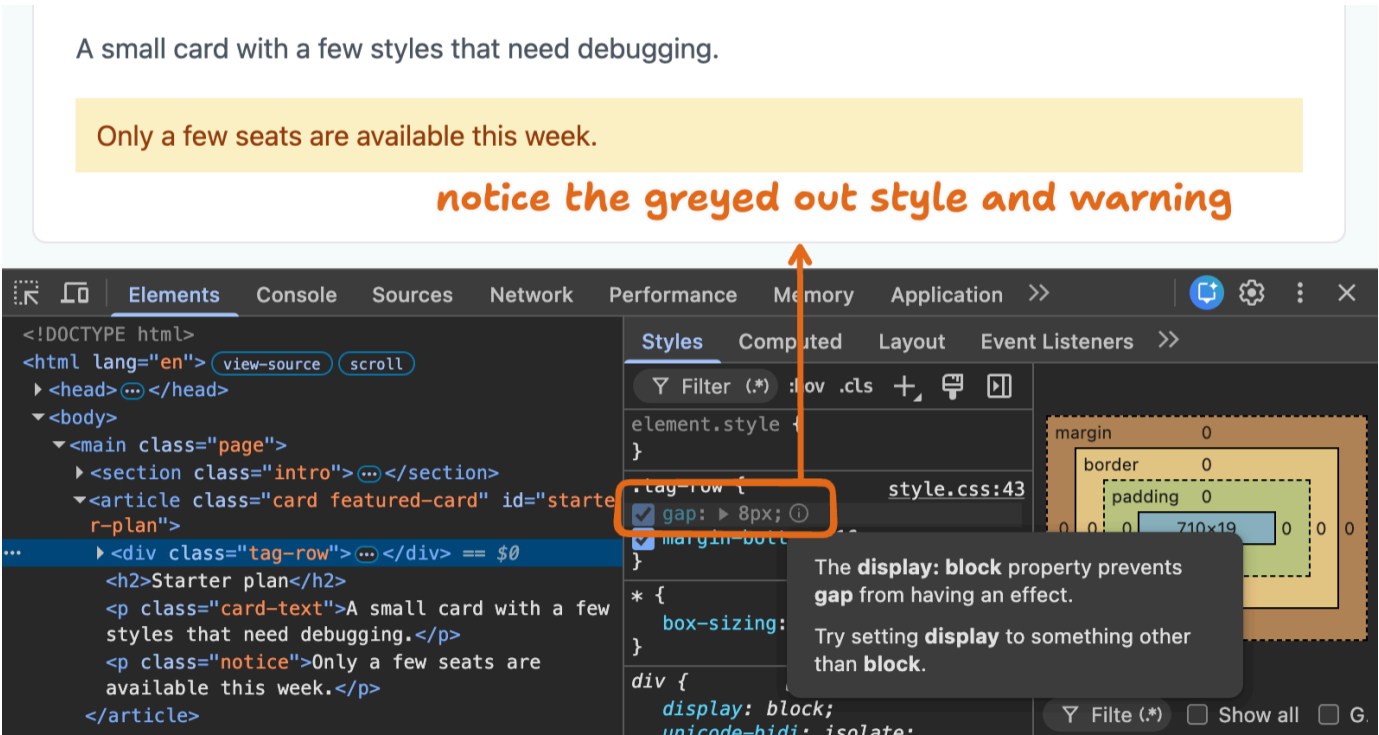
The reason is that `gap` needs a layout mode like flex or grid on the parent. You saw this in the spacing lesson.

Fix the parent by adding `display: flex`:

```
CSS

.tag-row {
  display: flex;
  gap: 8px;
  margin-bottom: 16px;
}
```

Reload the page and the tag pills should now have space between them.



This is another debugging category: the selector matched and the declaration may not be crossed out, but the property still needs the right surrounding

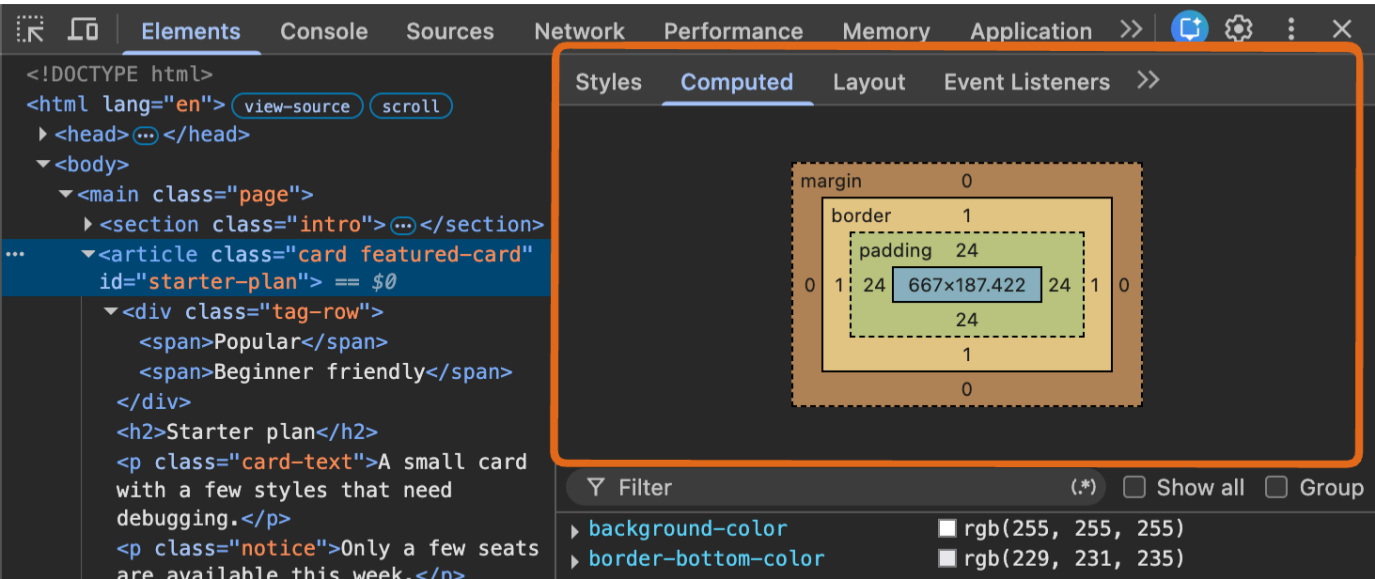
CSS to have a visible effect.

⚠ **Not every CSS surprise is the cascade** - If a declaration is not crossed out but still does not seem to change the page, ask whether the property needs another setup rule. `gap` needs flex or grid. Some layout properties only work in certain contexts.

Use the box model diagram

DevTools can also show the box model.

Inspect the card and look for the box model diagram in the **Computed** panel or layout area. You should see content in the middle, then padding, border, and margin around it.



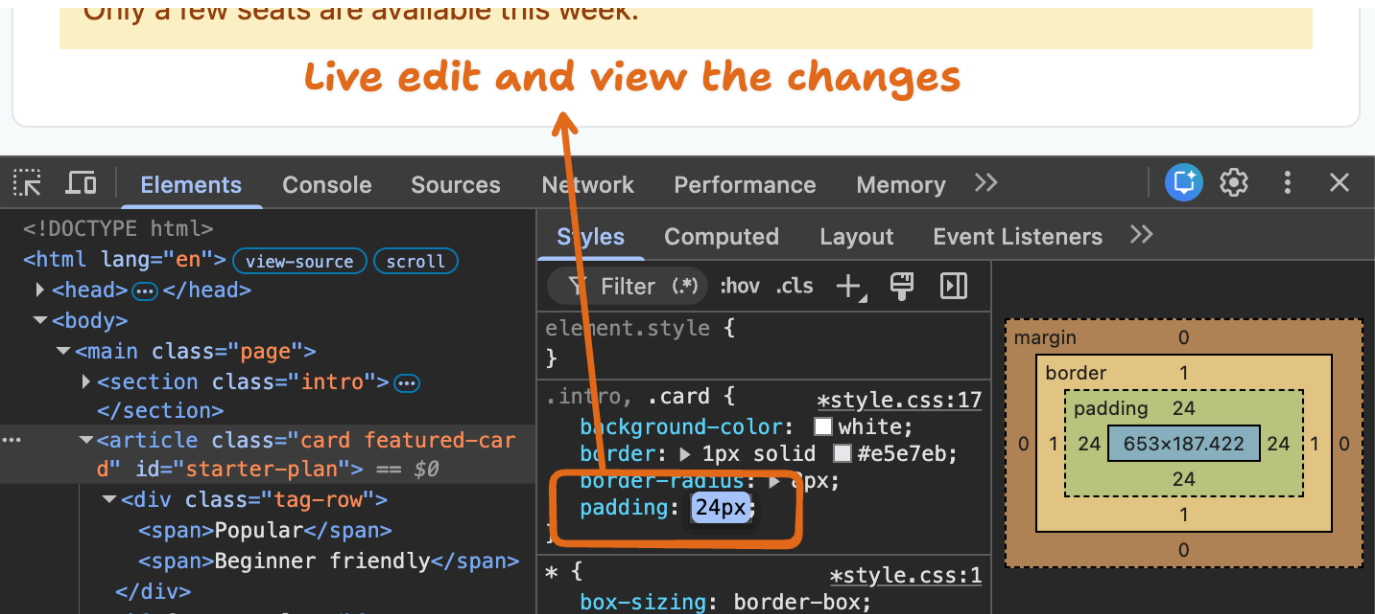
Find these values from the CSS:

```
CSS

.card {
  border: 1px solid #e5e7eb;
  padding: 24px;
}
```

The diagram should show **24px** padding and a **1px** border.

Now temporarily change `padding: 24px;` to `padding: 8px;` inside DevTools, and the card should update immediately.



This live edit does not save to your file. It is a safe way to test a value before changing `style.css`.

 **Use DevTools as a scratchpad** - When you are unsure about a spacing, color, or size value, try it in DevTools first. Once it looks right, copy the final value into your CSS file.

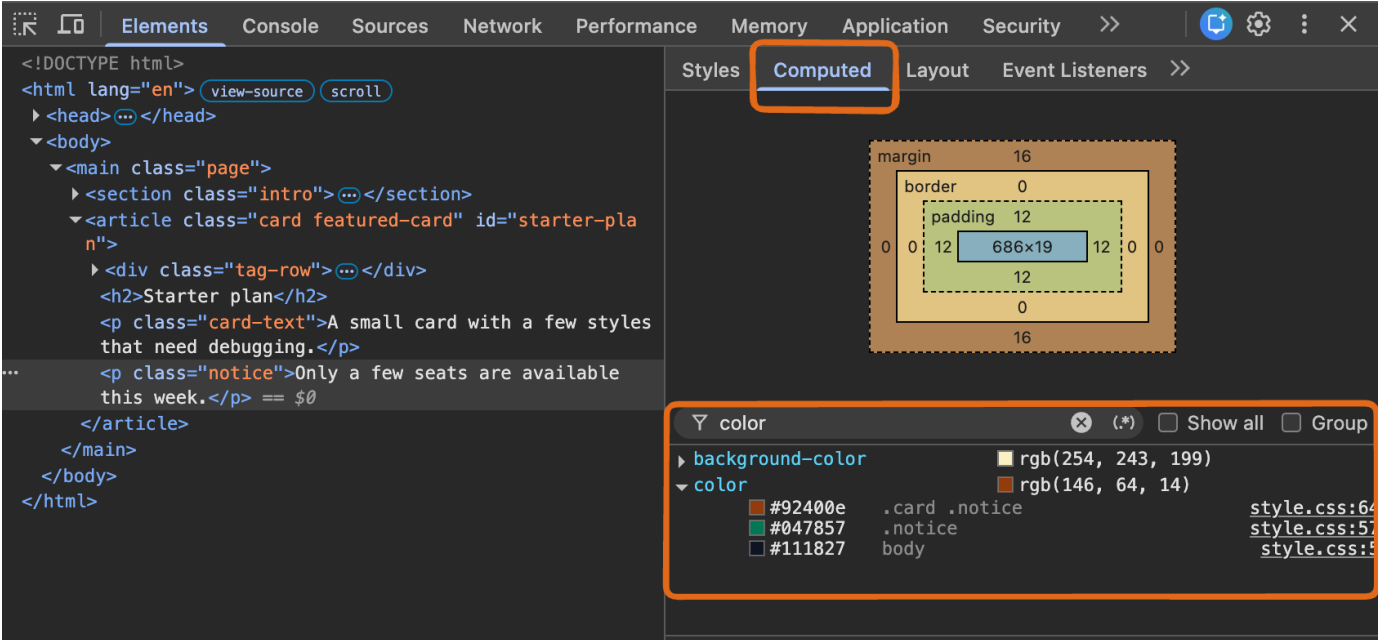
Use Computed for the final answer

The `Styles` panel shows the rules that matched. The `Computed` panel shows the final value the browser is using.

For example, inspect the notice and search for `color` in the `Computed` panel.

You should see the final text color value from `.card .notice`. Some browsers may show it as `#92400e`; others may show the same color as `rgb(146, 64, 14)`.

If DevTools shows a small arrow or expandable row next to the computed value, open it. It can often show which CSS rule supplied that final value.



This is useful when a page has many stylesheets. You do not have to read every file first. Start with the element, find the property, and let DevTools show the winner.

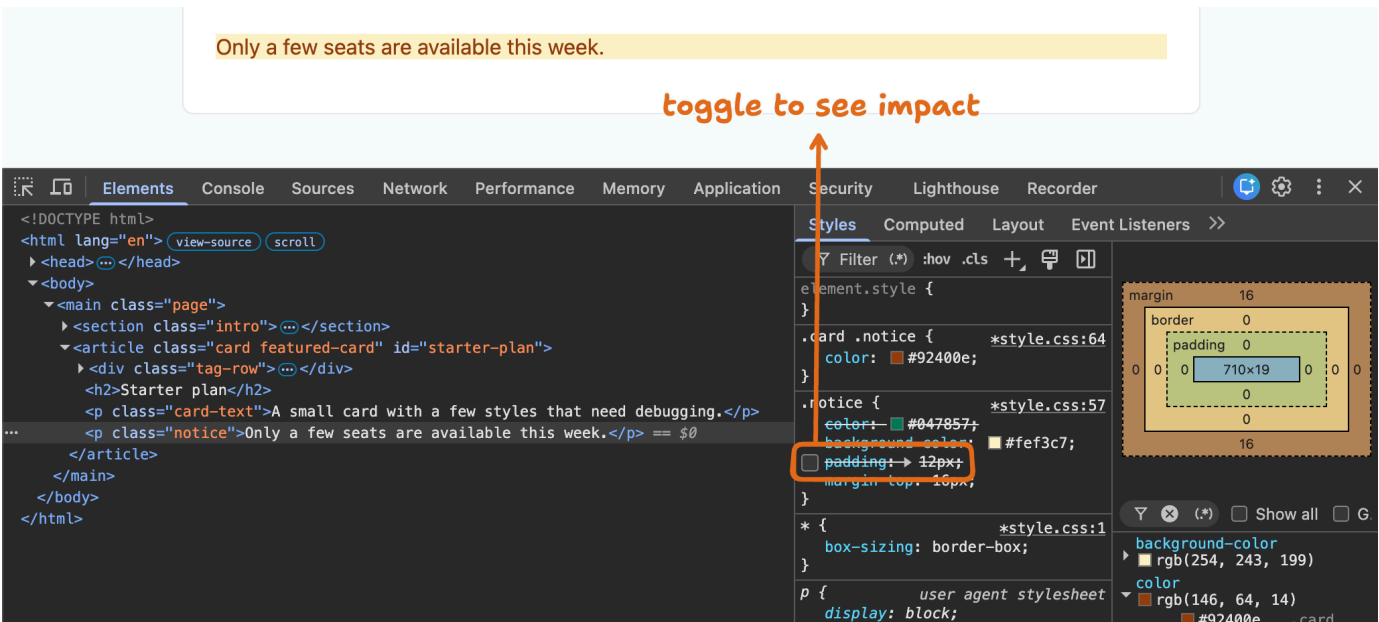
Toggle declarations on and off

Most DevTools let you turn a declaration off with a checkbox.

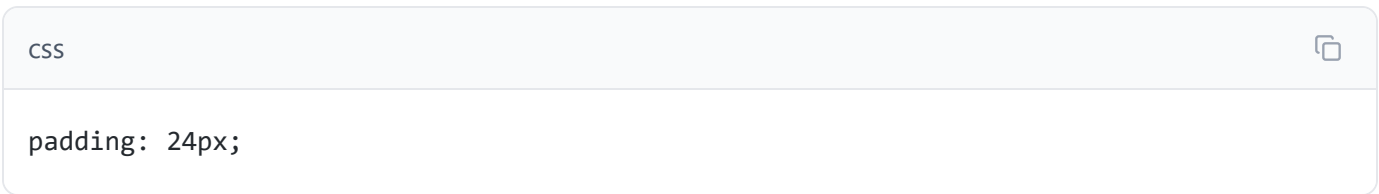
Try the toggle with this small sequence:

1. Inspect the notice.
2. Find `color: #92400e;` in `.card .notice`.
3. Turn that declaration off.
4. Watch the notice text change to the `.notice` color.

This is a quick way to test cause and effect. You are asking the browser, "What happens if this declaration is not here?"



You can also click values and edit them:



Try changing it to `16px` , `32px` , or `40px` in DevTools. Then put the value you like into `style.css` .

Try it

Before moving on, recreate the same debugging clues in a few new ways:

- Change the HTML class from `featured-card` to `featured-plan`, but do not change the CSS yet. Inspect the card and confirm the `.featured-card` rule no longer appears as a match.
- Change `.card .notice` to `.notice` and reload. Predict which notice color wins, then inspect the notice to confirm.
- Remove `display: flex` from `.tag-row`. Inspect the tag row and notice that `gap` no longer creates visible space.
- Inspect `.card`, change its padding in DevTools, and watch the box model diagram update.
- In DevTools, turn off `background-color: #fef3c7;` on `.notice`. Notice that the text remains, but the warning box no longer feels like a message.
- Add an invalid value like `color: brightblue;` to `.lead` in DevTools. Watch how the browser ignores invalid CSS.

The goal is not to memorize every DevTools panel. The goal is to build a habit: inspect the element, find the property, and look for the clue.



Knew

0 Items



Learnt

0 Items



Skipped

0 Items



ResetProgress

Test Your Understanding

What should you do first when a CSS style surprises you?

[Click to Reveal the Answer](#)



Already Know that



Didn't Know that



Skip Question



LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →

Block, inline, and display

You have used DevTools to inspect what the browser did with your CSS. Now it is time to inspect a different kind of default: how elements naturally sit on the page before you write layout CSS.

Some elements start on a new line, some sit inside a line of text, and some can be changed when the default behavior is not what you need. That behavior is controlled by the `display` property.

Start with a mixed page

Start with an `index.html` file that mixes block and inline elements:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Display practice</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <main class="page">
      <section class="intro">
        <p class="eyebrow">CSS practice</p>
        <h1>Block, inline, and display</h1>
        <p class="summary">Some elements stack like sections. Some sit inside a sentence</p>
      </section>

      <nav class="lesson-nav" aria-label="Lesson sections">
        <a href="#card">Card</a>
        <a href="#notice">Notice</a>
        <a href="#actions">Actions</a>
      </nav>

      <article class="card" id="card">
        <span class="badge">New</span>
        <h2>Display controls how an element participates in layout</h2>
        <p class="card-text">
          A paragraph is a block element by default, but a
          <a href="/docs">link</a>
          sits inline with the sentence.
        </p>
      </article>

      <p class="notice" id="notice">
        This notice is a paragraph, so it starts on its own line and takes the available
      </p>

      <div class="actions" id="actions">
        <a class="action-link" href="/start">Start practice</a>
        <a class="action-link secondary-action" href="/notes">Read notes</a>
      </div>
    </main>
  </body>
</html>
```



Then add this stylesheet:

CSS

```
* {
  box-sizing: border-box;
}

body {
  font-family: system-ui, sans-serif;
  color: #111827;
  background-color: #f9fafb;
}

.page {
  max-width: 760px;
  margin: 40px auto;
}

.intro,
.card,
.notice {
  background-color: white;
```

```

}

.intro,
.lesson-nav,
.card,
.notice {
  margin-bottom: 24px;
}

.eyebrow {
  color: #2563eb;
  font-weight: 700;
}

.summary,
.card-text {
  color: #4b5563;
  line-height: 1.6;
}

.badge {
  background-color: #fff6ff;
  color: #1d4ed8;
  padding: 6px 10px;
  border-radius: 999px;
  font-size: 14px;
  font-weight: 700;
}

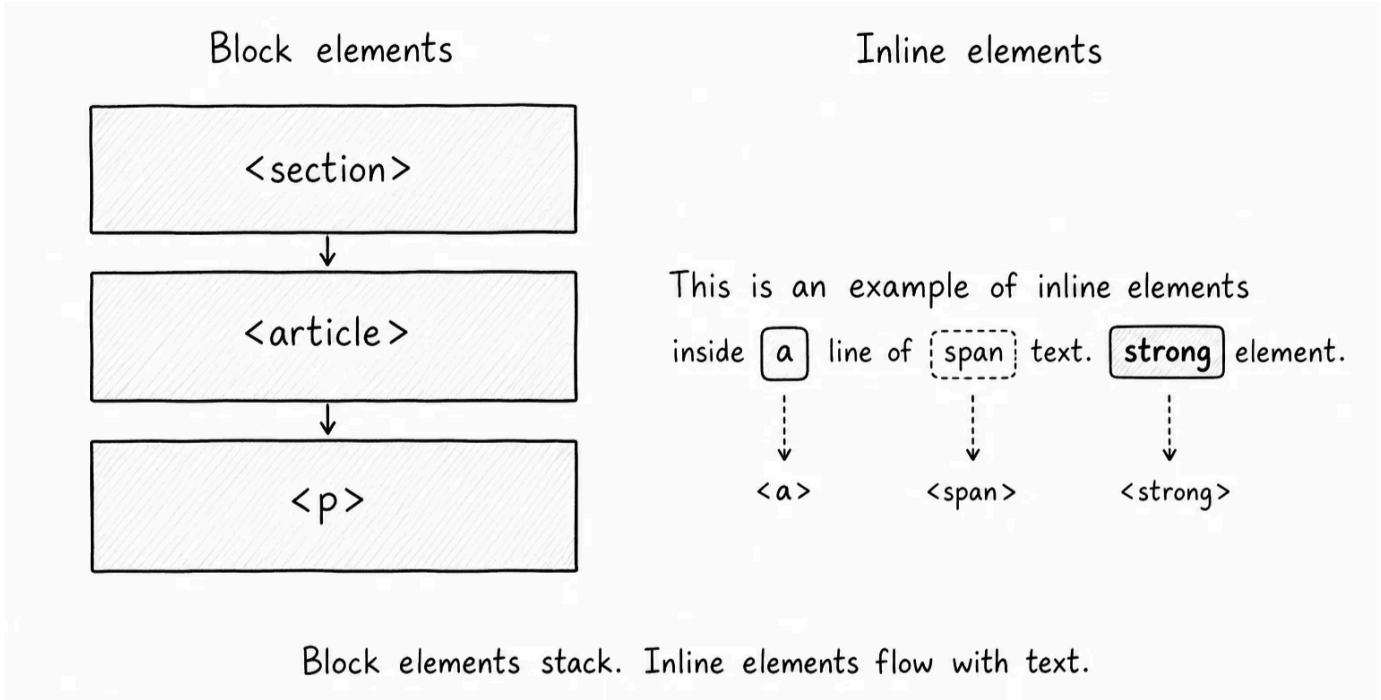
.action-link {
  width: 180px;
  padding: 12px 16px;
  color: white;
  background-color: #2563eb;
  border-radius: 6px;
  text-align: center;
  text-decoration: none;
  font-weight: 700;
}

.secondary-action {
  color: #2563eb;
  background-color: white;
  border: 1px solid #bfdbfe;
}
```

Open the page and notice a few things before changing the CSS:

- The intro, card, and notice stack vertically.
- The navigation links sit side by side because links are inline by default.
- The badge sits before the heading because `span` is inline by default.
- The action links have padding and color, but `width: 180px` does not behave the way you might expect.

Those differences are the point of this lesson.



Block elements stack

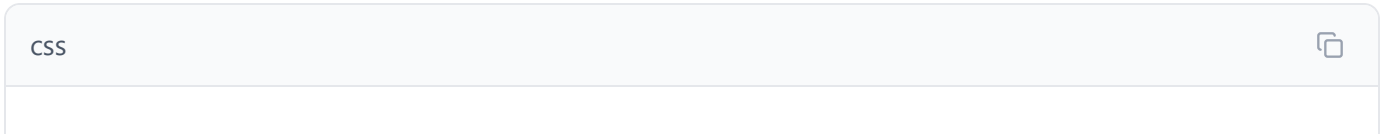
A block element starts on a new line and usually takes the full available width. Common block elements include:

- `<h1>` through `<h6>`
- `<p>`
- `<div>`
- `<section>`
- `<article>`
- `<header>`
- `<footer>`
- `<nav>`

That is why in our webpage example, the intro, card, and notice stack one after another. Their default display behavior is block-like.

Inspect the notice in DevTools. In the `Computed` panel, search for `display` ; you should see that the paragraph is `display: block` .

You can make this behavior explicit:




```
.notice {  
  display: block;  
}
```

That does not change much because the paragraph was already block by default. It just names the behavior the browser was already using.

💡 **Defaults are still CSS behavior** - Browser defaults are not magic. A paragraph stacks because the browser treats it as a block-level element by default, and DevTools can help you confirm that default instead of guessing.

Inline elements flow with text

An inline element stays inside the current line of text. Common inline elements include:

- `<a>`
- `<span>`
- `<strong>`
- `<em>`
- `<code>`

That is why the link inside the paragraph sits inside the sentence:

html

```
<p class="card-text">  
  A paragraph is a block element by default, but a  
  <a href="/docs">link</a>  
  sits inline with the sentence.  
</p>
```

The link does not start a new line. It flows with the words around it.

The badge works the same way:

html

```
<span class="badge">New</span>
```

A `span` is inline by default, so it sits near the text around it instead of becoming a full-width box.

Inline elements do not behave like full boxes

Inline elements can accept many visual styles, but they do not behave like full block boxes.

Look at the action links:

CSS

```
.action-link {  
  width: 180px;  
  padding: 12px 16px;  
}
```

The padding is visible, but `width: 180px` does not create a consistent button width because links are inline by default.

That is a common beginner surprise: `width` and `height` do not work on most normal inline elements the way they work on block elements. Some elements, like images and inputs, are exceptions. You will see those later.

Inspect an action link in DevTools. The `width` declaration may appear in the Styles panel, but the final layout still follows inline behavior. The CSS is not broken; the display type is the missing piece.

Change display when the default is wrong

The `display` property changes how an element participates in layout.

For action-style links, `inline-block` is often useful:

CSS

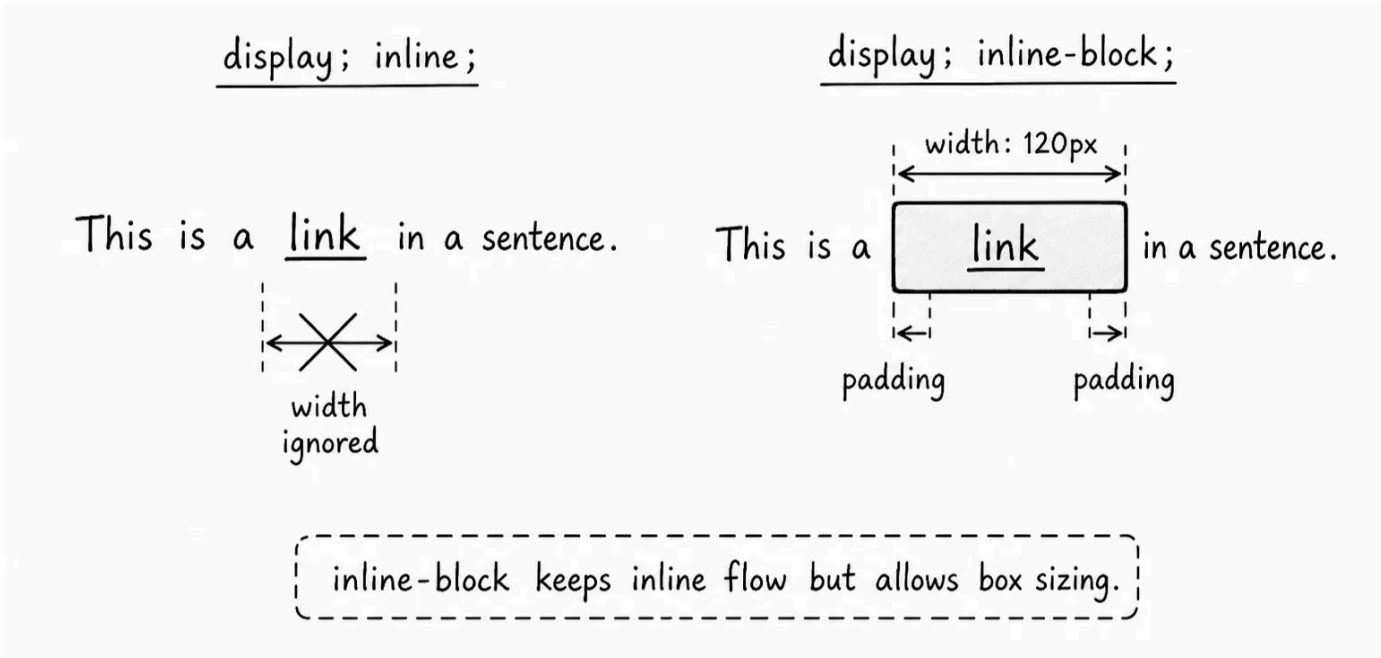
```
.action-link {  
  display: inline-block;  
  width: 180px;  
  padding: 12px 16px;  
  color: white;  
  background-color: #2563eb;  
  border-radius: 6px;  
  text-align: center;  
  text-decoration: none;  
  font-weight: 700;  
}
```

Reload the page and the two action links should now keep their button-like shape and use the width you set.

`inline-block` is a middle ground:

- It sits in a line like inline content.
- It can use `width` , `height` , padding, border, and margin more like a block box.

That makes it useful for small things like badges, labels, and simple button-style links.



Use display block for full-width actions

Sometimes you want a link to behave like a full-width block.

Try changing the action link rule:

CSS

```
.action-link {
  display: block;
  width: 100%;
}
```

Now each action link starts on its own line and fills the available width.

That may be useful for a vertical menu or a stacked mobile layout, but it is not the right choice for every link. A normal text link inside a paragraph should stay inline.

For these action links, **inline-block** is usually the better fit because the links can sit side by side while still behaving like button-shaped boxes.

The useful question is: should this element behave like text, like a box, or like a full row?

Hide elements with display none

You can hide an element with **display: none** :

CSS

```
.notice {
  display: none;
}
```

The notice disappears, and the page closes the space where it used to be.

That is different from making the text transparent or changing the background. **display: none** removes the element from layout, so it no longer takes up space.

There is a related property worth knowing: **visibility: hidden** . It hides the element, but it leaves the space the element would have taken up. Use **display: none** when the element should not exist in the layout. Use **visibility: hidden** when it should still hold its space, but not show.

Use this carefully. If content is important for the user, hiding it can create accessibility and usability problems.

Display does not change meaning

CSS can change how an element is displayed, but it does not change what the HTML means.

For example, you can make a link look like a button:

CSS

```
.action-link {
  display: inline-block;
  background-color: #2563eb;
  color: white;
}
```

It is still a link, and it still navigates somewhere because the HTML is **<a>** .

You can also make a heading look small:

CSS

```
h1 {
  font-size: 18px;
}
```

It is still the page heading because the HTML is **<h1>** .

Screen readers, search engines, keyboard navigation, and browser features use the HTML element, not just the CSS display value. A link styled like a button is still announced as a link. A small-styled **<h1>** is still the page's top heading for navigation tools.

Choose HTML for meaning, and use CSS display for layout behavior.

 **Keep meaning and layout separate** - Do not pick **** just because you want something inline, or **<div>** just because you want something block. Pick HTML for meaning first, then use CSS when you need to change the display behavior.

Display grows into layout systems

You have already seen this rule:

CSS

```
.tag-row {  
  display: flex;  
  gap: 8px;  
}
```

You used `display: flex` in the spacing lesson to make `gap` work. That was a display value all along. It told the browser to use a flex layout mode for that parent's children.

`grid` is also a `display` value.

You do not need to learn them fully in this lesson. For now, the important idea is that `display` chooses the layout mode for an element.

- `block` stacks as a full-width box.
- `inline` flows with text.
- `inline-block` flows with text but behaves more like a box.
- `none` removes the element from layout.
- `flex` and `grid` create layout systems for an element's children.

Flexbox and grid get their own lessons later. This lesson gives you the foundation for why those layout systems start with `display`.

Try it

Now change display behavior on purpose and watch how the layout responds:

- Inspect the notice in DevTools and confirm its computed `display` value.
- Add `display: block;` to `.lesson-nav a`. Notice that each navigation link starts on its own line.
- Change `.lesson-nav a` back to inline behavior by removing `display: block;`.
- Add `display: inline-block;` to `.badge`, then add `margin-bottom: 12px;`. Notice that the badge can now create more reliable space around itself.
- Change `.action-link` from `inline-block` to `block`. Notice that each action link becomes its own row.
- Add `display: none;` to `.notice`, then inspect the page. Notice that the notice no longer takes up space.

- Replace `display: none;` with `visibility: hidden;` on `.notice`.
Notice that the notice disappears, but its space stays.

The goal is to stop treating element layout as random. If an element will not size, stack, or sit inline the way you expect, check its `display` value.

< 0 / 12 >

✔ Knew 0 Items

✦ Learnt 0 Items

⏮ Skipped 0 Items

🔄 ResetProgress

Test Your Understanding

What does display control?

[Click to Reveal the Answer](#)

✔ Already Know that

✦ Didn't Know that

⏮ Skip Question

✓

LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →

Images, media, and backgrounds

In the [HTML pack](#), you learned that images and media need meaningful HTML: `src`, `alt`, captions, controls, and useful fallback text. CSS has a different job for that media.

CSS decides how that media fits into the page: how wide it is, whether it keeps its shape, whether it crops, and whether an image is real content or just a decorative background.

Start with a media card

Before you start, save a landscape photo, roughly `1200px` wide and `800px` tall, with the name `course-cover.jpg` in the same folder as your HTML. You can [download this sample image](#) if you need one. If you choose a different filename, make sure to update the `src` and `background-image` values in the code below.

Use two files for this media exercise. Start with this `index.html` :

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Image sizing practice</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <main class="page">
      <section class="intro">
        <p class="eyebrow">CSS practice</p>
        <h1>Images, media, and backgrounds</h1>
        <p class="summary">Media has its own natural size. CSS helps it fit the boxes or</p>
      </section>

      <article class="course-card">
        
          <p class="badge">Featured lesson</p>
          <h2>Design systems for beginners</h2>
          <p class="course-text">Learn how spacing, color, and reusable components make
        </div>
      </article>

      <section class="promo">
        <p class="eyebrow">Workshop</p>
        <h2>Build a visual landing page</h2>
        <p>Use one strong image, readable text, and simple spacing.</p>
      </section>
    </main>
```

```
</body>
</html>
```

Then add the starting CSS:

CSS

```
* {
  box-sizing: border-box;
}

body {
  font-family: system-ui, sans-serif;
  color: #111827;
  background-color: #f9fafb;
}

.page {
  max-width: 760px;
  margin: 40px auto;
}

.intro,
.course-card,
.promo {
  margin-bottom: 24px;
}

.intro,
.course-card {
  background-color: white;
  border: 1px solid #e5e7eb;
```

```
.intro,
.course-body,
.promo {
  padding: 24px;
}

.eyebrow {
  color: #2563eb;
  font-weight: 700;
}

.summary,
.course-text {
  color: #4b5563;
  line-height: 1.6;
}

.badge {
  display: inline-block;
  color: #1d4ed8;
  background-color: #eff6ff;
  padding: 6px 10px;
  border-radius: 999px;
  font-size: 14px;
  font-weight: 700;
}

.course-image {
  border-radius: 8px 8px 0 0;
}

.promo {
  color: white;
  background-color: #111827;
  border-radius: 8px;
}

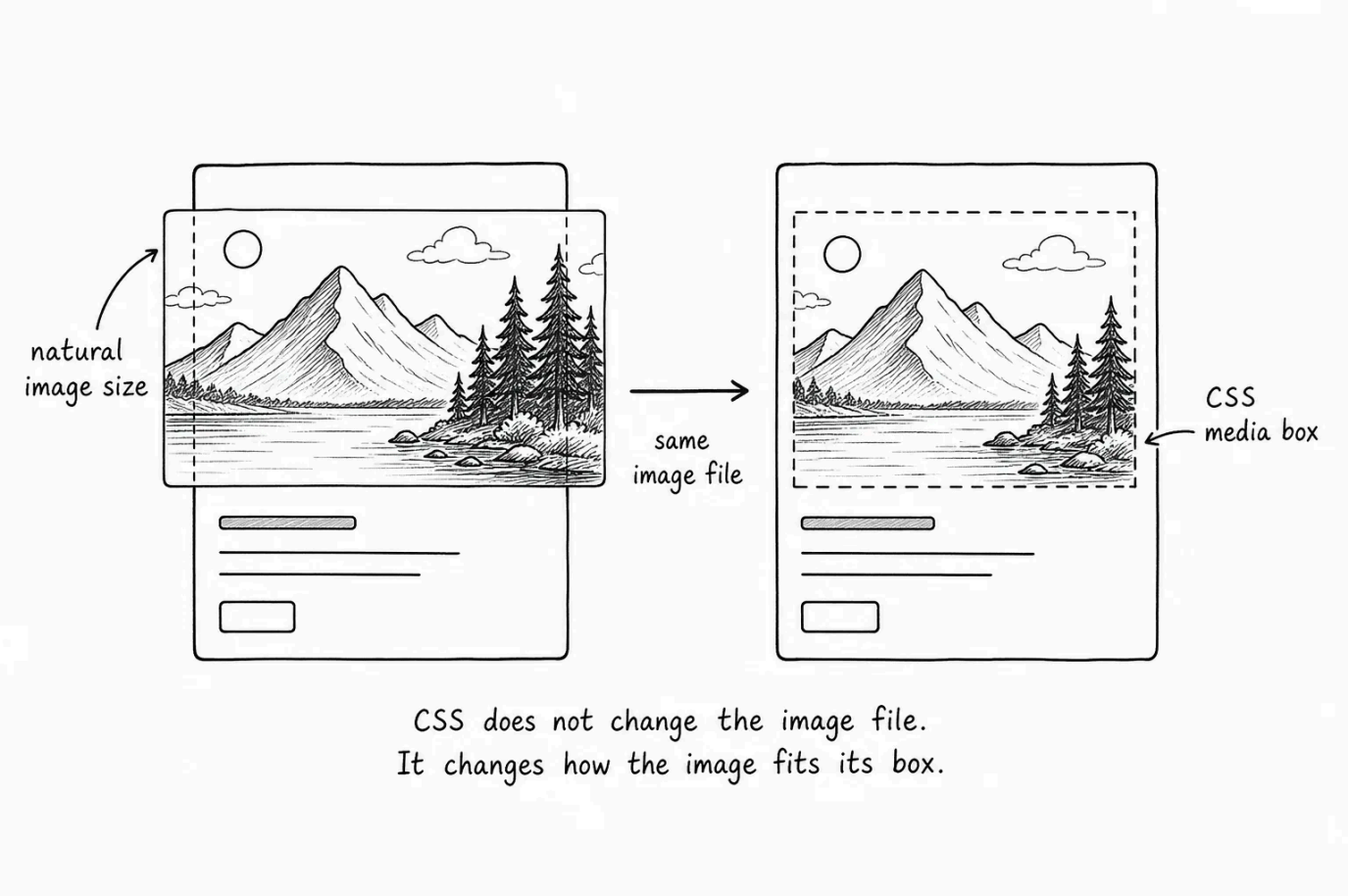
.promo .eyebrow {
  color: #bfdbfe;
```



```
}
```

Open the page and you will notice that the image may look too large and awkward inside the card. That is not a failure. It is the image showing at its natural size.

CSS needs to tell the image how to fit the card.



Images have a natural size

An image file has its own dimensions. A photo might be **1600px** wide and **1000px** tall. An icon might be **64px** wide and **64px** tall. If you do not give the browser any CSS sizing instructions, the image uses its natural size.

The usual first fix is to give the image a fluid width and let its height follow naturally:

```
CSS

.course-image {
  display: block;
  width: 100%;
  height: auto;
  border-radius: 8px 8px 0 0;
}
```

Reload the page and the image should now fit inside the card.

- width: 100%** makes the image fill the width of the card.
- height: auto** preserves the image's natural aspect ratio, so the image does not stretch.

display: block connects back to the last lesson. Images are inline by default, which can leave a tiny bit of space under them. Making the image

block-level removes that inline spacing behavior and makes the card edge cleaner.

💡 **Start with fluid/responsive images** - For normal content images, `display: block; width: 100%; height: auto;` is a safe beginner pattern. It makes the image fit its container without distorting it.

Crop an image with object-fit

Sometimes you want every card image to have the same shape, even if the original images have different dimensions. You can do that with `aspect-ratio` and `object-fit`. For the sample image, update the style like this:

CSS

```
.course-image {
  display: block;
  width: 100%;
  aspect-ratio: 16 / 9;
  object-fit: cover;
  object-position: center;
  border-radius: 8px 8px 0 0;
}
```

Reload the page and notice how the image now has a consistent wide shape.

`aspect-ratio: 16 / 9` gives the image box a consistent wide shape.

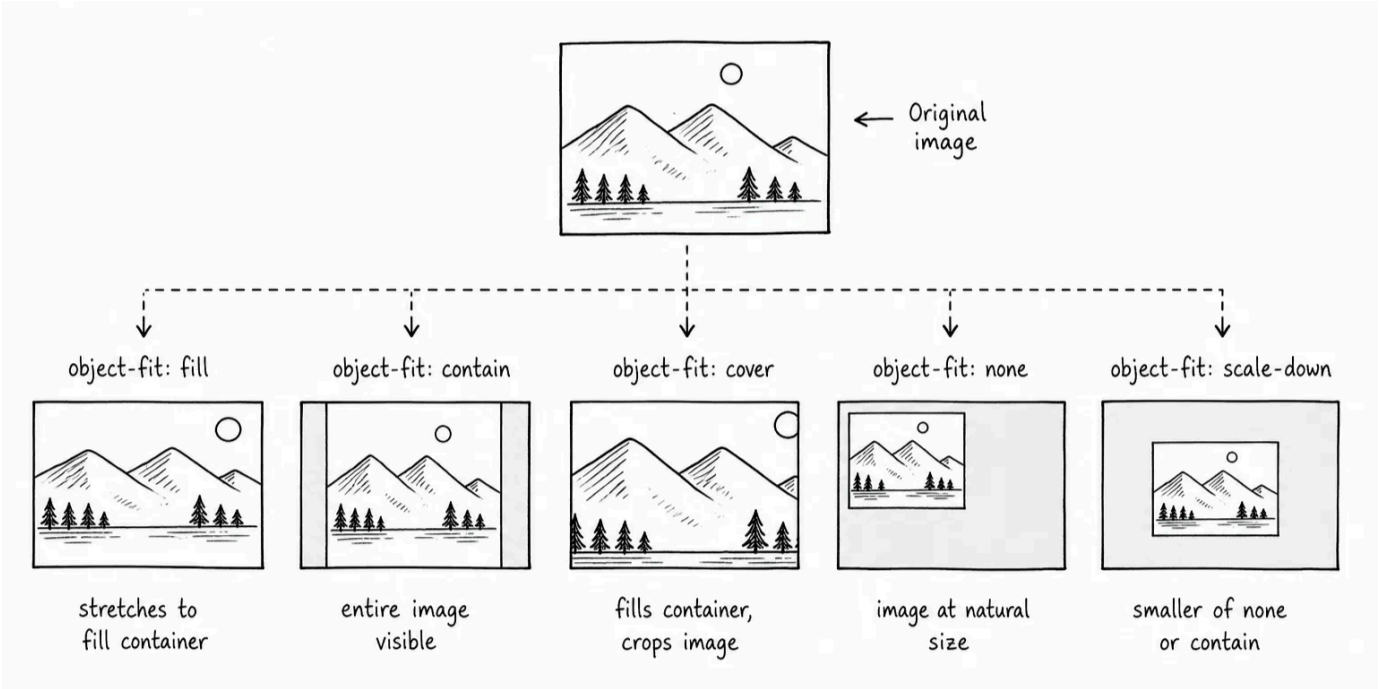
✦ AI Tutor

What is aspect ratio for images in CSS?

>

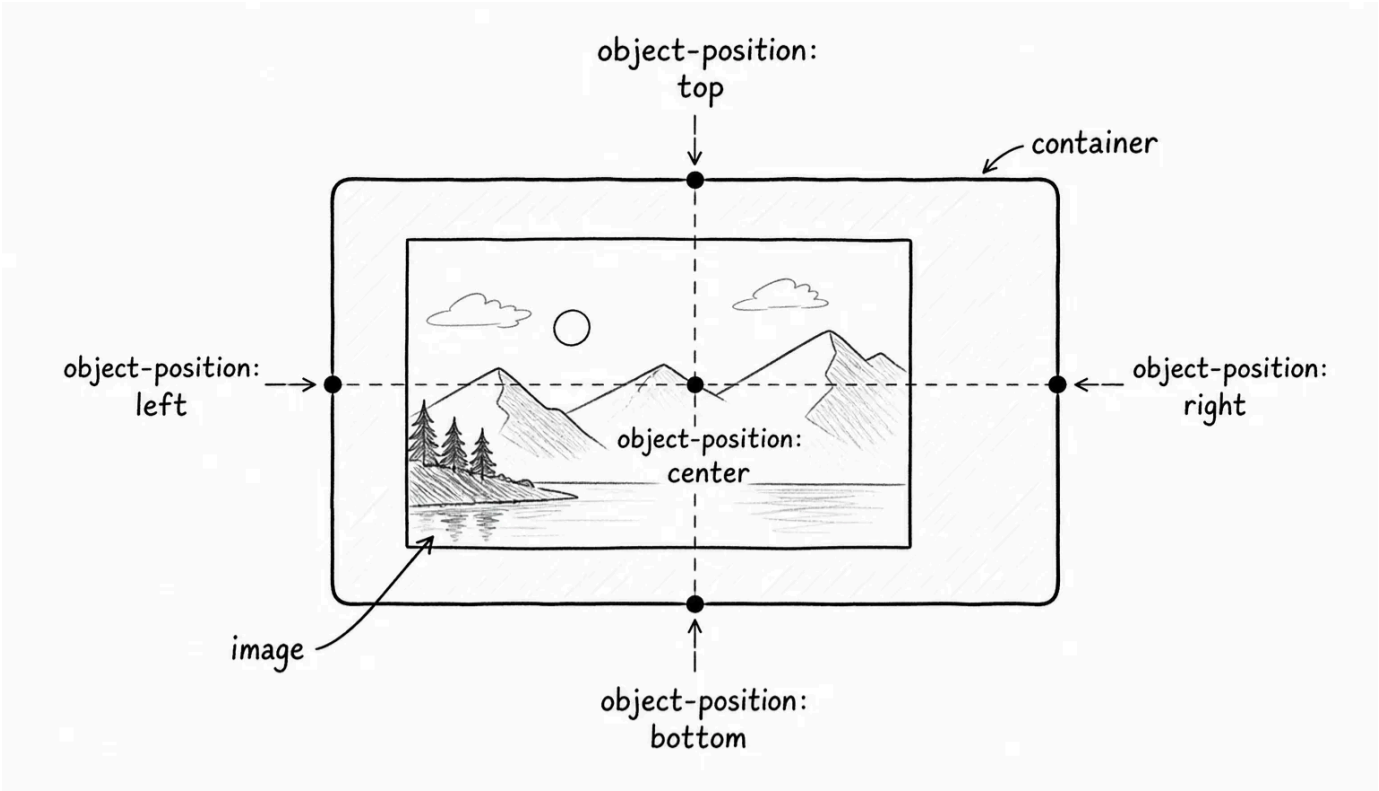
`object-fit: cover` fills that box without stretching the image. If the image does not match the box shape, the browser crops the extra part.

There are other possible values for `object-fit` as well. The image below shows how each behaves for an image inside a container.



`object-position: center` chooses which part of the image stays centered during the crop applied by `object-fit`. The image below shows different

object-position values and their impact.



Try changing **object-position** values and see how they affect the crop.

Avoid stretching images

If you assign a fluid width and set a fixed height, it usually results in an awkward stretched image:

CSS

```
.course-image {  
  width: 100%;  
  height: 240px;  
}
```

Without **object-fit**, the browser stretches the image to fit both dimensions. Faces, logos, and screenshots can look squeezed.

If you need to set a fixed height for an image box, you usually add **object-fit** too:

CSS

```
.course-image {  
  width: 100%;  
  height: 240px;  
  object-fit: cover;  
}
```

That tells the browser to crop instead of distort.

 **Check for stretched images** - If an image looks stretched, check whether both **width** and **height** are being forced without **object-fit**. Cropping is usually better than distortion, but important content can still be cropped away, so inspect the result.

Use background images for decoration

The card image is real content. It is in the HTML as `<img>` , and it has `alt` text.

The HTML also has a `.promo` section near the end. Use the same image as a decorative background with `background-image` :

CSS

```
.promo {
  color: white;
  background-color: #111827;
  background-image: url("course-cover.jpg");
  background-size: cover;
  background-position: center;
  background-repeat: no-repeat;
  border-radius: 8px;
  padding: 32px;
}
```

Notice that background images use `background-size` and `background-position` instead of `object-fit` and `object-position` . Reload the page and you should see the image filling the promo box.

Depending on your image, the text may be hard to read. You can fix that by adding a dark overlay to the background image:

CSS

```
.promo {
  color: white;
  background-color: #111827;
  background-image:
    linear-gradient(rgba(17, 24, 39, 0.78), rgba(17, 24, 39, 0.78)),
    url("course-cover.jpg");
  background-size: cover;
  background-position: center;
  background-repeat: no-repeat;
  border-radius: 8px;
  padding: 32px;
}
```

Reload the page and you should see a slightly darkened background image.

`background-image` paints an image behind the content.

`linear-gradient(fromColor, toColor)` creates a gradient. In this example, both color stops are the same dark transparent color, so it acts like a solid overlay. CSS gradients count as images, which is why they can sit inside `background-image` with `url(...)` .

`background-size: cover` makes the background fill the section.

`background-position: center` chooses which part of the image is centered.

background-repeat: no-repeat prevents the image from tiling if the background does not fully cover the box.

The overlay darkens the image so the text stays readable. You used **rgba()** earlier for transparent colors. This is the same idea applied as a background layer.

 **Background images do not have alt text** - A CSS background image is not content. Screen readers do not get **alt** text for it. If the image is important to understanding the page, use **** in the HTML instead.

Choose **img** or **background-image**

Use an **** when the image is content:

- Product photo.
- Avatar.
- Chart or diagram.
- Screenshot.
- Article image that helps explain the article.

Use **background-image** when the image is decoration:

- Texture.
- Abstract hero background.
- Decorative pattern.
- Image behind text where the text already says the important thing.

The difference is not only visual. It affects accessibility, loading behavior, search engines, and how easy the image is to replace from app data.

You just saw the accessibility angle: background images have no **alt** text. The same choice matters for search engines, which can understand meaningful **** content more easily than decorative backgrounds. It also matters in real apps, where an image from product data, user data, or article data usually belongs in HTML, not hidden inside a CSS file.

Choose meaning first, then style the image to fit that meaning.

Media elements are boxes too

In the [HTML lesson pack](#), you learned about videos and embedded media, and those media elements need sizing too.

You do not need to add this video example to your practice page. It is here so you can recognize the pattern later when you embed videos or third-party players.

For a normal video element, the same responsive idea works:

CSS

```
video {
  display: block;
  width: 100%;
  height: auto;
}
```

For an embedded video inside an iframe, developers often wrap the iframe in a box with an aspect ratio:

html

```
<div class="video-frame">
  <iframe src="https://www.youtube.com/embed/example" title="Course preview"></iframe>
</div>
```

CSS

```
.video-frame {
  aspect-ratio: 16 / 9;
}

.video-frame iframe {
  display: block;
  width: 100%;
  height: 100%;
  border: 0;
}
```

You do not need to memorize every media pattern yet.

The mental model is enough: media has a shape, and CSS controls how that shape fits the box.

Try it

Before moving on, test how the same image behaves under different fitting rules:

- Remove `width: 100%;` from `.course-image` and reload. Notice the image return toward its natural size.
- Add `height: 240px;` to `.course-image` and temporarily remove `object-fit: cover;`. Notice whether the image stretches.

- Change `aspect-ratio: 16 / 9;` to `aspect-ratio: 1 / 1;` . Notice the image box become square.
- Change `object-fit: cover;` to `object-fit: contain;` . Notice that the whole image appears, but the box may show empty space.
- Change `object-position: center;` to `object-position: top;` .
- Remove the `linear-gradient(...)` layer from `.promo` . Notice whether the text is still readable on top of the image.
- Change `background-size: cover;` to `background-size: contain;` . Notice how the background behaves differently from the `<img>` .
- Remove `background-repeat: no-repeat;` from `.promo` . Notice whether the background image repeats.

The goal is to see images as boxes with fitting rules. When an image looks wrong, ask whether the problem is size, aspect ratio, cropping, or whether the image should be content instead of decoration.

< 0 / 13 >

✔ Knew 0 Items

✧ Learnt 0 Items

⏮ Skipped 0 Items

🔄 ResetProgress

Test Your Understanding

What is the difference between an image's natural size and its CSS size?

[Click to Reveal the Answer](#)

✔ Already Know that

✧ Didn't Know that

⏮ Skip Question

✓

LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →

Links, buttons, and states

In the [HTML pack](#), you learned the difference between links and buttons: a link navigates somewhere, while a button performs an action.

CSS can make both of them look polished, but it should not erase that difference. A link styled like a button is still a link. A button styled like plain text is still a button.

This lesson is about styling interactive elements and their states: normal, hover, focus, active, visited, and disabled.

Start with an interactive page

Use a small interactive page with navigation links, a text link, a link styled like a primary action, and a few buttons:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Links and buttons practice</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <main class="page">
      <section class="intro">
        <p class="eyebrow">CSS practice</p>
        <h1>Links, buttons, and states</h1>
        <p class="summary">Interactive elements should look clickable, show focus, and r
      </section>

      <nav class="site-nav" aria-label="Practice navigation">
        <a href="/lessons">Lessons</a>
        <a href="/projects">Projects</a>
        <a href="/account">Account</a>
      </nav>

      <article class="panel">
        <p class="badge">Draft lesson</p>
        <h2>Publish your CSS notes</h2>
        <p class="panel-text">
          Review the <a href="/style-guide">style guide</a> before publishing.
          If the lesson is ready, use the action buttons below.
        </p>

        <div class="actions">
          <a class="primary-link" href="/preview">Preview lesson</a>
          <button class="secondary-button" type="button">Save draft</button>
          <button class="danger-button" type="button">Delete draft</button>
          <button class="secondary-button" type="button" disabled>Syncing</button>
        </div>
      </article>
    </main>
```



```
</body>
</html>
```

Then add the starting styles:

CSS

```
* {
  box-sizing: border-box;
}

body {
  font-family: system-ui, sans-serif;
  color: #111827;
  background-color: #f9fafb;
}

.page {
  max-width: 760px;
  margin: 40px auto;
}

.intro,
.panel {
  background-color: white;
  border: 1px solid #e5e7eb;
  border-radius: 8px;
  padding: 24px;
}

.intro,
.site-nav {
  margin-bottom: 24px;
}

.eyebrow,
.site-nav a {
  color: #2563eb;
  font-weight: 700;
}

.summary,
.panel-text {
  color: #4b5563;
  line-height: 1.6;
}

.badge {
  display: inline-block;
  color: #1d4ed8;
```

```
border-radius: 999px;
font-size: 14px;
font-weight: 700;
}

a {
  color: #2563eb;
  text-decoration-thickness: 2px;
  text-decoration-color: #2563eb;
  text-decoration: underline;
}

.site-nav a {
  display: inline-block;
  margin-right: 12px;
  text-decoration: none;
}

.actions {
  margin-top: 20px;
}

.primary-link,
```



```
.secondary-button,
.danger-button {
  display: inline-block;
  margin-right: 8px;
  margin-bottom: 8px;
  padding: 10px 14px;
  border-radius: 6px;
  font-family: inherit;
  font-size: 16px;
  font-weight: 700;
  text-decoration: none;
}

.primary-link {
  color: white;
  background-color: #2563eb;
}

.secondary-button,
.danger-button {
  background-color: white;
  cursor: pointer;
}

.secondary-button {
  color: #2563eb;
  border: 1px solid #bfdbfe;
}

.danger-button {
  color: #b91c1c;
  border: 1px solid #fecaca;
}
```

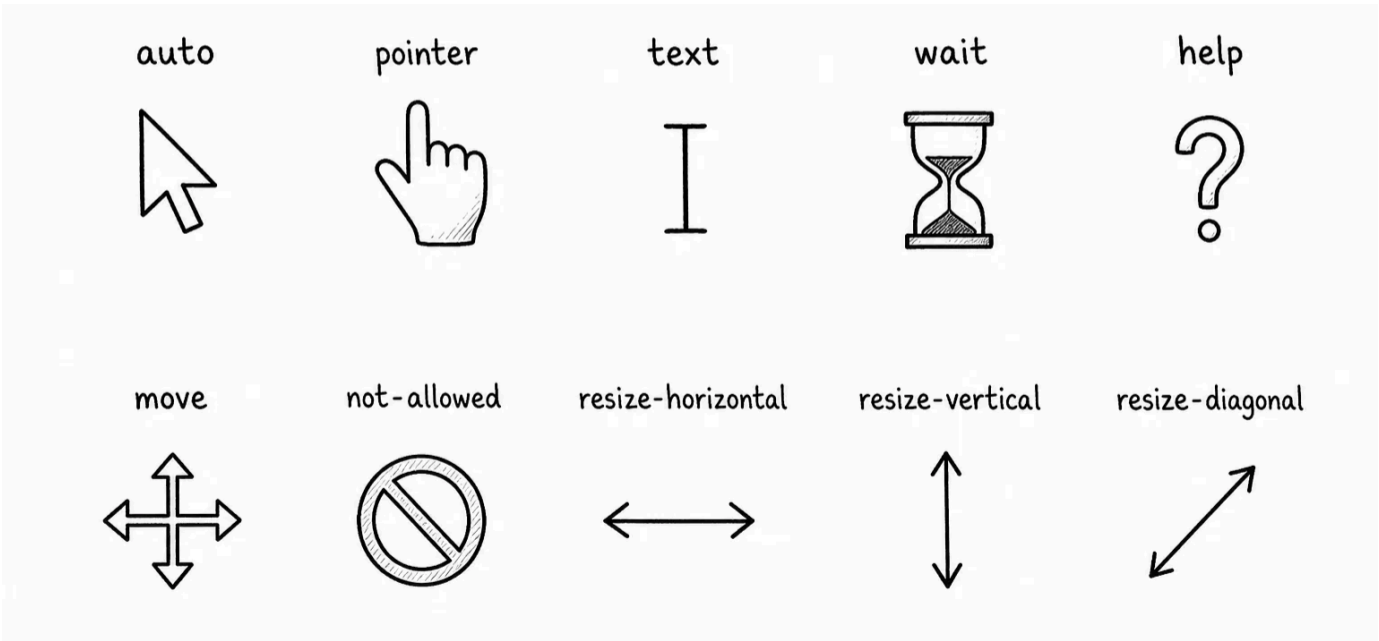
Open the page and you should see normal links, navigation links, a link styled like a primary action, regular buttons, a danger button, and a disabled button.

Right now the controls have base styles only. The page can show what each control is, but it does not yet show how each control responds when someone hovers, tabs, clicks, or reaches a disabled state.

Cursors

If you look closely at the CSS file, you will notice the `cursor` property. We have not used it much yet in this pack. The `cursor: pointer` declaration gives the element the familiar hand pointer.

The image below shows some possible `cursor` values and how each one looks. You can [look at this page](#) for more details about the property and its values.



Links and buttons have different jobs

Start by choosing the right element for the job.

In the practice HTML, links are used for navigation. For example:

html

```
<a href="/preview">Preview lesson</a>
```

Buttons are used for actions, such as saving the current draft:

html

```
<button type="button">Save draft</button>
```

CSS can make the link look like a button:

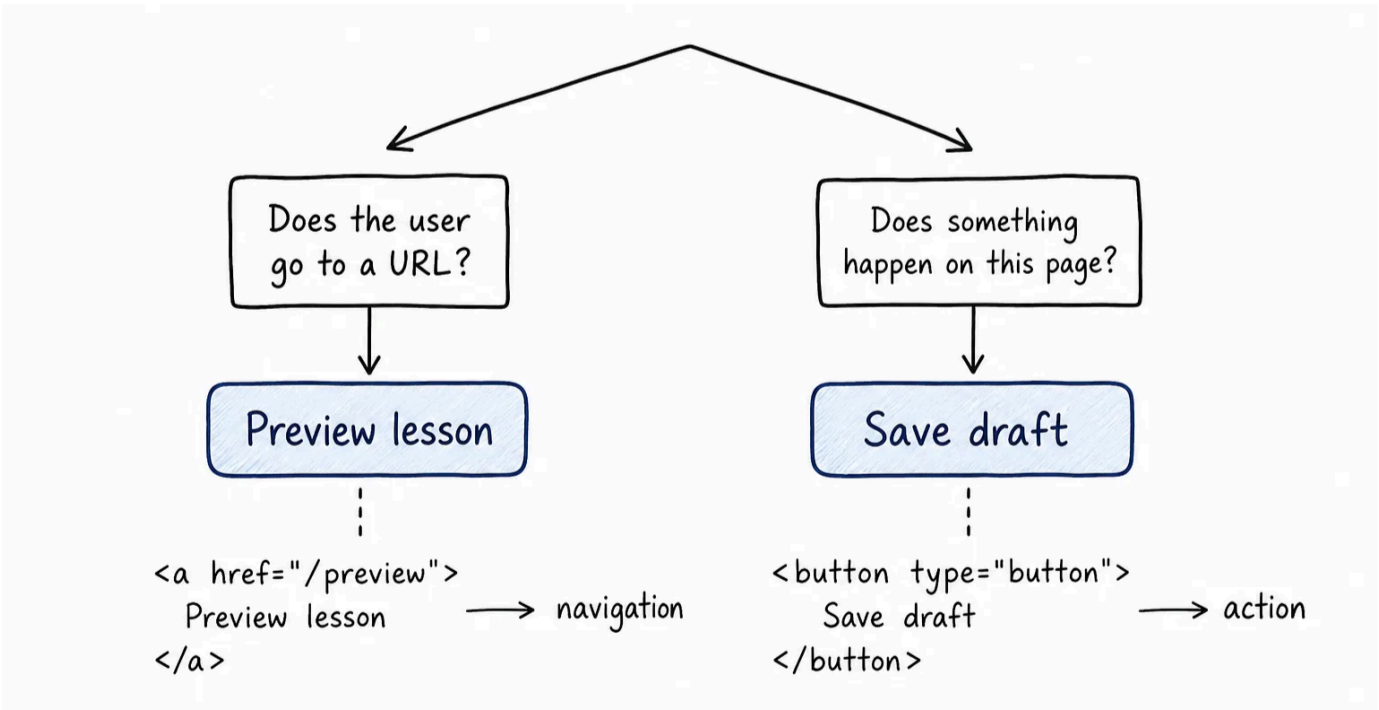
CSS

```
.primary-link {
  color: white;
  background-color: #2563eb;
}
```

But it is still a link because the HTML is `<a href="/preview">` , so it navigates to a URL.

In our example file, the save and delete controls are buttons because they represent actions on the current page.

That choice matters for keyboard behavior, screen readers, browser features, and forms.



💡 **Choose meaning first** - Decide whether the user is navigating or performing an action before styling the element. Use `<a>` for navigation and `<button>` for actions, then use CSS to make it fit the design.

Keep links recognizable

Links should look like links, especially inside paragraphs.

The base rule keeps paragraph links underlined and adjusts the underline:

CSS

```
a {
  color: #2563eb;
  text-decoration-thickness: 2px;
  text-underline-offset: 3px;
}
```

`text-decoration-thickness` controls how thick the underline is. `text-underline-offset` controls how far the underline sits from the text. Together, they keep the link recognizable while making the underline feel more deliberate than the browser default.

For navigation links, removing the underline can be okay because their placement and grouping already signal that they are links:

CSS

```
.site-nav a {
  display: inline-block;
  margin-right: 12px;
  text-decoration: none;
}
```

If you remove an underline, make sure the link still looks clickable through color, placement, spacing, or another clear cue.

⚠ **Do not hide links in plain text** - If a link inside a paragraph has the same color and decoration as the surrounding text, people may not notice it, so keep body links visibly link-like.

State selectors match interaction states

A state selector is a selector that matches an element in a specific condition.

The practice page currently has base styles, but it does not have state styling yet. The controls do not visibly respond when someone hovers, tabs, presses, or reaches a disabled state. CSS handles these conditions with pseudo-classes.

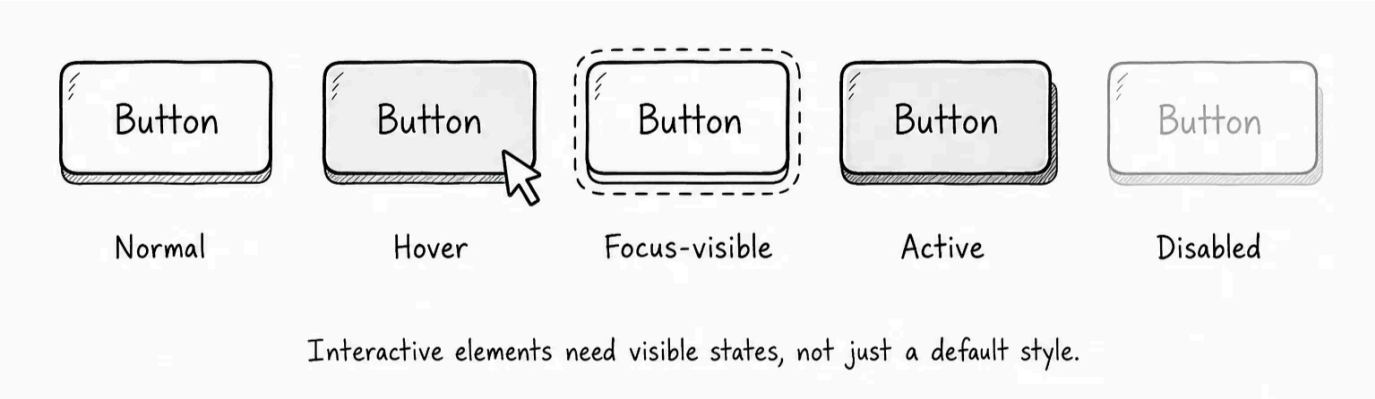
For example, `:hover` is a pseudo-class that matches when the pointer is over an element:

CSS

```
a:hover {
  color: #1d4ed8;
}
```

The colon is the clue: selectors like `:hover`, `:focus-visible`, `:active`, `:visited`, and `:disabled` are called pseudo-classes.

The image below shows common states an interactive control can have. The next sections style those states one at a time:



State selectors are still selectors, so they follow the cascade from Lesson 7. Source order still matters when two state rules set the same property on the same element.

Add hover states

`:hover` matches when the user's pointer is over an element. Add these hover styles:

CSS

```
a:hover {
  color: #1d4ed8;
}
```

```
.primary-link:hover {
  color: white;
  background-color: #1d4ed8;
}

.secondary-button:hover {
  background-color: #eff6ff;
}

.danger-button:hover {
  background-color: #fef2f2;
}
```

Reload the page and move your pointer over the links and buttons.

The first rule changes the color of normal links on hover. The other rules change the background color of the button-like controls. The link styled as `.primary-link` also keeps `color: white;` in its hover rule so the text stays readable on the darker blue background.

Hover states should be noticeable, but they do not need to be dramatic. A small color or background change is usually enough.

Hover is useful for pointer users. It is not enough on its own because keyboard and touch users may never see it.

Add keyboard focus states

Focus is the state an element enters when it is selected for keyboard interaction.

Press `Tab` in the browser, and the focus should move through the navigation links, the action link, and the buttons.

The disabled `Syncing` button should be skipped. Disabled controls cannot receive focus, which is one reason they need a clear disabled style.

Use `:focus-visible` for a keyboard-friendly focus style:

CSS

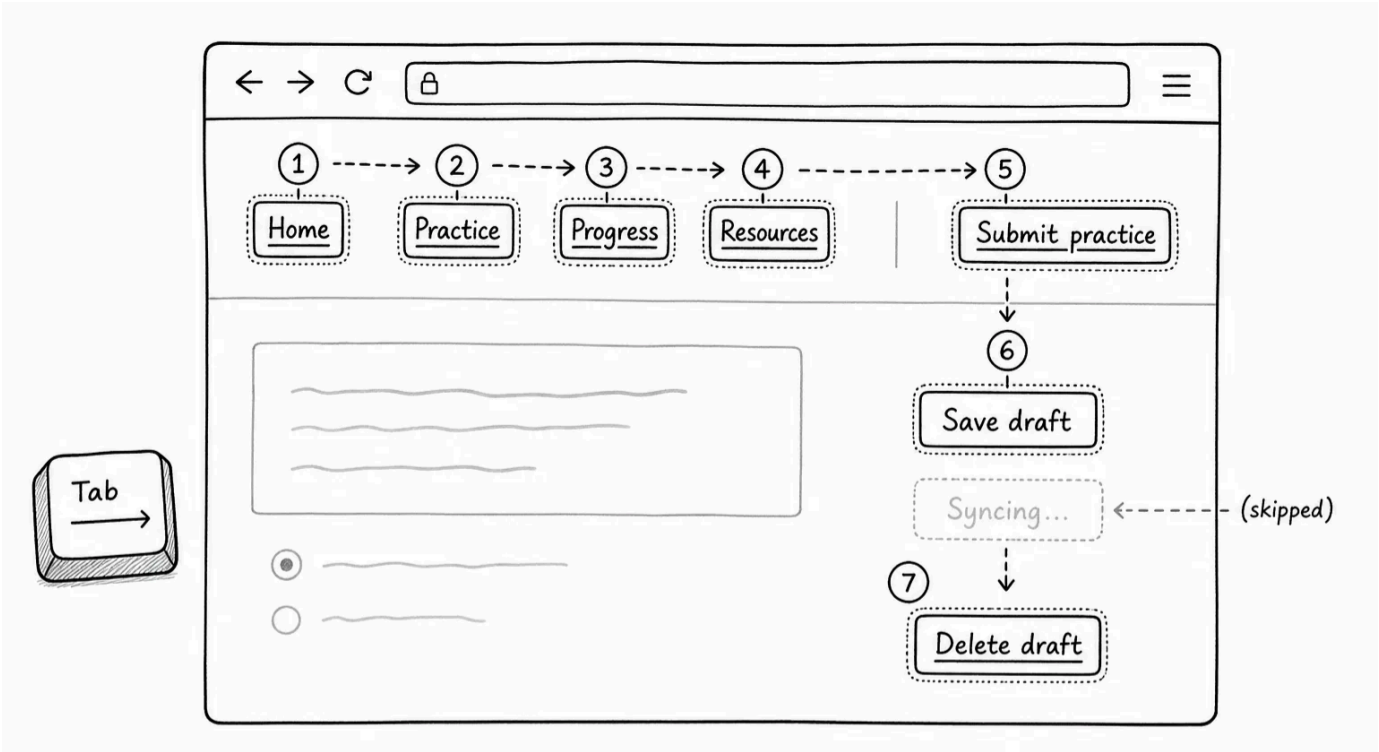
```
a:focus-visible,
button:focus-visible {
  outline: 3px solid #93c5fd;
  outline-offset: 3px;
}
```

Reload the page and press `Tab` again. The focused element should be easy to find.

You may also see `:focus` in older CSS. `:focus` matches whenever the element has focus, including after some mouse clicks. `:focus-visible` matches when the browser thinks a visible focus ring is useful, usually during

keyboard navigation. For modern code, `:focus-visible` is usually the better default.

`outline` draws a focus ring outside the element without changing the element's size. `outline-offset` moves the ring away from the edge so it is easier to see.



⚠ Do not remove focus without replacing it - Avoid writing `outline: none;` unless you provide another clear focus style. Keyboard users need to see where focus is.

Add active states

`:active` matches while the user is pressing the element. Add this small pressed state:

```
CSS

.primary-link:active,
.secondary-button:active,
.danger-button:active {
  transform: translateY(1px);
}
```

Click and hold one of the controls, and it should feel slightly pressed.

The `transform` property can move, scale, or rotate an element visually. Here it moves the element down by `1px`. You will learn more about transforms later, but this small use is common for press feedback.

Keep active states subtle because the goal is to show that the click registered, not to make the layout jump around.

Style disabled buttons

The final button has the `disabled` attribute:

html

```
<button class="secondary-button" type="button" disabled>Syncing</button>
```

A disabled button cannot be clicked or focused, so style it with `:disabled` :

CSS

```
button:disabled {
  opacity: 0.55;
  cursor: not-allowed;
}

button:disabled:hover {
  background-color: white;
}
```

Reload the page and the disabled button should look unavailable.

`opacity: 0.55;` fades the whole button. `cursor: not-allowed;` changes the pointer cue when someone hovers over it. The `button:disabled:hover` rule keeps the disabled button from picking up the normal hover background.

Do not use color alone to communicate disabled state. Lower opacity, cursor change, and text like `Syncing` help make the state clearer.

Links do not have a real `disabled` attribute. If a navigation option should not be available, it is often better to remove the link, explain why it is unavailable, or use a button if the control is truly an action.

Visited links can look different

Browsers can style links the user has already visited with `:visited` :

CSS

```
.panel-text a:visited {
  color: #6d28d9;
}

.panel-text a:hover {
  color: #1d4ed8;
}
```

This can help users remember which pages they have already opened.

The selector above targets normal text links inside `.panel-text` . That keeps the visited color away from button-style links like `.primary-link` , where a purple visited color could clash with the blue background.

There is one cascade detail worth knowing: a link can be visited and hovered at the same time. When state selectors have the same specificity, source order decides. That is why `.panel-text a:hover` comes after `.panel-text`

a:visited in the example. The hover color can still win while the user is interacting with a visited text link.

A common order for link states is **:link**, **:visited**, **:hover**, **:active**, often remembered as LVHA. The **:link** state means an unvisited link. You do not need to write every one of those states today, but it helps to recognize the order when reading CSS.

Browsers limit what you can style with **:visited** for privacy reasons. Color changes are the main safe use. You do not need to use **:visited** for every site, but it is worth recognizing when you see it.

Put the states together

The page now has a small state system:

CSS

```
.panel-text a:visited {
  color: #6d28d9;
}

a:hover {
  color: #1d4ed8;
}

.panel-text a:hover {
  color: #1d4ed8;
}

a:focus-visible,
button:focus-visible {
  outline: 3px solid #93c5fd;
  outline-offset: 3px;
}

.primary-link:hover {
  color: white;
  background-color: #1d4ed8;
}

.secondary-button:hover {
  background-color: #eff6ff;
}

.danger-button:hover {
  background-color: #fef2f2;
}

.primary-link:active,
.secondary-button:active,
.danger-button:active {
  transform: translateY(1px);
}

button:disabled {
  opacity: 0.55;
  cursor: not-allowed;
}

button:disabled:hover {
  background-color: white;
}
```


The important habit is not memorizing every state. It is remembering that interactive elements need visible feedback for different ways people interact with them.

Try it

Before moving on, test the page like a user would. Use the pointer, use the keyboard, and deliberately break a few states so you can feel why they matter:

- Hover over the paragraph link, navigation links, primary action link, and buttons.
- Press **Tab** to move through the interactive elements. Confirm that every focused element has a visible ring.
- Temporarily remove the **a:focus-visible, button:focus-visible** rule. Press **Tab** again and notice how much harder the page is to use. Then add it back.
- Change the hover background on **.primary-link:hover** to a much lighter blue. Check whether the white text is still readable.
- Add **disabled** to the delete button in the HTML and reload. Confirm that **button:disabled** styles it too.
- Add a new navigation link and check whether the base link, hover, and focus styles apply without writing a new class.
- Add another link inside **.panel-text** and check whether the scoped visited style belongs to text links, not button-style links.

The goal is to make interaction visible. If a user can click, tap, or tab to something, its states should be clear.

< 0 / 15 >

Knew

0 Items

Learnt

0 Items

Skipped

0 Items

ResetProgress

Test Your Understanding

When should you use a link?

Click to Reveal the Answer

✔ Already Know that

⚡ Didn't Know that

⏩ Skip Question



LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →

Flexbox layout

In one of the previous lessons, you used `display: flex` just enough to make `gap` work. Now it is time to learn what Flexbox is actually doing.

Flexbox is a layout mode for arranging items in one direction at a time: across a row or down a column. It is useful when a group of items needs spacing, alignment, wrapping, or equal sharing of available space.

Think about the layouts you have already seen:

- A header with text on the left and an action on the right.
- A row of filter buttons.
- A group of cards that should sit beside each other when there is room.
- A small card footer where metadata and a link should line up.

Those are the kinds of problems Flexbox is designed to solve.

Start with the layout problem

A page can have good colors, spacing, borders, and states, but still feel awkward if grouped items do not line up.

In our example webpage below, the header action should sit beside the title, the filter buttons should have consistent space between them, and the lesson cards should wrap when the screen gets narrow.

Use this HTML as `index.html` :

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Flexbox practice</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <main class="page">
      <header class="page-header">
        <div>
          <p class="eyebrow">CSS practice</p>
          <h1>Plan your next CSS session</h1>
          <p class="summary">Flexbox helps groups of items line up, share space, and wrap</p>
        </div>

        <a class="header-action" href="/new">New lesson</a>
      </header>
```



```
<section class="toolbar" aria-label="Lesson filters">
  <button class="filter is-active" type="button">All</button>
  <button class="filter" type="button">Layout</button>
  <button class="filter" type="button">Practice</button>
  <button class="filter" type="button">Review</button>
</section>

<section class="lesson-list">
  <article class="lesson-card">
    <p class="card-label">Layout</p>
    <h2>Flexbox basics</h2>
    <p>Use a row or column layout when a group of items needs alignment and spacing.</p>
    <div class="card-footer">
      <span>7 min</span>
      <a class="card-link" href="/flexbox">Open</a>
    </div>
  </article>

  <article class="lesson-card">
    <p class="card-label">Practice</p>
    <h2>Spacing decisions</h2>
    <p>Compare padding, margin, and gap so each kind of space has a clear job.</p>
    <div class="card-footer">
      <span>9 min</span>
      <a class="card-link" href="/spacing">Open</a>
    </div>
  </article>

  <article class="lesson-card">
    <p class="card-label">Review</p>
    <h2>DevTools check</h2>
    <p>Inspect the final layout and confirm which elements became flex items.</p>
    <div class="card-footer">
      <span>5 min</span>
      <a class="card-link" href="/devtools">Open</a>
    </div>
  </article>
</section>
</main>
</body>
</html>
```

Add this starting CSS to `style.css` . It uses styling you already know: colors, spacing, borders, rounded corners, and button-like links. The layout is still

CSS

```
* {
  box-sizing: border-box;
}

body {
  font-family: system-ui, sans-serif;
  color: #111827;
  background-color: #f9fafb;
}

.page {
  max-width: 920px;
  margin: 40px auto;
}

.page-header,
.lesson-card {
  background-color: white;
  border: 1px solid #e5e7eb;
  border-radius: 8px;
  padding: 24px;
}

.eyebrow,
.card-label {
```

```
color: #2563eb;
font-size: 14px;
font-weight: 700;
}

h1,
h2,
p {
  margin-top: 0;
}

.summary {
  color: #4b5563;
  line-height: 1.6;
  max-width: 560px;
}

.header-action,
.filter,
.card-link {
  border-radius: 6px;
  font-family: inherit;
  font-size: 16px;
  font-weight: 700;
}

.header-action,
.card-link {
  color: #2563eb;
  text-decoration-thickness: 2px;
  text-underline-offset: 3px;
}

.header-action {
  display: inline-block;
  color: white;
  background-color: #2563eb;
  padding: 10px 14px;
  text-decoration: none;
}

.toolbar {
  margin: 24px 0;
}

.filter {
  color: #2563eb;
  background-color: white;
  border: 1px solid #bfdbfe;
  padding: 10px 14px;
  cursor: pointer;
}

.filter.is-active {
  color: white;
  background-color: #2563eb;
  border-color: #2563eb;
}

.lesson-list {
  margin-top: 24px;
}

.lesson-card {
  margin-bottom: 16px;
}

.card-footer {
  margin-top: 18px;
  color: #6b7280;
  font-size: 14px;
}
```

Open the page in your browser. The header action sits after the text, the buttons depend on normal inline spacing, and the cards stack in one column.

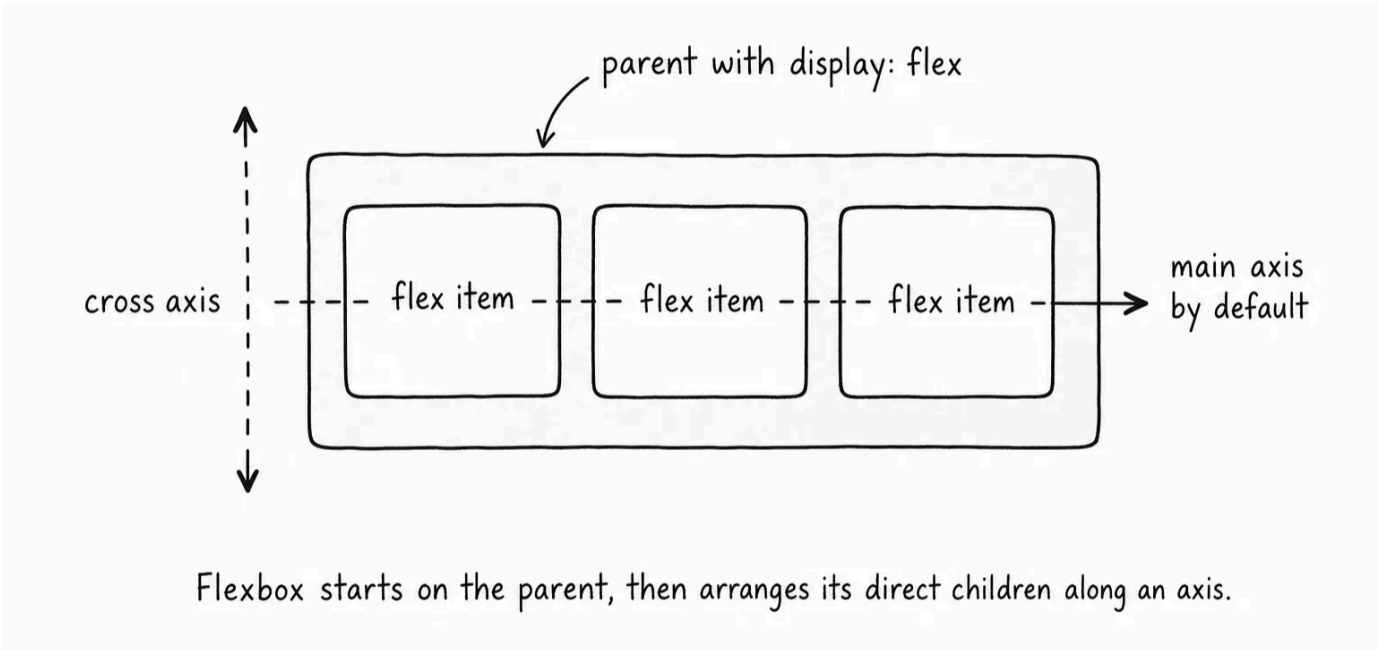
Nothing is broken, but the page is not using layout intentionally yet.

Flexbox starts on the parent

Flexbox is a parent-child layout mode. You put `display: flex` on a parent element, and the direct children of that parent become flex items.

That direct-child detail matters. In our webpage, if you put Flexbox on `.lesson-list`, the `.lesson-card` elements become flex items. If you put Flexbox on `.card-footer`, the time label and link inside that footer become flex items.

By default, when you apply `display: flex` on an element, flexbox arranges the direct children in a row. That row is called the **main axis**. The opposite direction is called the **cross axis**.



Put the header on one row

The header has two direct children: the text group and the `New lesson` link. Those two pieces belong on the same row, with space between them.

Add Flexbox to the header:

```
css

.page-header {
  display: flex;
  justify-content: space-between;
  align-items: flex-start;
  gap: 16px;
}
```

`display: flex` turns `.page-header` into a flex container. Its two direct children now sit in a row.

`justify-content` controls the main axis. In the default row direction, that means horizontal distribution. `space-between` pushes the first item to the start, the last item to the end, and leaves the extra space between them.

align-items controls the cross axis. In a row, that means vertical alignment.

flex-start keeps the action link aligned near the top of the header instead of stretching or floating awkwardly.

gap creates space between flex items. It is safer than adding margin to only one item because the spacing belongs to the group, not to a specific child.

 **Read Flexbox from the parent** - When you see a Flexbox rule, first ask: which parent has **display: flex**, and which direct children is it arranging?

Use gap for button rows

The filter buttons are a group. The space between them should belong to the group, so **gap** is a better fit than random margins on each button.

Add Flexbox to **.toolbar** :

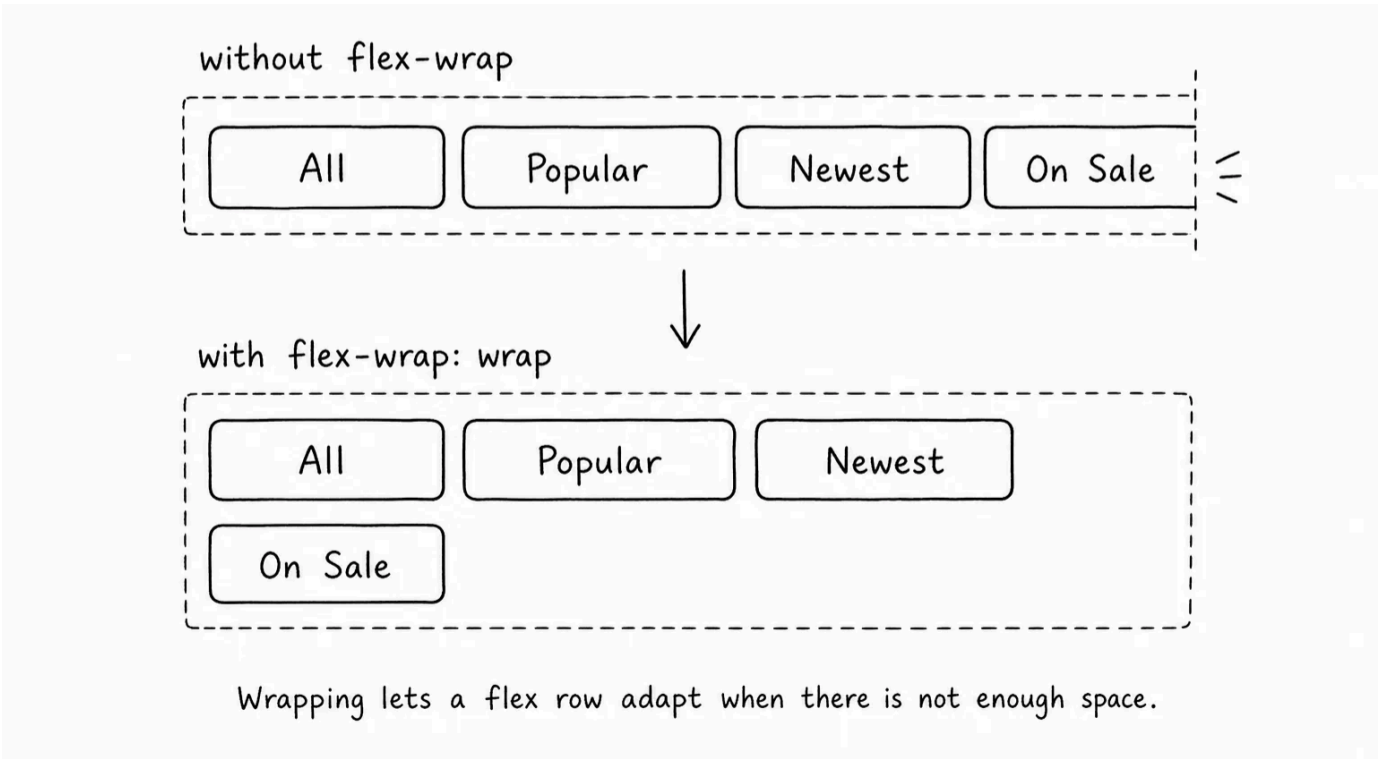
CSS

```
.toolbar {
  display: flex;
  gap: 8px;
  flex-wrap: wrap;
  margin: 24px 0;
}
```

display: flex makes the buttons flex items. **gap: 8px** creates consistent space between them.

flex-wrap: wrap lets items move onto a new line when there is not enough horizontal space. Without wrapping, Flexbox tries to keep items on one line, which can make a row feel cramped or overflow on smaller screens. In the current HTML, you may not feel the impact immediately, so try adding more items to **.toolbar** and compare the row with and without **flex-wrap**.

Alternatively, resize the browser window. The filter buttons should wrap more gracefully when the row runs out of space.



Let cards share available space

The lesson cards currently sit in a vertical list. They should sit beside each other when there is room, but they should not become too narrow. This is where Flexbox becomes more useful than fixed widths.

Add Flexbox to the card list:

```
CSS

.lesson-list {
  display: flex;
  gap: 16px;
  flex-wrap: wrap;
  margin-top: 24px;
}

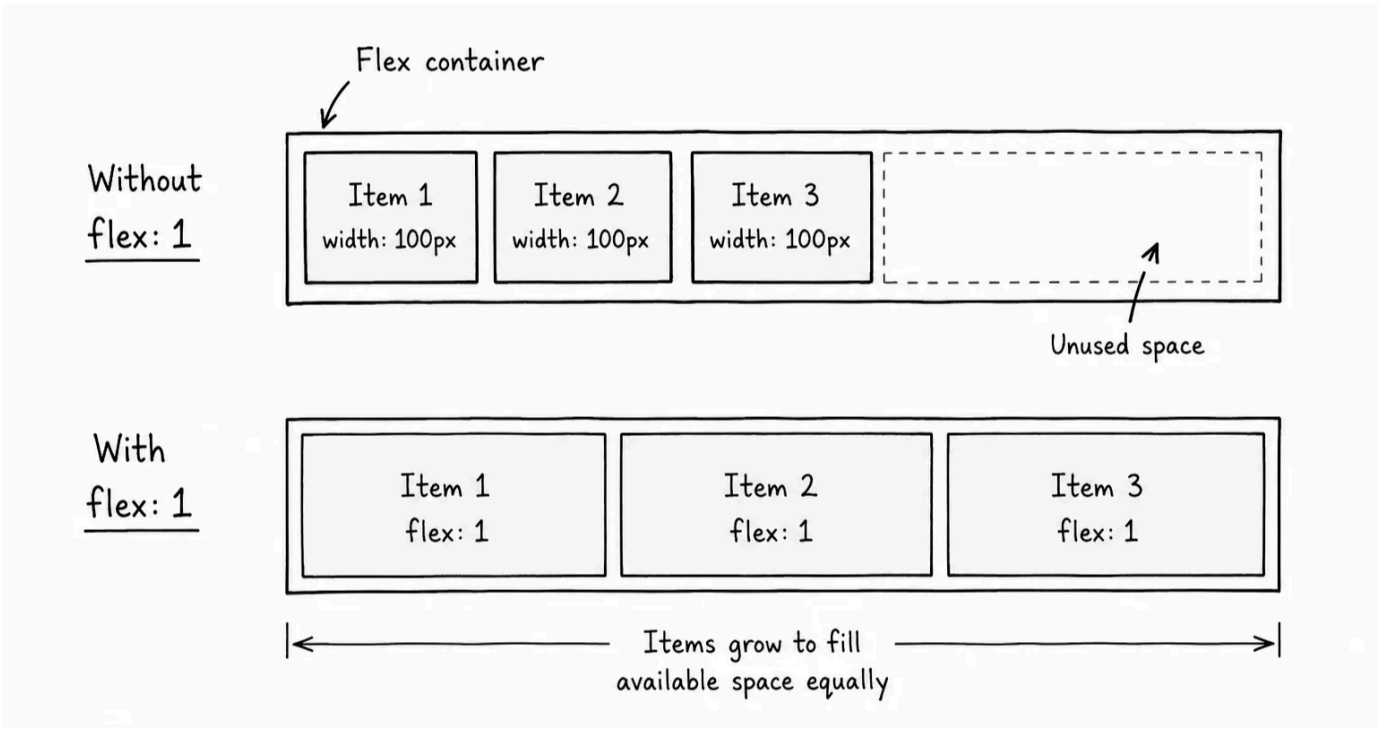
.lesson-card {
  flex: 1;
  min-width: 220px;
  margin-bottom: 0;
}
```

.lesson-list is the parent, so its direct children, the cards, become flex items.

flex: 1 tells each card it can grow and share the available space with the other cards. For now, read it as "let this item take an equal share of the row."

min-width: 220px protects each card from becoming too narrow. When there is not enough room for all cards to stay at least **220px** wide, **flex-wrap: wrap** lets the next card move to a new line.

margin-bottom: 0 removes the old vertical spacing from the stacked version. Once the cards are in a flex group, **gap** owns the spacing between them.



⚠ **Flexbox only arranges direct children** - If `display: flex` does not affect the elements you expected, inspect the HTML. You may have put Flexbox on the wrong parent. Only direct children of the flex container become flex items.

Align small rows inside a card

Flexbox is not only for big page sections. It is also useful for small rows inside components.

Each card has a footer with two direct children:

html

```
<div class="card-footer">
  <span>7 min</span>
  <a class="card-link" href="/flexbox">Open</a>
</div>
```

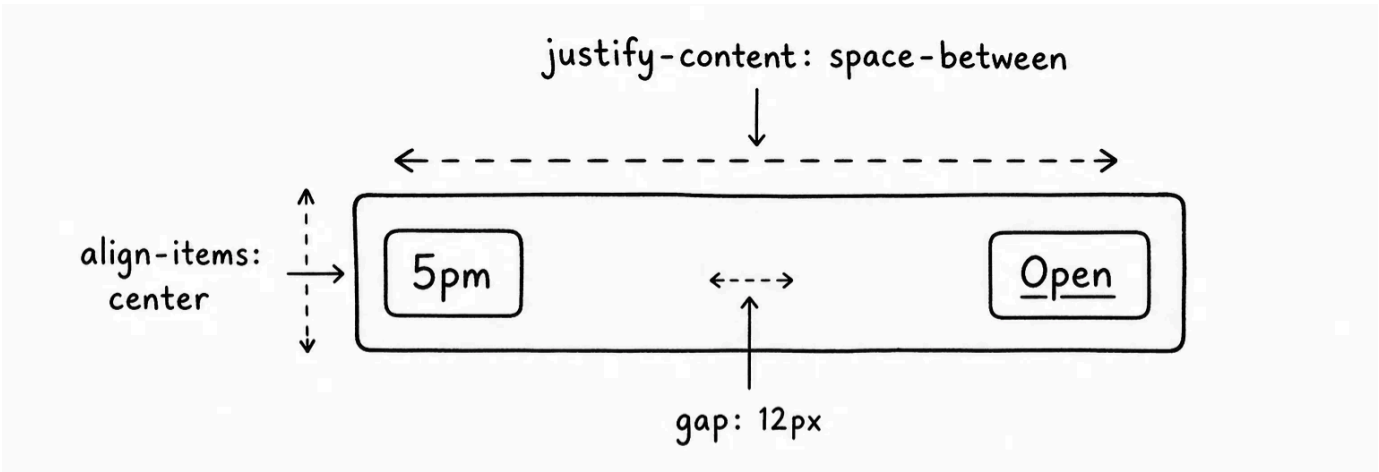
The time label and the link are direct children of `.card-footer`, so `.card-footer` is the parent that needs Flexbox:

CSS

```
.card-footer {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 12px;
  margin-top: 18px;
  color: #6b7280;
  font-size: 14px;
}
```

`justify-content: space-between` pushes the time label and link apart, `align-items: center` lines them up vertically, and `gap: 12px` keeps a little space between them if the row gets tight.

This is the same idea as the header, just on a smaller parent.



Row or column

Flexbox uses a row by default:

CSS

```
.toolbar {
  display: flex;
  flex-direction: row;
}
```

You can switch the main axis with **flex-direction** :

CSS

```
.toolbar {
  display: flex;
  flex-direction: column;
  gap: 8px;
}
```

flex-direction: column stacks the direct children vertically, but keeps Flexbox features like **gap** and alignment. Use a row when items should sit beside each other. Use a column when items should stack but still behave like a managed group.

Flexbox is best for one direction at a time. If you need strong control over rows and columns together, that is usually a grid layout problem, which is the next lesson.

Keep the HTML order sensible - CSS layout should not make the visual order tell a different story from the HTML order. Keyboard users, screen readers, and anyone reading the source still depend on a sensible document order.

Put the Flexbox rules together

Here are the Flexbox rules you added:

CSS

```
.page-header {
  display: flex;
}
```

```
justify-content: space-between;
align-items: flex-start;
gap: 16px;
}

.toolbar {
  display: flex;
  gap: 8px;
  flex-wrap: wrap;
  margin: 24px 0;
}

.lesson-list {
  display: flex;
  gap: 16px;
  flex-wrap: wrap;
  margin-top: 24px;
}

.lesson-card {
  flex: 1;
  min-width: 220px;
  margin-bottom: 0;
}

.card-footer {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 12px;
  margin-top: 18px;
  color: #6b7280;
  font-size: 14px;
}
```

The pattern is the same each time: choose the parent, decide whether its direct children should form a row or column, then use `gap` , `justify-content` , `align-items` , wrapping, or flexible item sizing to solve the layout problem.

 AI Tutor

I am learning Flexbox. Quiz me with small HTML examples and ask me which element should get display: flex. Then ask me whether the problem needs gap, justify-content, align-items, flex-wrap, or flex: 1. Focus on reasoning from the parent and direct children.

>

Try it

Test Flexbox by changing one layout decision at a time:

- Remove `display: flex` from `.page-header` . Notice that `justify-content` , `align-items` , and `gap` no longer create the header layout.
- Change `.page-header` from `justify-content: space-between;` to `justify-content: flex-start;` , then to `justify-content: center;` .
- Change `.page-header` from `align-items: flex-start;` to `align-items: center;` .
- Remove `flex-wrap: wrap;` from `.toolbar` , then make the browser narrower.

- Change `.lesson-card min-width` from `220px` to `300px` . Notice when the cards wrap.
- Temporarily remove `.lesson-card flex: 1;` . Notice that the cards stop sharing the row in the same way.
- Add a fourth `.lesson-card` and check whether the layout still works without writing a new class.
- Inspect `.lesson-list` in DevTools and confirm that the `.lesson-card` elements are the flex items.

The goal is to stop guessing at layout. When a group of items needs to line up, ask which parent owns the group, which children should be arranged, and which axis the layout should use.

< 0 / 11 >

✔ Knew 0 Items

✳️ Learnt 0 Items

⏮ Skipped 0 Items

🔄 ResetProgress

Test Your Understanding

What does display: flex do?

[Click to Reveal the Answer](#)

✔ Already Know that

✳️ Didn't Know that

⏮ Skip Question

✓

LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →

Grid layout

In the last lesson, Flexbox helped you arrange items in one direction: a row of buttons, a row of cards, or a small footer inside a card.

Grid is for layouts where rows and columns matter at the same time.

Use Grid when you want a page region, dashboard, gallery, pricing table, or card layout to have a clear two-dimensional structure. Flexbox asks, "How should this group flow along one axis?" Grid asks, "What columns and rows should this space have?"

Start with a page that needs structure

The easiest way to feel Grid is to take a page that almost works and give it structure. This dashboard has a sidebar, a workspace, stat cards, and lesson cards. Without Grid, those pieces stack; with Grid, the page gets columns, rows, and deliberate gaps.

Use this as `index.html` :

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Grid practice</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <main class="page">
      <section class="intro">
        <p class="eyebrow">CSS practice</p>
        <h1>Organize a learning dashboard</h1>
        <p class="summary">Grid helps a page use rows and columns instead of only stacking</p>
      </section>

      <section class="dashboard">
        <aside class="sidebar">
          <h2>Topics</h2>
          <a href="/layout">Layout</a>
          <a href="/spacing">Spacing</a>
          <a href="/debugging">Debugging</a>
        </aside>

        <section class="workspace">
          <section class="stats" aria-label="Lesson stats">
            <article class="stat-card">
              <span class="stat-number">12</span>
              <span class="stat-label">Lessons</span>
            </article>

            <article class="stat-card">
              <span class="stat-number">4</span>
              <span class="stat-label">Projects</span>
            </article>
          </section>
        </section>
      </main>
    </body>
  </html>
```



```

    </article>

    <article class="stat-card">
      <span class="stat-number">2</span>
      <span class="stat-label">Reviews</span>
    </article>
  </section>

  <section class="lesson-grid">
    <article class="lesson-card featured-card">
      <p class="card-label">Featured</p>
      <h2>Build a responsive card grid</h2>
      <p>Practice columns, gaps, and spanning so the layout feels intentional.</p>
    </article>

    <article class="lesson-card">
      <p class="card-label">Layout</p>
      <h2>Flexbox recap</h2>
      <p>Review parent-child layout, direct children, and one-direction alignmer</p>
    </article>

    <article class="lesson-card">
      <p class="card-label">Grid</p>
      <h2>Track sizing</h2>
      <p>Use fixed and flexible column sizes to divide space.</p>
    </article>

    <article class="lesson-card">
      <p class="card-label">DevTools</p>
      <h2>Inspect grid lines</h2>
      <p>Use the browser overlay to see columns, rows, and gaps.</p>
    </article>
  </section>
</section>
</main>
</body>
</html>
```

Now add the non-grid styling to `style.css` . This gives the page readable text, cards, borders, and spacing. The actual grid layout comes later.

CSS

```
* {
  box-sizing: border-box;
}

body {
  font-family: system-ui, sans-serif;
  color: #111827;
  background-color: #f9fafb;
}

.page {
  max-width: 1040px;
  margin: 40px auto;
}

.intro,
.sidebar,
.stat-card,
.lesson-card {
  background-color: white;
  border: 1px solid #e5e7eb;
  border-radius: 8px;
  padding: 24px;
}

.intro {
  margin-bottom: 24px;
}

.eyebrow,
```



```
.card-label {
  color: #2563eb;
  font-size: 14px;
  font-weight: 700;
}

h1,
h2,
p {
  margin-top: 0;
}
```

```

  line-height: 1.6;
  max-width: 640px;
}

.sidebar {
  margin-bottom: 24px;
}

.sidebar a {
  display: block;
  color: #2563eb;
  font-weight: 700;
  margin-top: 10px;
  text-decoration-thickness: 2px;
  text-underline-offset: 3px;
}

.stat-card,
.lesson-card {
  margin-bottom: 16px;
}

.stat-number {
  display: block;
  color: #2563eb;
  font-size: 32px;
  font-weight: 800;
}

.stat-label {
  color: #6b7280;
  font-weight: 700;
}

.lesson-card p {
  color: #4b5563;
  line-height: 1.6;
}
```

Open the page. It should be readable, but the layout is still a stack: intro, sidebar, stats, and lesson cards all flow downward. That is the problem Grid will solve.

Grid starts on the parent

Grid is a parent-child layout mode, like Flexbox. You put `display: grid` on a parent, and the direct children become grid items.

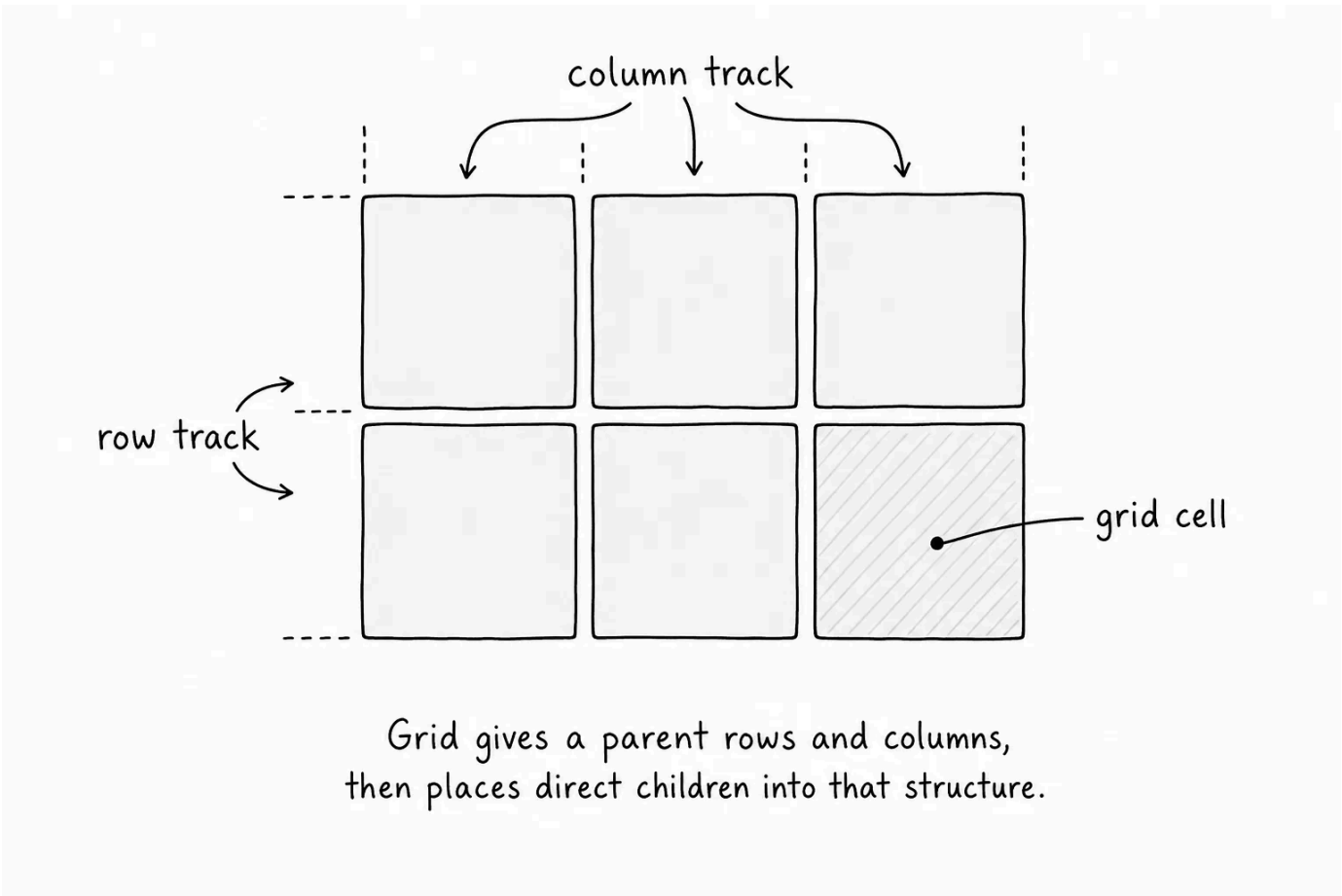
The difference is what the parent can control. A flex container controls one main direction. A grid container can define columns and rows.

CSS

```
.some-parent {
```

```
display: grid;
}
```

Grid divides a container into tracks. A column is a vertical track. A row is a horizontal track. The spaces where rows and columns cross are grid cells.



Split the dashboard into columns

The dashboard has two direct children: `.sidebar` and `.workspace`. Those two pieces should sit beside each other, so the `.dashboard` parent needs Grid.

Add Grid to the dashboard parent:

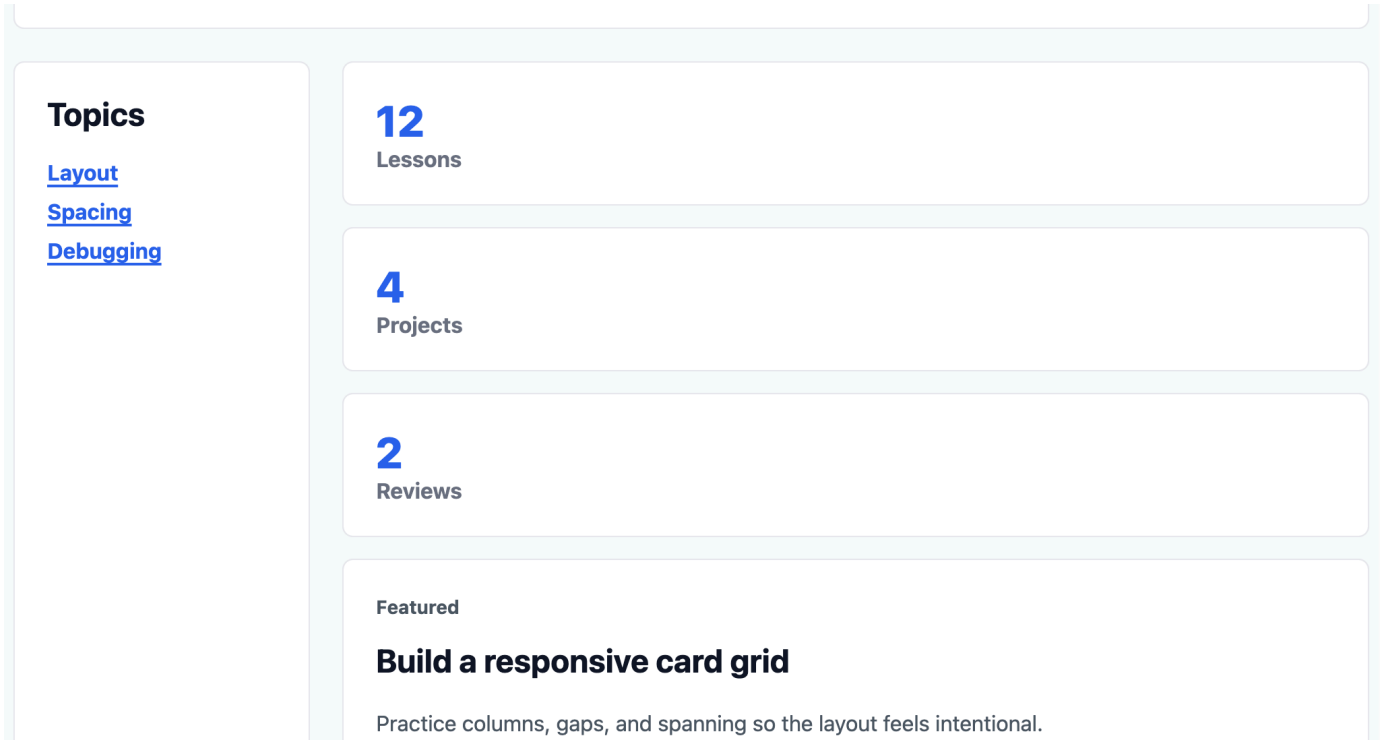
CSS

```
.dashboard {
  display: grid;
  grid-template-columns: 220px 1fr;
  gap: 24px;
}

.sidebar {
  margin-bottom: 0;
}
```

`display: grid` turns `.dashboard` into a grid container.

`grid-template-columns` defines the columns. In this case, the first column is `220px` wide for the sidebar. The second column is `1fr` for the workspace.



`1fr` means a fraction of the available space. After the fixed `220px` sidebar and the gap are accounted for, `1fr` takes the remaining space.

`gap` creates space between grid tracks. You already used `gap` with Flexbox. In Grid, it creates space between rows and columns.

The `.sidebar` had `margin-bottom` in the stacked version. Once the dashboard uses a grid gap, that old margin is no longer the right spacing tool, so the rule sets it back to `0`.

This layout is intentionally desktop-shaped for now. The next lesson will teach responsive rules for smaller screens.

Build equal stat columns

The stat cards are another grid problem. There are three cards, and each should get an equal column.

Add Grid to the stats parent:

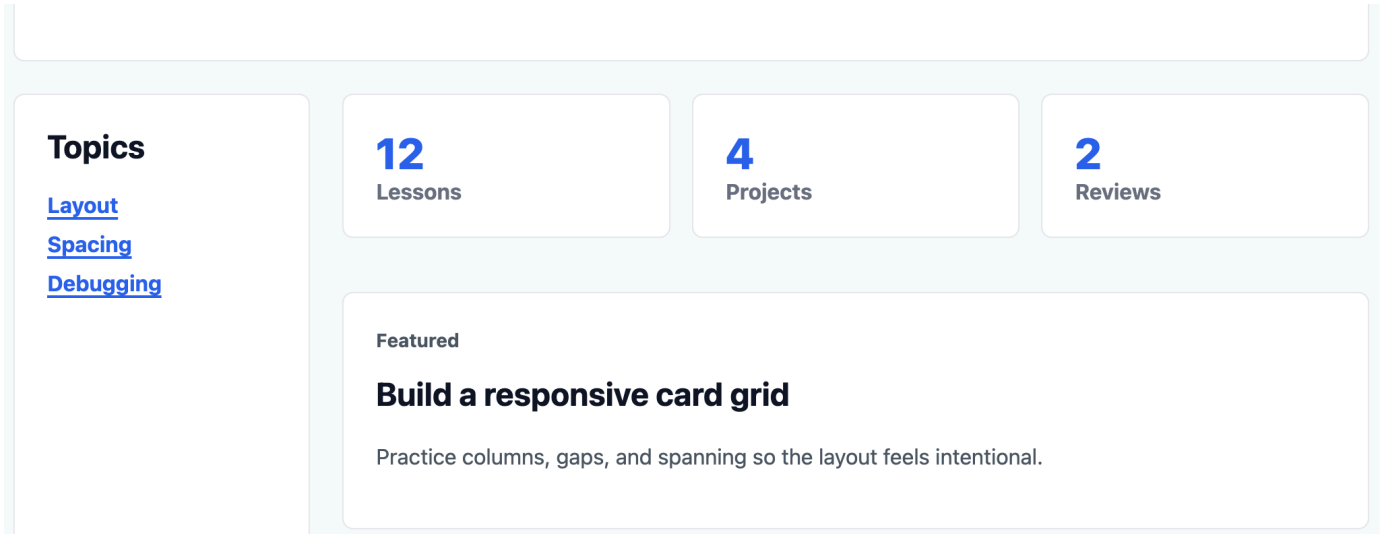
```
CSS

.stats {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 16px;
  margin-bottom: 24px;
}

.stat-card {
  margin-bottom: 0;
}
```

`display: grid` makes `.stats` the parent grid container. Its direct children, the three `.stat-card` elements, become grid items.

`grid-template-columns: repeat(3, 1fr);` creates three equal columns. You could write `1fr 1fr 1fr`, but `repeat(3, 1fr)` says the same thing with less repetition.



Each `1fr` column gets one share of the available space. Because all three columns use the same value, they are equal. Try playing with CSS by writing `grid-template-columns: 1fr 1fr 1fr;` and `grid-template-columns: 1fr 2fr 1fr;` and see what each produces.

The stats also use `gap` for the space between cards. The old `margin-bottom` on each `.stat-card` is removed because the grid parent now owns the spacing.

Let cards flow into rows

Grid can create rows automatically. If a grid has two columns and four cards, the first two cards fill the first row, and the next two cards move into the second row.

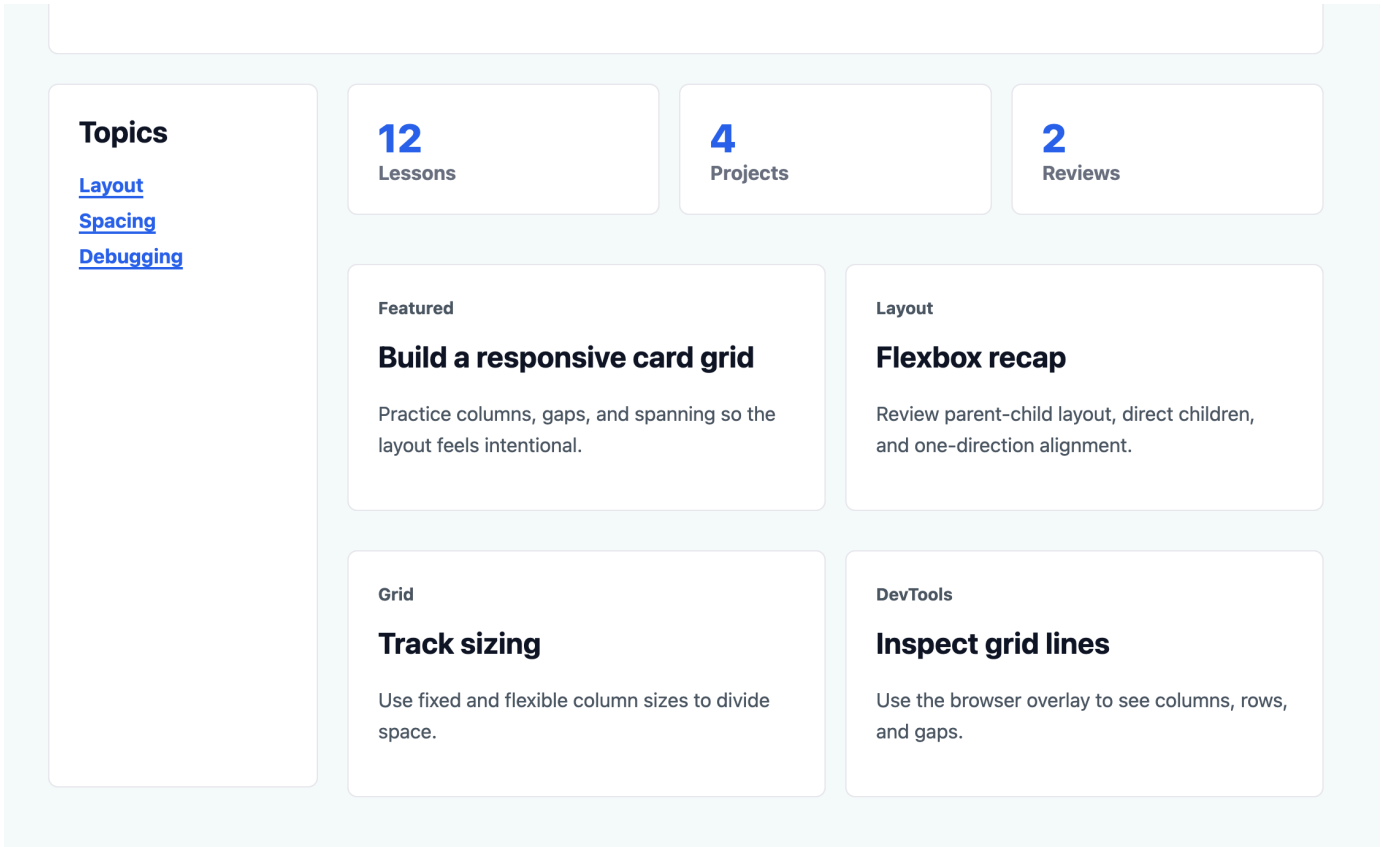
Add a two-column grid for the lesson cards:

CSS

```
.lesson-grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 16px;
}

.lesson-card {
  margin-bottom: 0;
}
```

The `.lesson-grid` parent defines two equal columns. The cards are placed into those columns in HTML order.



You did not define rows here. The browser creates the rows as needed because there are more grid items than columns. This is called auto-placement: the browser places items into the next available grid cell.

💡 **Define the columns first** - In many beginner Grid layouts, you define the columns and let the rows appear automatically. That is enough for card grids, galleries, dashboards, and many content sections.

Make one card span columns

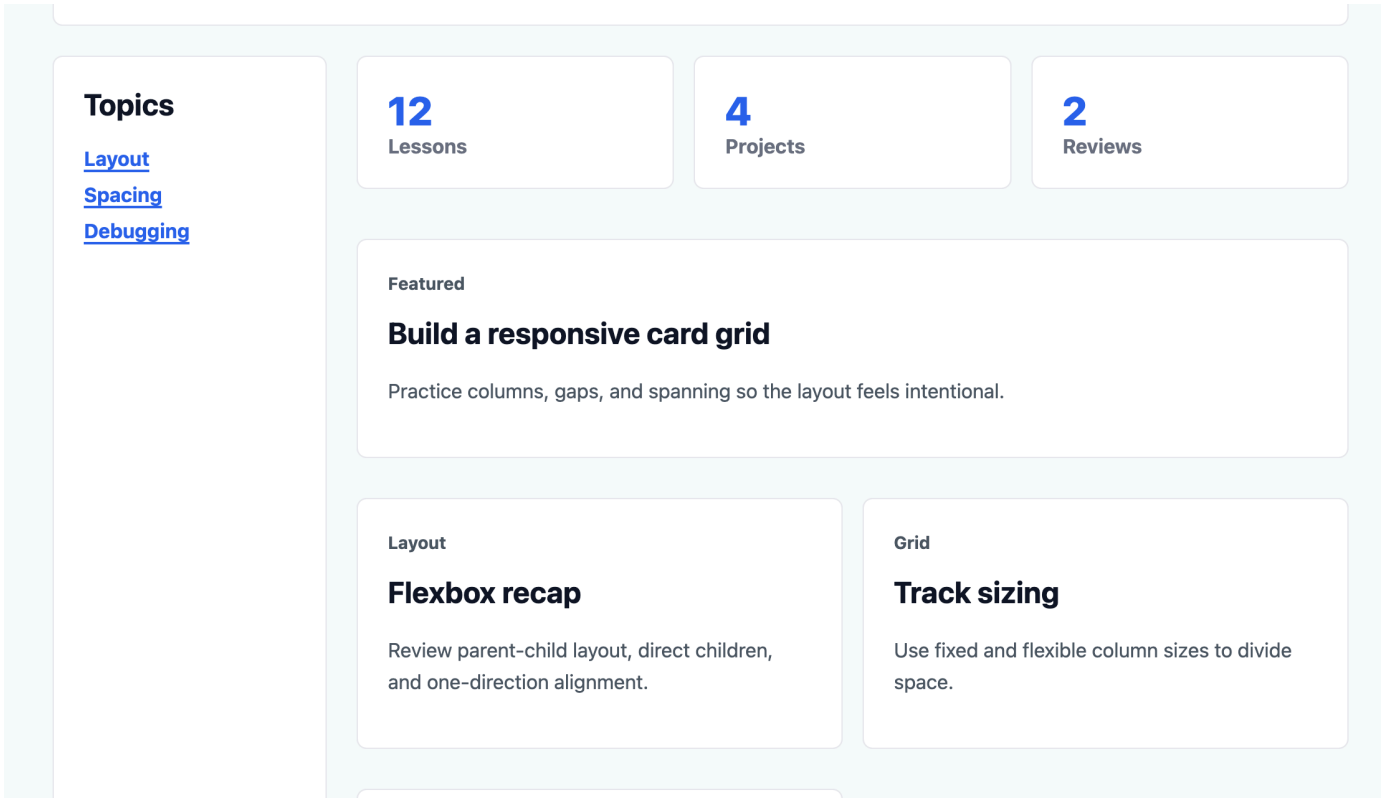
Sometimes one item should take more space than the others. In this page, the featured card can span both columns to feel more important.

Add the spanning rule:

CSS

```
.featured-card {  
  grid-column: span 2;  
}
```

grid-column controls how a grid item uses columns. **span 2** means "take up two columns." Because **.lesson-grid** has two columns, the featured card becomes a full-width card across that grid.



The item still keeps its place in the HTML order. Grid changes where the item appears visually inside the grid, but it does not change the document meaning.

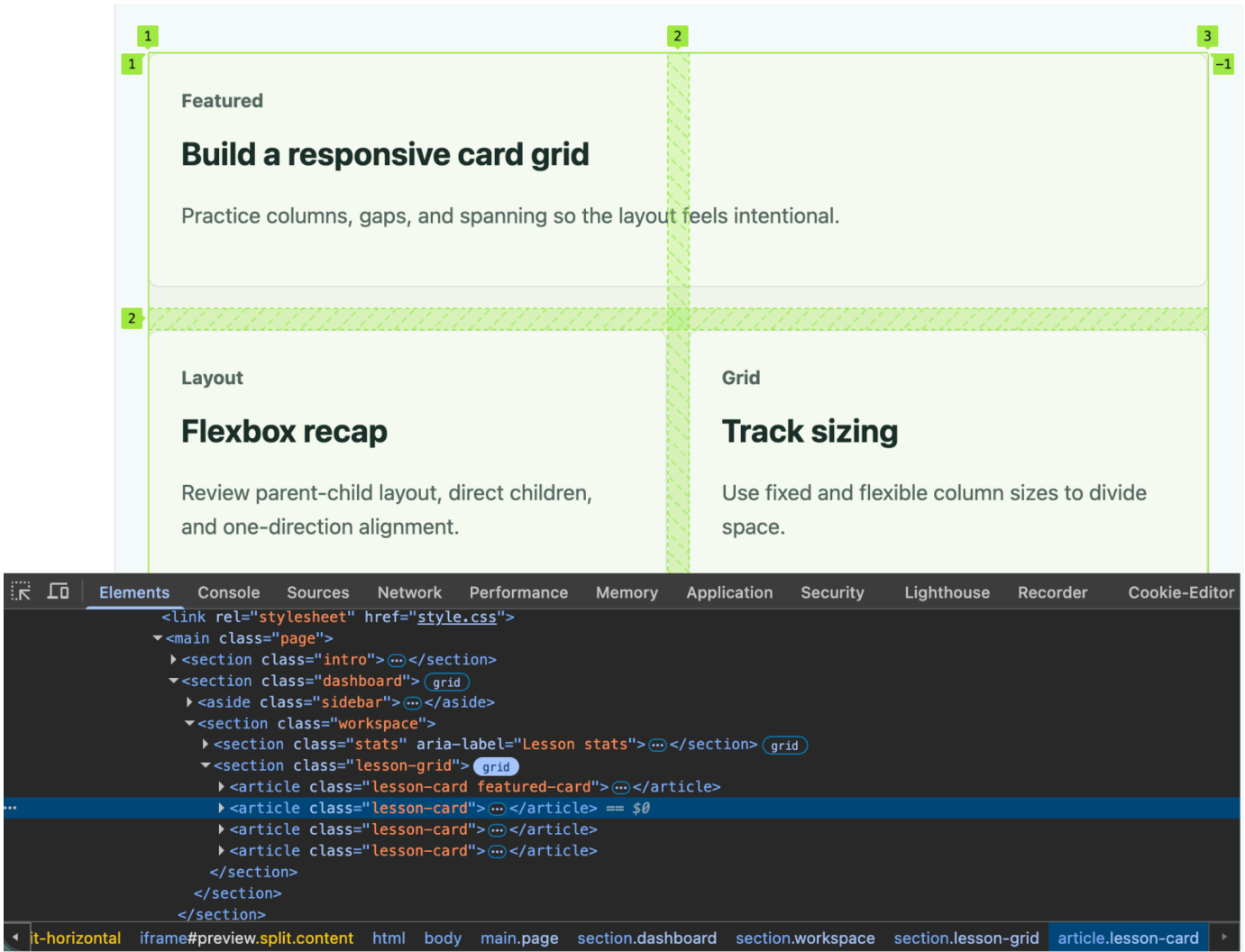
Inspect the grid in DevTools

Grid is much easier to understand when you can see the tracks.

Inspect `.lesson-grid` in DevTools. Most modern browsers show a small `grid` badge near elements with `display: grid`. Turn on the grid overlay if your browser offers it.

The overlay can show column lines, row lines, cells, and gaps. That makes it easier to confirm which parent owns the grid and where the direct children are being placed.

This is the same debugging habit from earlier lessons: inspect the actual element before guessing.



Flexbox or Grid

Flexbox and Grid are both layout tools, but they solve different problems.

Problem	Usually choose
A row of buttons	Flexbox
A card footer with text on one side and a link on the other	Flexbox
A sidebar plus main content	Grid
A card gallery with rows and columns	Grid
A dashboard with sections lined up in columns	Grid

The short version: use Flexbox when the layout mainly flows in one direction. Use Grid when the layout needs rows and columns working together.

Put the Grid rules together

Here are the Grid rules you added:

```
CSS

.dashboard {
  display: grid;
  grid-template-columns: 220px 1fr;
  gap: 24px;
}

.sidebar {
  margin-bottom: 0;
}
```

```
.stats {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 16px;
  margin-bottom: 24px;
}

.stat-card {
  margin-bottom: 0;
}

.lesson-grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 16px;
}

.lesson-card {
  margin-bottom: 0;
}

.featured-card {
  grid-column: span 2;
}
```

The pattern is similar to Flexbox: choose the parent, turn on the layout mode, and let the direct children become layout items. The Grid-specific part is that you define columns and rows instead of only arranging along one axis.

Try it

Test Grid by changing the structure on purpose:

- Remove `display: grid` from `.dashboard`. Notice that `grid-template-columns` and `gap` no longer create the sidebar layout.
- Change `.dashboard` from `grid-template-columns: 220px 1fr;` to `grid-template-columns: 280px 1fr;`.
- Change `.stats` from `repeat(3, 1fr)` to `repeat(2, 1fr)`. Notice how the third card moves to a new row.
- Remove `gap` from `.lesson-grid`, then add it back.
- Remove `.featured-card { grid-column: span 2; }`. Notice that the featured card becomes the same width as the other cards.
- Add a fifth `.lesson-card` and reload. Watch the browser create another row automatically.
- Inspect `.lesson-grid` in DevTools and turn on the grid overlay if your browser supports it.

The goal is to see Grid as a structure tool. When a layout needs rows and columns, ask which parent owns the grid, how many columns it needs, and whether any items should span more space.



Knew

0 Items



Learnt

0 Items



Skipped

0 Items



ResetProgress

Test Your Understanding

What does display: grid do?

[Click to Reveal the Answer](#)



Already Know that



Didn't Know that



Skip Question



LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →

Responsive design

Responsive design means a page can adapt when the available space changes. It is about making the same content work in a narrow browser, a wide monitor, and the sizes in between.

A responsive page usually needs a few habits:

- Let the main container shrink instead of forcing a fixed width.
- Let images and media fit their parent.
- Start with a simple layout that works on small screens.
- Add wider layouts only when there is enough room.
- Test by resizing the browser, not by guessing.

Start with the narrow screen first

This lesson uses a small workshop page. The page has a headline, an image, a pair of action links, and three feature cards.

The important move is the order: first make the page comfortable in one column. Then add a wider layout with a media query.

Use this as `index.html` :

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Responsive practice</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <main class="page">
      <section class="hero-card">
        <div class="hero-copy">
          <p class="eyebrow">CSS practice</p>
          <h1>Design a page that adapts</h1>
          <p class="summary">A responsive page should stay readable and usable as the br

          <div class="actions">
            <a class="primary-action" href="/start">Start practice</a>
            <a class="secondary-action" href="/notes">Read notes</a>
          </div>
        </div>

        
        <article class="feature-card">
```




```
<h2>Fluid containers</h2>
<p>Let the page use available space without forcing one fixed width.</p>
</article>

<article class="feature-card">
  <h2>Flexible media</h2>
  <p>Let images shrink with their parent so they do not overflow.</p>
</article>

<article class="feature-card">
  <h2>Useful breakpoints</h2>
  <p>Change the layout when the content needs more room, not because of a device
</article>
</section>
</main>
</body>
</html>
```

The **meta viewport** line in the **head** matters:

html

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

It tells mobile browsers to use the real device width as the layout width. Without it, a phone browser may pretend the page is much wider than the screen, then shrink the whole page down. That makes responsive CSS harder to reason about.

Then add the base CSS:

```
^ {
  box-sizing: border-box;
}

body {
  font-family: system-ui, sans-serif;
  color: #111827;
  background-color: #f9fafb;
}

.page {
  max-width: 960px;
  margin: 32px auto;
  padding: 0 16px;
}

.hero-card,
.feature-card {
  background-color: white;
  border: 1px solid #e5e7eb;
  border-radius: 8px;
  padding: 24px;
}

.hero-card {
  margin-bottom: 20px;
}

.eyebrow {
  color: #2563eb;
  font-size: 14px;
  font-weight: 700;
}

h1,
```

```
h2,
p {
  margin-top: 0;
}

h1 {
  font-size: 36px;
  line-height: 1.1;
}

.summary,
.feature-card p {
  color: #4b5563;
  line-height: 1.6;
}

.actions {
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
  margin: 20px 0;
}

.primary-action,
.secondary-action {
  border-radius: 6px;
  font-weight: 700;
  padding: 10px 14px;
  text-decoration: none;
}

.primary-action {
  color: white;
  background-color: #2563eb;
}

.secondary-action {
  color: #2563eb;
  border: 1px solid #bfdbfe;
}

.hero-image {
  display: block;
  width: 100%;
  height: auto;
  border-radius: 8px;
}

.feature-card {
  margin-bottom: 16px;
}
```

Open the page and make the browser narrow. The page should still be readable because the base layout is a single column.

Use a fluid container

The following `.page` rule is the first responsive habit:

CSS

```
.page {
  max-width: 960px;
  margin: 32px auto;
  padding: 0 16px;
}
```

`max-width: 960px` says the page should not grow wider than `960px`. That keeps long lines from becoming hard to read on wide screens.

The page does not set a fixed `width` . That means it can shrink naturally when the viewport is narrower than `960px` .

`margin: 32px auto` adds space above and below the page, and centers it horizontally when there is extra room.

`padding: 0 16px` gives the content breathing room on small screens so text and cards do not touch the edge of the viewport.

 **Avoid fixed page widths** - A rule like `width: 960px` can force horizontal scrolling on small screens. Prefer a flexible container with `max-width` and side padding.

Make images flexible

Images have a natural size from the file. If that natural size is wider than the screen, the image can overflow.

This rule keeps the image inside its parent:

CSS

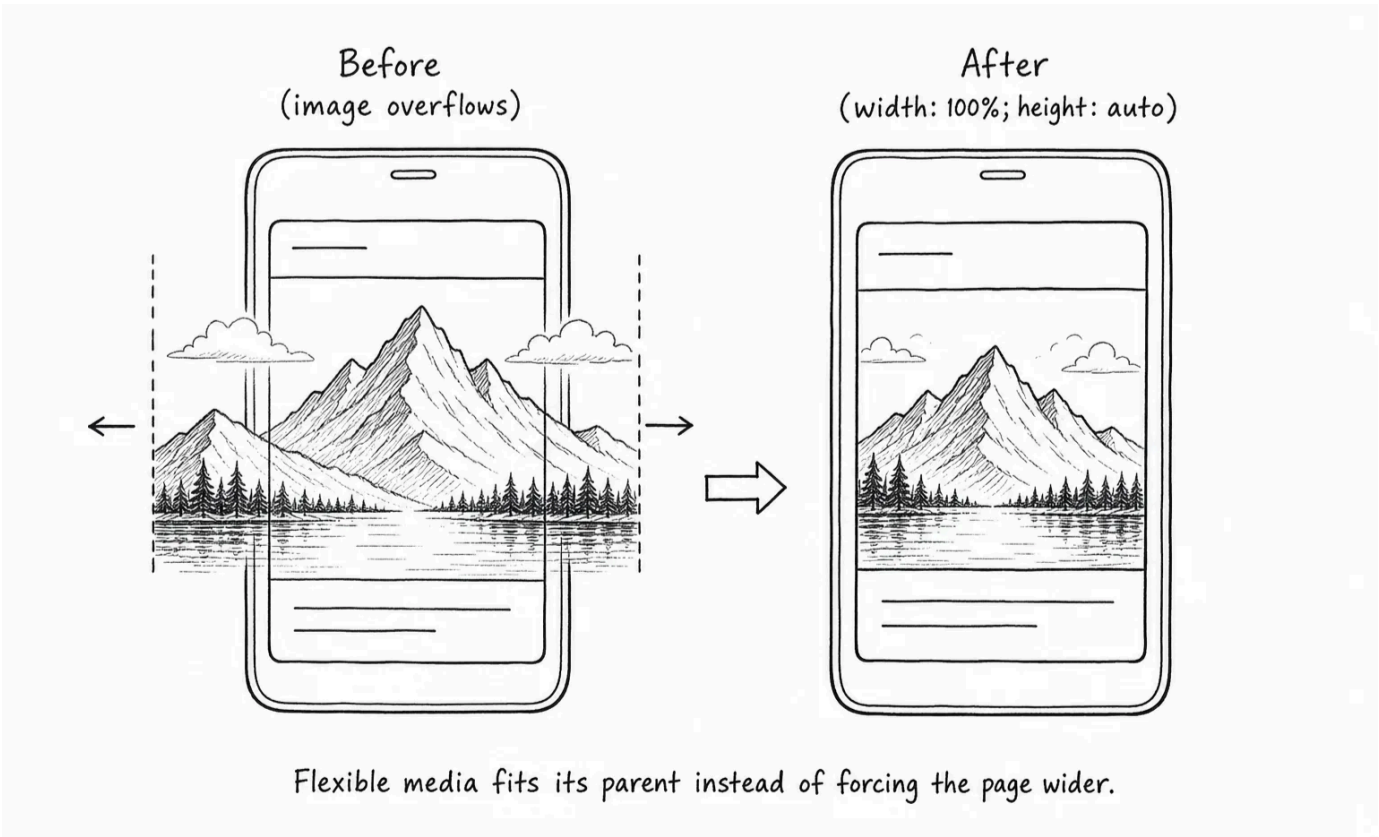
```
.hero-image {
  display: block;
  width: 100%;
  height: auto;
  border-radius: 8px;
}
```

`width: 100%` makes the image fill the available width of its parent.

`height: auto` keeps the image's natural proportions, so the browser does not stretch it.

`display: block` removes inline image spacing behavior, which you saw in the images lesson.

The `border-radius` is visual polish that rounds the image corners so the image matches the card shape.



Add a breakpoint when the layout needs one

The single-column layout works well on narrow screens. On wider screens, the hero image has enough room to sit beside the text.

That is what a media query is for. A media query applies CSS only when a condition is true.

Add this after the base CSS:

CSS

```
@media (min-width: 720px) {  
  .hero-card {  
    display: flex;  
    align-items: center;  
    gap: 24px;  
    justify-content: space-between;  
  }  
  
  .hero-copy,  
  .hero-image {  
    flex: 1;  
    max-width: 300px;  
  }  
}
```

`@media (min-width: 720px)` means "use these rules when the viewport is at least `720px` wide."

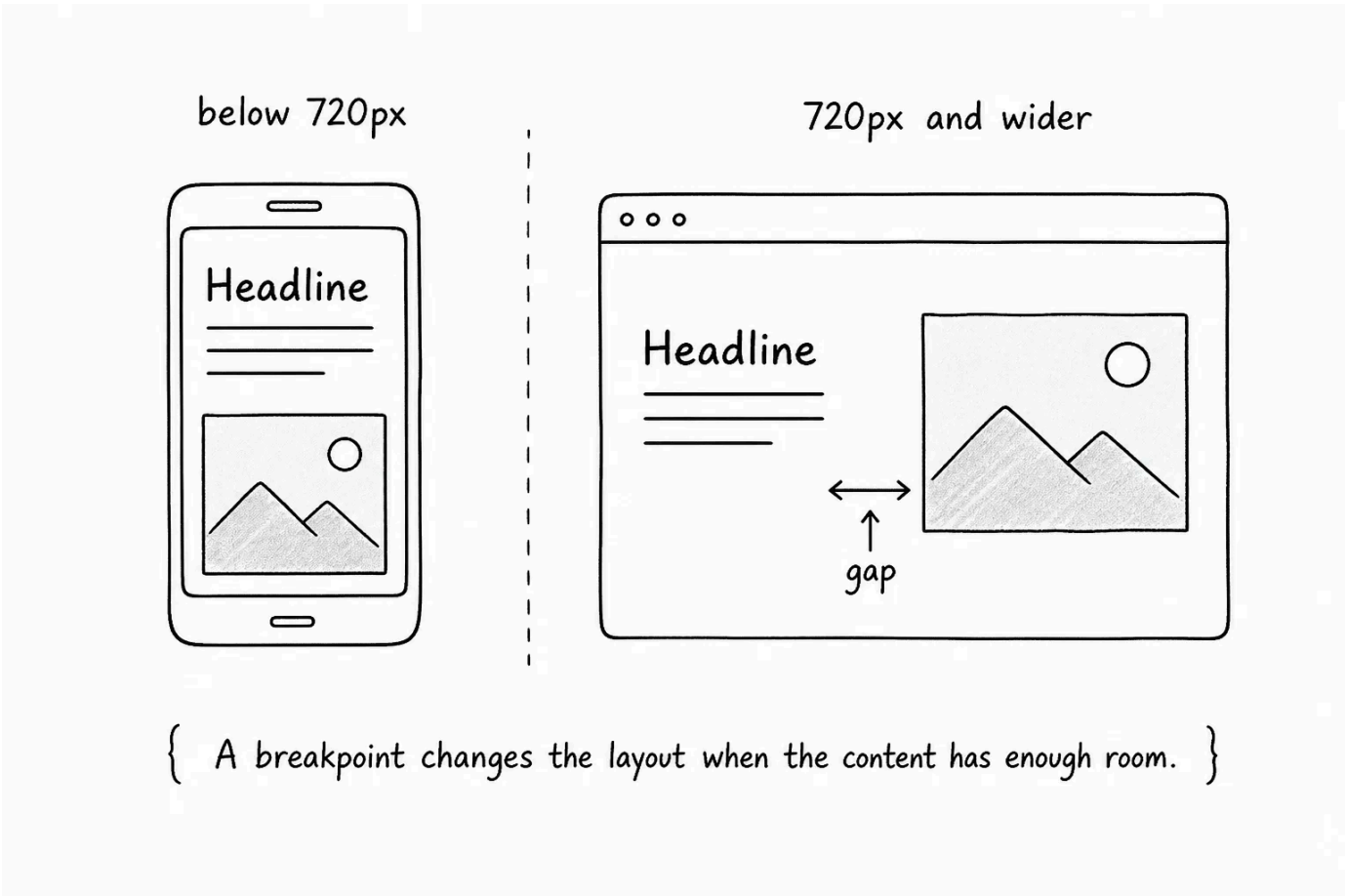
This is a mobile-first breakpoint: the normal CSS handles small screens first, and the media query adds a wider layout only when there is enough space.

Inside the media query, `.hero-card` becomes a flex container. Its direct children, `.hero-copy` and `.hero-image`, sit in a row.

`align-items: center` lines the text and image up vertically, and `gap: 24px` creates space between them.

flex: 1 lets the text area and image share the available row space.

Resize the browser. Below **720px**, the hero should stack; at **720px** and above, it should become a two-column hero.



Use breakpoints for content, not devices

A breakpoint is the point where your CSS changes because the layout needs a different shape.

The breakpoint should come from the content. If the hero feels cramped at **680px**, do not force it into two columns. If it has enough room at **720px**, that may be a good breakpoint.

Avoid thinking only in device names like "phone", "tablet", and "desktop". Device sizes change, and content needs are easier to reason about.

💡 **Resize until the layout complains** - To choose a breakpoint, resize the browser slowly. Add a media query where the content starts to feel cramped, stretched, or awkward.

Make the cards responsive too

The feature cards stack on small screens, which is a good default. On wider screens, they can sit in a row.

Add this to the same media query:

```
CSS
@media (min-width: 720px) {
  .hero-card {
    display: flex;
    align-items: center;
```

```
gap: 24px;
justify-content: space-between;
}

.hero-copy,
.hero-image {
  flex: 1;
  max-width: 300px;
}

.feature-list {
  display: flex;
  gap: 16px;
}

.feature-card {
  flex: 1;
  margin-bottom: 0;
}
}
```

The `.feature-list` becomes a flex container only on wider screens. Its direct children, the `.feature-card` elements, become flex items.

`gap: 16px` owns the space between the cards.

`flex: 1` lets each card share the row evenly.

`margin-bottom: 0` removes the stacked-card spacing from the small-screen version. On wide screens, `gap` handles the spacing between cards.

This is a common responsive pattern: stack by default, then switch to a row when there is enough room.

Check text size and spacing

Responsive design is not only layout. Text and spacing can also feel different at different viewport sizes.

The base `h1` is large enough for the small layout:

CSS

```
h1 {
  font-size: 36px;
  line-height: 1.1;
}
```

On wider screens, the heading can grow a little because it has more room:

CSS

```
@media (min-width: 720px) {
  h1 {
    font-size: 48px;
  }
}
```


Do this carefully. Bigger text can make a wide layout feel stronger, but it can also create awkward wrapping. Resize the browser after changing heading size and check where the lines break.

You do not need a media query for every property. Add one only when the current value stops working at a different size.

Test with DevTools

DevTools can simulate different viewport widths.

Open DevTools and look for the device toolbar or responsive mode. Drag the viewport wider and narrower, and watch for these problems:

- Horizontal scrolling.
- Text touching the edge of the screen.
- Images overflowing their cards.
- Buttons becoming too cramped to read or tap.
- Lines of text becoming too long on wide screens.
- Layout changes that happen too early or too late.

Responsive design is a testing habit as much as a CSS technique.

Put the responsive rules together

Here are the responsive pieces you added:

CSS

```
.page {
  max-width: 960px;
  margin: 32px auto;
  padding: 0 16px;
}

.hero-image {
  display: block;
  width: 100%;
  height: auto;
  border-radius: 8px;
}

@media (min-width: 720px) {
  .hero-card {
    display: flex;
    align-items: center;
    gap: 24px;
    justify-content: space-between;
  }

  .hero-copy,
  .hero-image {
    flex: 1;
    max-width: 300px;
  }

  .feature-list {
```

```
display: flex;
gap: 16px;
}

.feature-card {
  flex: 1;
  margin-bottom: 0;
}

h1 {
  font-size: 48px;
}
}
```

The pattern is simple: write the small-screen layout first, then add media queries when wider screens have enough room for a different layout.

Try it

Test the page by resizing it on purpose:

- Change `.page` from `max-width: 960px;` to `width: 960px;`. Make the browser narrow and notice the horizontal scrolling. Then change it back.
- Remove `width: 100%;` from `.hero-image` and check whether the image can overflow.
- Change the media query from `min-width: 720px` to `min-width: 900px`. Notice when the hero switches to two columns.
- Change the media query to `min-width: 500px`. Decide whether the two-column layout happens too early.
- Remove `.feature-card { margin-bottom: 0; }` inside the media query. Notice whether old stacked spacing still affects the row.
- Add a fourth feature card and check whether the row still feels comfortable.
- Use DevTools responsive mode and test at several widths, not just one.

The goal is not to memorize a perfect breakpoint. The goal is to make the page respond when the content needs a different amount of space.

< 0 / 11 >

Knew **0 Items**

Learnt **0 Items**

Skipped **0 Items**

ResetProgress

Test Your Understanding

What does responsive design mean?

[Click to Reveal the Answer](#)

✔ Already Know that

✳ Didn't Know that

⏩ Skip Question



LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →

Container queries

In the last lesson, media queries changed the page when the **viewport** reached a certain width. That works when the whole page needs to respond to the **browser size**, but sometimes a component needs to respond to the space it gets inside a container.

Imagine the same lesson card appears in two places:

- A narrow sidebar.
- A wider main content area.

The browser viewport might be wide, but the sidebar card is still narrow. A media query can only ask, "How wide is the viewport?" A container query can ask, "How wide is the space around this component?" That is the point of container queries.

Start with the component problem

To see the problem, build a page where the same card component appears in two different places. One card lives in a narrow sidebar. Another card lives in a wider main area.

Use this HTML for the experiment:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Container query practice</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <main class="page">
      <section class="intro">
        <p class="eyebrow">CSS practice</p>
        <h1>Make a card respond to its container</h1>
        <p class="summary">The same component can have different amounts of space depend
      </section>

      <section class="layout">
        <aside class="sidebar">
          <h2>Sidebar</h2>

          <div class="card-shell">
            <article class="lesson-card">
              
                <p class="card-label">Featured</p>
                <h3>Container queries</h3>
                <p>Adapt a component based on the space around it, not only the viewport
```



```
        <a class="card-link" href="/container-queries">Open lesson</a>
      </div>
    </article>
  </div>
</aside>

<section class="workspace">
  <h2>Main area</h2>

  <div class="card-shell">
    <article class="lesson-card">
      
        <p class="card-label">Featured</p>
        <h3>Container queries</h3>
        <p>Adapt a component based on the space around it, not only the viewport
        <a class="card-link" href="/container-queries">Open lesson</a>
      </div>
    </article>
  </div>
</section>
</section>
</main>
</body>
</html>
```

Now add the base CSS:

CSS

```
* {
  box-sizing: border-box;
}

body {
```

```
  background-color: #f9fafb;
}

.page {
  max-width: 1040px;
  margin: 40px auto;
  padding: 0 16px;
}

.intro,
.sidebar,
.workspace,
.lesson-card {
  background-color: white;
  border: 1px solid #e5e7eb;
  border-radius: 8px;
}

.intro,
.sidebar,
.workspace {
  padding: 24px;
}

.intro {
  margin-bottom: 24px;
}

.eyebrow,
.card-label {
  color: #2563eb;
  font-size: 14px;
  font-weight: 700;
}

h1,
h2,
```

```
h3,
p {
  margin-top: 0;
}

.summary,
.lesson-card p {
  color: #4b5563;
  line-height: 1.6;
}

.layout {
  display: flex;
  gap: 24px;
  align-items: flex-start;
}

.sidebar {
  width: 280px;
}

.workspace {
  flex: 1;
}

.lesson-card {
  overflow: hidden;
}

.card-image {
  display: block;
  width: 100%;
  height: auto;
}

.card-body {
  padding: 18px;
}

.card-link {
  color: #2563eb;
  font-weight: 700;
  text-decoration-thickness: 2px;
  text-decoration-offset: 3px;
}
```

Open the page. The two cards use the same HTML and the same CSS, but they sit in different spaces: the sidebar card is narrow, and the main-area card has more room.

Right now both cards use the same stacked layout: image on top, text below. That is safe for narrow spaces, but the wide card could use its space better.

A few base rules create that experiment:

- **.layout** uses Flexbox only to place the sidebar and workspace beside each other.
- **.sidebar** uses **width: 280px** so one card has a narrow container.
- **.workspace** uses **flex: 1** so the other card gets the remaining space.
- **.lesson-card** uses **overflow: hidden** so the image respects the rounded card corners.

- `.card-image` uses `display: block`, `width: 100%`, and `height: auto` so the image fits the stacked card before any container query runs.

Those rules set up two different component widths. The container query will decide how the card should behave inside each one.

Why a media query is not enough

You might try a media query like this:

CSS

```
@media (min-width: 800px) {  
  .lesson-card {  
    display: flex;  
  }  
  
  .card-image,  
  .card-body {  
    flex: 1;  
  }  
}
```

This asks a viewport question: "Is the browser at least `800px` wide?"

On a wide browser, the answer is yes. Both cards match the rule, including the sidebar card, and that is the problem. The sidebar card is still narrow even when the viewport is wide.

The component does not need to know the viewport width. It needs to know the width of the space it was placed in.

 **Viewport size and component size are different** - A wide browser does not mean every component has wide space. Sidebars, cards, panels, and columns can all be narrow inside a wide viewport.

Container queries have two parts

A container query needs two pieces:

1. A container to measure.
2. A query that runs when that container reaches a size.

The container is usually a wrapper around the component. In this HTML,

`.card-shell` wraps each `.lesson-card`:

html

```
<div class="card-shell">
  <article class="lesson-card">
    ...
  </article>
</div>
```

That wrapper is useful because it gives CSS a clear box to measure. The card can then change its internal layout based on the wrapper's width.

The wrapper also avoids a common confusion: a container query styles elements based on an ancestor container. The card should not try to query itself and style itself at the same time. A small wrapper makes the relationship clear: `.card-shell` is measured, and `.lesson-card` changes.

Mark the container

Before CSS can query an element's size, you need to mark that element as a query container.

Mark `.card-shell` as the container:

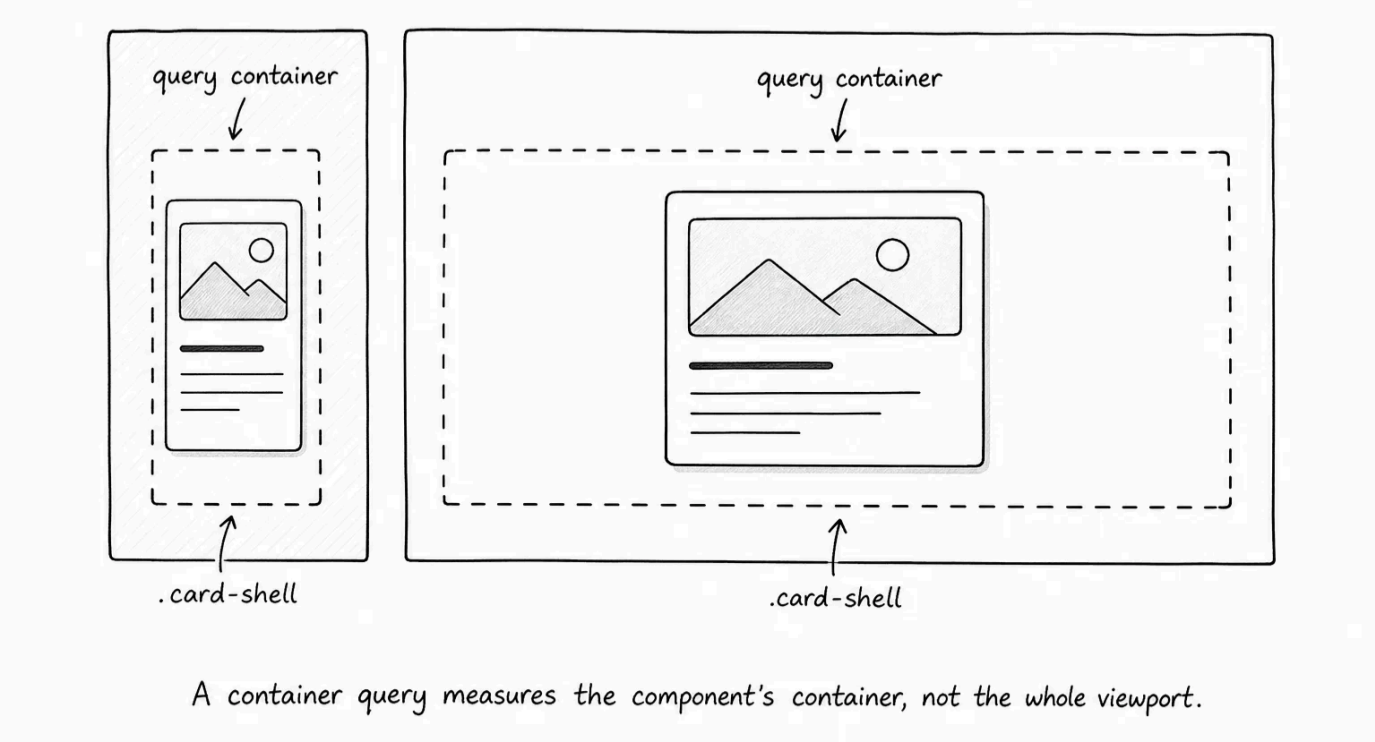
CSS

```
.card-shell {
  container-type: inline-size;
}
```

`container-type: inline-size` says this element can be queried by its inline size.

In normal English-language pages, inline size means width. So for this lesson, read it as: "allow CSS to query how wide `.card-shell` is."

This rule does not style the card visually by itself. It gives the browser permission to use `.card-shell` as a measurable container.



Query the container

Now that `.card-shell` is a query container, CSS can ask whether that container is wide enough for a different card layout.

Add the container query:

CSS

```
@container (min-width: 520px) {  
  .lesson-card {  
    display: flex;  
  }  
  
  .card-image {  
    width: 42%;  
    object-fit: cover;  
  }  
  
  .card-body {  
    flex: 1;  
  }  
}
```

`@container (min-width: 520px)` means "use these rules when the nearest query container is at least `520px` wide."

The sidebar `.card-shell` is narrower than `520px`, so its card stays stacked.

The workspace `.card-shell` is wider than `520px`, so its card becomes a side-by-side layout.

Inside the query, `.lesson-card` becomes a flex container. The image and body become flex items.

`width: 42%` gives the image a clear share of the row. It is not exactly half because text often needs a little more room than the image.

`object-fit: cover` keeps the image from looking stretched when it shares a row with the text. The image may crop slightly, but it keeps its proportions.

`flex: 1` lets `.card-body` take the remaining space.

This is the core container-query habit: let the component stay simple in narrow containers, then enhance it when its own container has enough room.

Container queries are component breakpoints

Media queries create viewport breakpoints, while container queries create component breakpoints.

That difference is practical because the two tools ask different questions:

Question	Use
Is the browser wide enough for the whole page layout to change?	Media que
Is this card's container wide enough for the card layout to change?	Container
Should the header, page shell, or main navigation change?	Media que
Should a reusable card, panel, or widget change based on where it appears?	Container

A container query does not replace every media query; it solves a different problem.

💡 **Use container queries for reusable components** - If the same component can appear in a narrow sidebar, a wide content area, or a dashboard panel, a container query often fits better than a viewport media query.

Name a container when needed

The query above uses the nearest query container. That is enough for many component examples.

Sometimes a component sits inside several containers, and you want to query a specific one. You can give a container a name:

CSS

```
.card-shell {
  container-type: inline-size;
  container-name: card;
}
```

Then query that named container:

CSS

```
@container card (min-width: 520px) {
  .lesson-card {
    display: flex;
  }
}
```

container-name gives the query container a label. **@container card (...)** says the query should use a container named **card**.

You do not need a name for every container. Use names when they make the query clearer or prevent CSS from accidentally using the wrong ancestor.

Inspect the container

DevTools can help you confirm whether your query is working.

Inspect the wide card and look at the matched CSS rules. When the `.card-shell` is wide enough, the rules inside `@container (min-width: 520px)` should appear.

Then inspect the sidebar card. The same `@container` rule should not apply if the sidebar `.card-shell` is too narrow.

If the query does not work, check these things:

- Did you put `container-type` on an ancestor of the element you are styling?
- Is the container actually wide enough to match the query?
- Are you accidentally querying the viewport with `@media` instead of querying the container with `@container` ?
- If you used `container-name` , does the name match?

Container queries are easiest to debug when you inspect the component and compare the container width with the query condition.

Put the container query rules together

Here are the container-query rules you added:

CSS

```
.card-shell {
  container-type: inline-size;
}

@container (min-width: 520px) {
  .lesson-card {
    display: flex;
  }

  .card-image {
    width: 42%;
    object-fit: cover;
  }

  .card-body {
    flex: 1;
  }
}
```

The pattern is: wrap the component, mark the wrapper as a query container, then write a container query for the layout the component should use when that wrapper has enough space.

Try it

Test the card by changing the container, not the viewport:

- Change `.sidebar` from `width: 280px;` to `width: 560px;` . Notice whether the sidebar card now matches the container query.
- Change the query from `@container (min-width: 520px)` to `@container (min-width: 700px)` . Notice when the wide card stops using the side-by-side layout.
- Remove `container-type: inline-size;` from `.card-shell` . Confirm that the container query no longer works.
- Put `container-type: inline-size;` on `.lesson-card` instead of `.card-shell` . Notice whether the query behaves the way you expected.
- Add a second card inside `.workspace` and check whether the same component rules still apply.
- Try the named-container version with `container-name: card` , then update the query to `@container card (min-width: 520px)` .

The goal is to make component layout depend on component space. If the same component can live in different-sized places, a container query can often make it adapt more naturally than a media query.

< 0 / 10 >

✔ Knew 0 Items

✳️ Learnt 0 Items

⏮ Skipped 0 Items

🔄 ResetProgress

Test Your Understanding

What problem do container queries solve?

[Click to Reveal the Answer](#)

✔ Already Know that

✳️ Didn't Know that

⏮ Skip Question

✓

LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →

Positioning and layers

Most CSS layout you have written so far follows normal flow.

Normal flow is the browser's default way of placing elements: headings stack above paragraphs, block elements create rows, inline elements sit inside text, Flexbox arranges direct children, and Grid creates rows and columns. That should stay your default way to build most layouts.

But some interface pieces do not belong in the normal flow:

- A small label needs to sit on top of an image.
- A header should stay visible while the page scrolls.
- A help link should stay in the corner of the viewport.
- One overlapping piece should appear above another.

Those are positioning problems because a specific element needs to be anchored to a specific reference box.

Start with the visible problem

Use a page with one sticky-looking header, one course card, some scrollable content, and a help link. At first, everything sits in normal flow. That is useful because you can see what positioning changes.

Start with this HTML:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Positioning practice</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <header class="site-header">
      <div class="header-inner">
        <strong>CSS Studio</strong>
        <a href="/lessons">All lessons</a>
      </div>
    </header>

    <main class="page">
      <section class="intro">
        <p class="eyebrow">CSS practice</p>
        <h1>Position small pieces without breaking the page</h1>
        <p class="summary">The page layout should stay normal, but a few details need to
      </section>
```



```
<article class="course-card">
  <div class="card-media">
    
    <p class="status-badge">New</p>
  </div>

  <div class="card-body">
    <h2>Build responsive interfaces</h2>
    <p>Learn how layout, spacing, media, and component rules work together in real</p>
  </div>
</article>

<section class="notes">
  <h2>Lesson notes</h2>
  <p>Normal flow is still doing most of the work. The sections stack, the card tak</p>
  <p>Positioning should be saved for the pieces that need special placement, such</p>
  <p>Add enough content here so the page can scroll. That makes the sticky header</p>
  <p>When the page scrolls, notice which elements move with the document and which</p>
</section>
</main>

  <a class="help-link" href="/help">Need help?</a>
</body>
</html>
```

Then add the starting styles:

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: system-ui, sans-serif;
  color: #111827;
  background-color: #f3f4f6;
}

a {
  color: #2563eb;
  font-weight: 700;
}

.site-header {
  background-color: white;
  border-bottom: 1px solid #e5e7eb;
}

.header-inner,
.page {
  max-width: 960px;
  margin: 0 auto;
  padding: 0 16px;
}

.header-inner {
  min-height: 64px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 16px;
}

.page {
  padding-top: 32px;
  padding-bottom: 160px;
}

.intro,
.course-card,
.notes {
  background-color: white;
```

```
border: 1px solid #e5e7eb;
border-radius: 8px;
}

.intro,
.card-body,
.notes {
  padding: 24px;
}

.intro,
.course-card {
  margin-bottom: 24px;
}

.eyebrow {
  margin-top: 0;
  color: #2563eb;
  font-size: 14px;
  font-weight: 700;
}

h1,
h2,
p {
  margin-top: 0;
}

.summary,
.course-card p,
.notes p {
  color: #4b5563;
  line-height: 1.6;
}

.card-media img {
  display: block;
  width: 100%;
  height: auto;
}

.status-badge {
  display: inline-block;
  margin: 16px;
  padding: 6px 10px;
  color: #065f46;
  background-color: #d1fae5;
  border-radius: 999px;
  font-size: 14px;
  font-weight: 700;
}

.help-link {
  display: inline-block;
  margin: 24px;
  padding: 10px 14px;
  color: white;
  background-color: #2563eb;
  border-radius: 8px;
  text-decoration: none;
}
```

Open the page and notice three things:

- The header scrolls away like normal content.
- The **New** badge sits below the image because it is still in normal flow.
- The help link sits after the main content because it is also in normal flow.

That is not broken; it is the browser doing the normal thing.

Positioning starts when you decide that one specific element should leave that normal placement.

Position the badge inside the card image

The badge should belong visually on the image, not below it. **position: absolute** lets you remove that badge from normal flow and place it with offsets such as **top** and **right**.

Update **.status-badge** like this:

CSS

```
.status-badge {
  position: absolute;
  top: 12px;
  right: 12px;
  margin: 0;
}
```

If you open the browser, the badge jumps to the top-right corner of the page instead of the card.



That happens because an absolutely positioned element needs a reference box. If you do not give it a clear positioned ancestor, the browser uses a larger reference area, so **top: 12px; right: 12px** is measured from the page instead of the card image.

To fix that, the image wrapper needs to become the badge's reference box. Give **.card-media** **position: relative**, then the absolute badge can position itself against that parent.

Modify the CSS like this:

CSS

```
.card-media {
  position: relative;
}

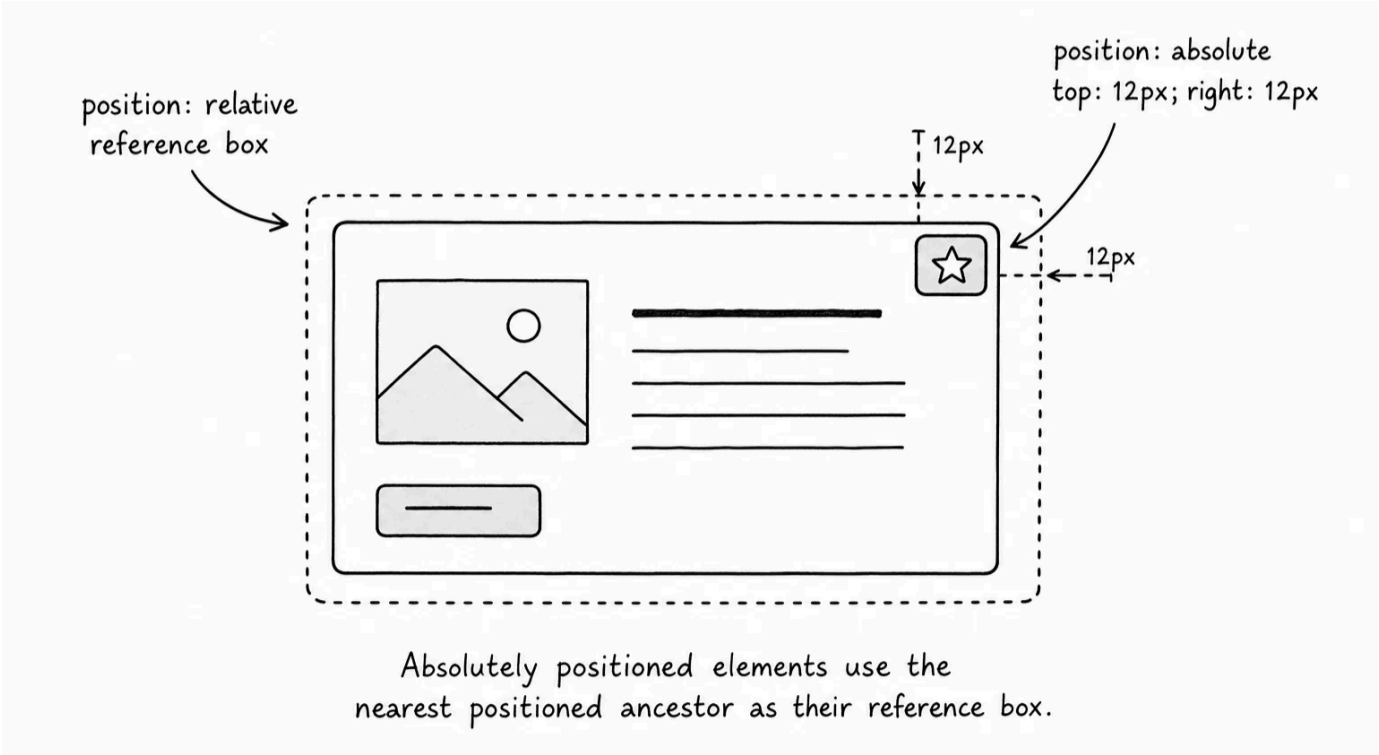
.status-badge {
  position: absolute;
  top: 12px;
  right: 12px;
  margin: 0;
}
```

Now the badge should sit correctly over the image.

`position: relative` does not move `.card-media` by itself. The card is still where it was, but it can now act as the reference box for absolute children.

`position: absolute` removes `.status-badge` from normal flow. The badge no longer takes up a row below the image; it is placed `12px` from the top and `12px` from the right of `.card-media`.

In CSS, a positioned element is any element whose `position` value is not `static`, which is the default.



💡 **Pair absolute with a clear parent** - When you use `position: absolute` on a child, look for the parent that should own the placement. Give that parent `position: relative` so the relationship is obvious.

Keep the header visible while scrolling

The header is part of the page, but it can be useful to keep it visible while the learner scrolls through the content. This is a good job for `position: sticky`.

Make the header sticky:

```
CSS

.site-header {
  position: sticky;
  top: 0;
}
```

`position: sticky` starts like normal flow, so the header still keeps its space at the top of the document.

`top: 0` tells the browser when the sticky behavior should start. When the top of the header reaches the top of the viewport, it sticks there. Without an

offset like `top: 0`, sticky positioning has no threshold to stick to. The browser needs to know where the element should become stuck.

Sticky positioning is useful for headers, table headings, section navigation, and small toolbars. It is not a replacement for normal page layout.

Fix the help link to the viewport

The help link is different from the sticky header.

It should not keep its normal place in the document. It should stay in the bottom-right corner of the browser window even while the page scrolls.

That is a good job for `position: fixed`.

Replace the existing `.help-link` rule with this version:

CSS

```
.help-link {
  position: fixed;
  right: 24px;
  bottom: 24px;
  display: inline-block;
  padding: 10px 14px;
  color: white;
  background-color: #2563eb;
  border-radius: 8px;
  text-decoration: none;
}
```

`position: fixed` removes the help link from normal flow and anchors it to the viewport.

The viewport is the visible browser area. So `right: 24px` and `bottom: 24px` mean the help link sits `24px` from the right and bottom edges of the browser window.

Fixed positioning is common for floating help buttons, cookie banners, persistent mobile actions, and modal backdrops. Use it carefully, because fixed elements can cover content if they are too large or placed in the wrong spot.

Layers decide what appears on top

Positioned elements can overlap. When that happens, the browser needs to know which one should appear closer to the user, and that is what `z-index` controls. Give the positioned elements a small layer scale:

CSS

```
.site-header {
  position: sticky;
```

```
top: 0;
z-index: 10;
}

.status-badge {
  position: absolute;
  top: 12px;
  right: 12px;
  margin: 0;
  z-index: 1;
}

.help-link {
  position: fixed;
  right: 24px;
  bottom: 24px;
  z-index: 20;
  display: inline-block;
  padding: 10px 14px;
  color: white;
  background-color: #2563eb;
  border-radius: 8px;
  text-decoration: none;
}
```

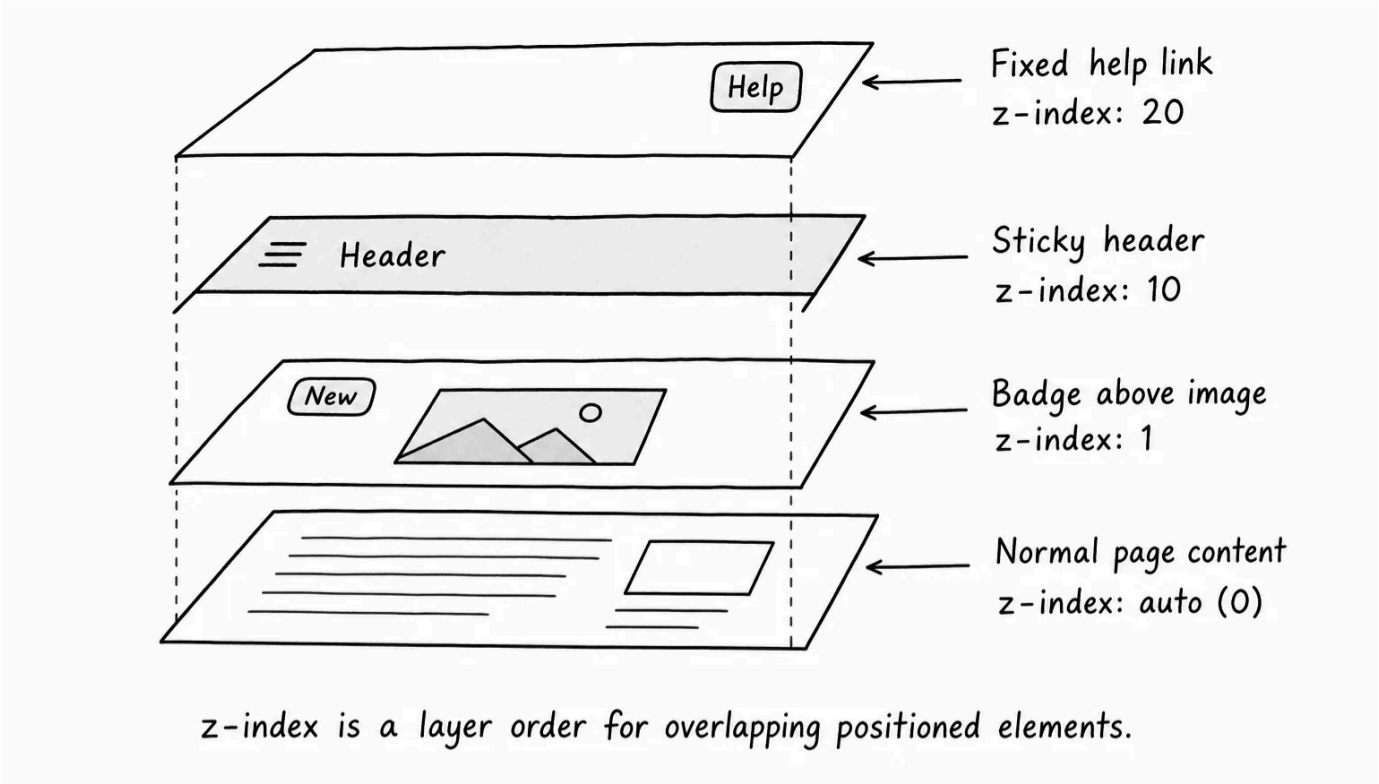
z-index only matters when elements are in layers that can overlap. A higher **z-index** usually appears above a lower one.

Here, the badge only needs to sit above the image, so **z-index: 1** is enough.

The sticky header may overlap page content while scrolling, so it gets **z-index: 10**.

The fixed help link should stay above normal page content, so it gets **z-index: 20**.

The numbers are not magic; they are just a small scale. You usually do not need values like **999999**. If you feel tempted to use a huge number, inspect the elements and find out which layer is actually competing.



Relative offsets are rarely the layout fix

position: relative can also move an element with offsets:

CSS

```
.course-card {  
  position: relative;  
  top: 12px;  
}
```

This moves the card visually, but the space where the card originally belonged is still kept.

The offset works because **.course-card** is positioned. On a normal **static** element, **top**, **right**, **bottom**, and **left** do not move anything.

That can feel strange because the card looks lower, but the surrounding layout still behaves as if the card stayed in its original spot.

For normal spacing between sections, use **margin** or **gap**. Use relative offsets for small visual nudges, not for page layout.

 **Do not use positioning as your main layout system** - If you are arranging a row, a column, a dashboard, or a responsive page, use normal flow, Flexbox, Grid, spacing, and sizing first. Use positioning for anchored details, floating controls, sticky pieces, and deliberate overlaps.

Inspect positioned elements

DevTools makes positioning easier to debug.

Inspect the badge and look at its matched rules. Toggle **position: absolute** off. The badge should return to normal flow below the image.

Turn **position: absolute** back on, then toggle **position: relative** off on **.card-media**. The badge may jump because its reference box changed.

Inspect the sticky header and fixed help link. In the Computed panel, look for **position**, **top**, **right**, **bottom**, and **z-index**.

When positioning does not behave as expected, ask these questions:

- Is the element still in normal flow, or has it been removed from normal flow?
- If it is absolute, which ancestor is its reference box?
- If it is sticky, does it have an offset such as **top: 0**?
- If it is fixed, is it anchored to the viewport?

- If it overlaps another element, does it need a clear `z-index` ?

That debugging habit is better than guessing bigger numbers.

Positioning values in one place

Here is the practical map:

Value	What it is good for
<code>static</code>	The default. The element stays in normal flow.
<code>relative</code>	Keeps the element in normal flow and can create a reference box for absolute
<code>absolute</code>	Removes the element from normal flow and anchors it to a positioned ances
<code>sticky</code>	Starts in normal flow, then sticks to an offset while scrolling.
<code>fixed</code>	Removes the element from normal flow and anchors it to the viewport.

The useful question is not "Which position value should I memorize?"

The useful question is: "What should this element be anchored to?"

Try it

Use the same page and test each positioning idea on purpose:

- Remove `position: relative` from `.card-media` . Notice where the badge goes, then add it back.
- Change the badge from `top: 12px; right: 12px;` to `bottom: 12px; left: 12px;` . Notice how the offsets choose different edges.
- Remove `top: 0` from `.site-header` . Check whether the sticky header still sticks.
- Change `.help-link` from `position: fixed` to `position: absolute` . Scroll the page and compare the behavior.
- Set `.site-header` to `z-index: 0` and `.help-link` to `z-index: 20` . Notice which element stays above if they ever overlap.
- Try moving the card with `position: relative; top: 20px;` , then remove that and use `margin-top: 20px;` . Compare how the surrounding layout behaves.

Positioning is easiest to learn when you keep asking what the element is anchored to. Parent box, viewport, scroll position, or normal flow. Once that is

clear, the CSS choice becomes much less random.

< 0 / 10 >

✔ Knew 0 Items

✧ Learnt 0 Items

⏮ Skipped 0 Items

🔄 ResetProgress

Test Your Understanding

What is normal flow?

[Click to Reveal the Answer](#)

✔ Already Know that

✧ Didn't Know that

⏮ Skip Question

✓

LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →

Overflow and scrolling

In the last lesson, you positioned a few elements outside normal flow.

This lesson looks at a different kind of problem: content that is larger than the box it lives inside.

That happens all the time in real interfaces:

- A notification panel has more messages than fit in the space.
- A code sample is wider than the card around it.
- A long title does not fit on one row.
- An image or decorative shape needs to be clipped to a rounded container.

Overflow is the browser's answer to one question:

What should happen when the content is bigger than the box?

Start with content that outgrows a box

Use a small release-notes page. It has a normal article, a messages panel, a long code sample, and one long lesson title.

The page is intentionally built so a few pieces have too much content for their space.

Start with this HTML:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Overflow practice</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <main class="page">
      <section class="intro">
        <p class="eyebrow">CSS practice</p>
        <h1>Decide what happens when content does not fit</h1>
        <p class="summary">Some content should make the page taller. Some content should
      </section>

      <section class="layout">
        <article class="release-card">
          <h2>Release notes</h2>
          <p>Version 2.4 improves layout tools, fixes several mobile spacing issues, and
```




```
<pre class="code-sample"><code>pnpm roadmap lesson publish --pack css --lesson 1</code></pre>

<h3 class="long-title">A very long lesson title that needs to stay on one line</h3>
<p>The title above is not a paragraph. It is a compact label, so it may need a compact treatment.</p>
</article>

<aside class="message-panel">
  <h2>Recent messages</h2>

  <div class="message-list">
    <p><strong>Ada:</strong> The card layout looks good on desktop.</p>
    <p><strong>Sam:</strong> The mobile header needs another pass.</p>
    <p><strong>Priya:</strong> The code sample is making the page too wide.</p>
    <p><strong>Lee:</strong> The release notes panel should not take over the whole page.</p>
    <p><strong>Noor:</strong> The long title needs a compact treatment.</p>
    <p><strong>Maya:</strong> The final screenshot is ready for review.</p>
  </div>
</aside>
</section>
```

Then add the base CSS:

CSS

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: system-ui, sans-serif;
  color: #111827;
  background-color: #f3f4f6;
}

.page {
  max-width: 1040px;
  margin: 40px auto;
  padding: 0 16px;
}

.intro,
.release-card,
.message-panel {
  background-color: white;
  border: 1px solid #e5e7eb;
  border-radius: 8px;
}

.intro,
.release-card,
.message-panel {
  padding: 24px;
}

.intro {
  margin-bottom: 24px;
}

.eyebrow {
  margin-top: 0;
  color: #2563eb;
  font-size: 14px;
  font-weight: 700;
}

h1,
h2,
h3,
p {
  margin-top: 0;
```

```
}

.summary,
.release-card p,
.message-list p {
  color: #4b5563;
  line-height: 1.6;
}

.layout {
  display: flex;
  gap: 24px;
  align-items: flex-start;
}

.release-card {
  flex: 1;
}

.message-panel {
  width: 320px;
}

.code-sample {
  padding: 16px;
  color: #e5e7eb;
  background-color: #111827;
  border-radius: 8px;
}

.long-title {
  padding: 12px;
  background-color: #eff6ff;
  border-radius: 8px;
}
```

Open the page and look for two problems:

- The code sample may push sideways because the command is one long line.
- The messages panel can become taller than you want because it has many messages.

That is the setup: overflow only becomes a real question when a box has less space than its content wants.

Overflow needs a constrained box

Overflow problems happen when you have more content than a constrained box can contain. So far, most of the pages in this pack have simply grown taller when they needed more vertical space.

Before we discuss **overflow**, the browser needs a box with a limit.

If a normal section has a lot of text, the browser usually makes the section taller. No overflow problem appears because the box grows to fit the content.

To create a real overflow decision, give the messages list a maximum height:

CSS	


```
.message-list {  
  max-height: 220px;  
  padding-right: 8px;  
}
```

max-height: 220px says the list may be shorter, but it should not grow taller than **220px** .

Now the messages may not all fit inside that height. The content is larger than the box.

padding-right: 8px adds a little breathing room on the right. It will matter once a scrollbar appears.

At this point, the browser still uses the default overflow behavior.

The default is visible overflow

The default value is **overflow: visible** , which means content is allowed to paint outside the box.

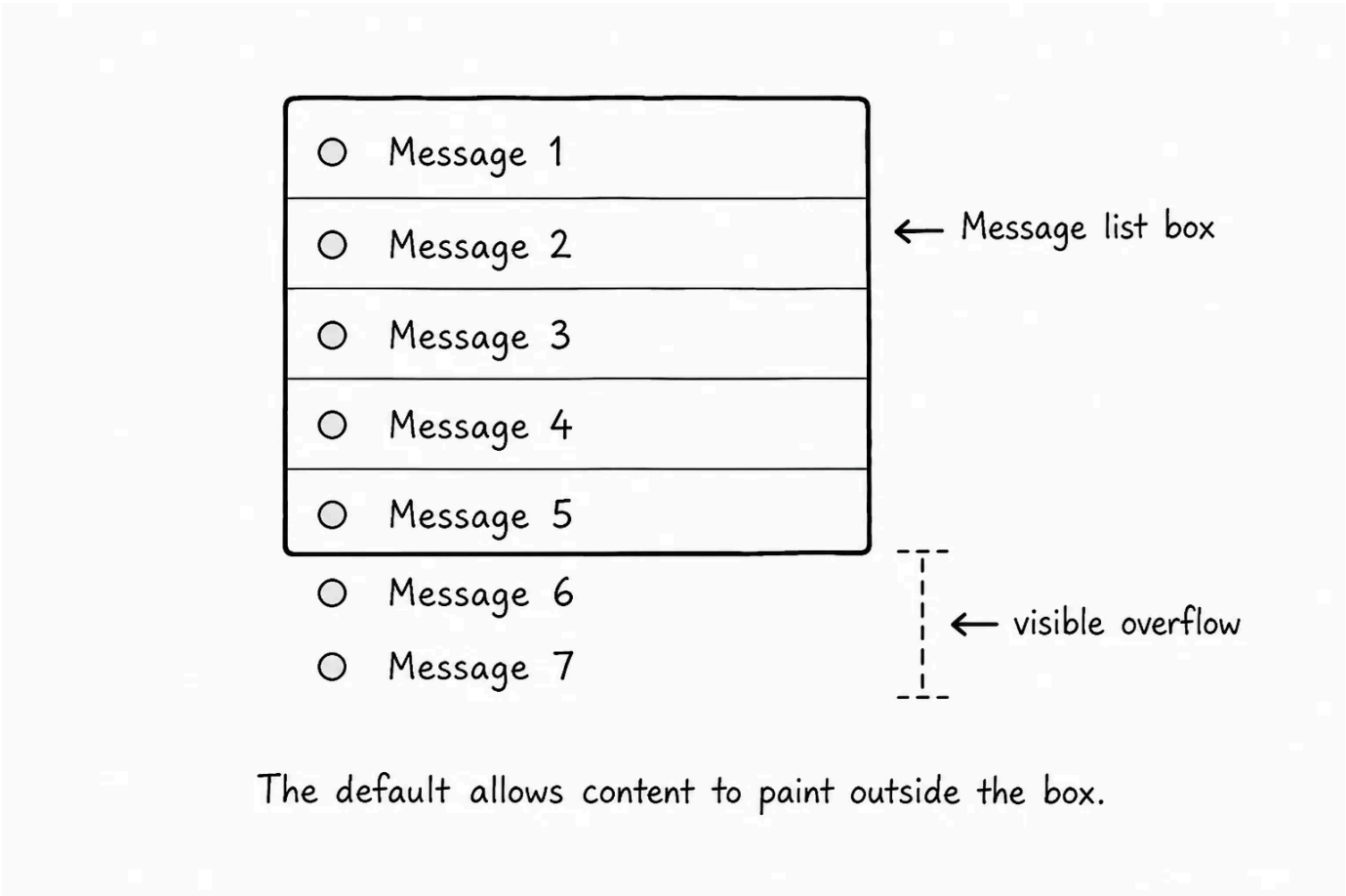
Add it explicitly so you can see the default:

CSS

```
.message-list {  
  max-height: 220px;  
  padding-right: 8px;  
  overflow: visible;  
}
```

The list has a height limit, but the extra messages may still spill out visually.

That can be useful for some decorative effects, but it is not good for a message panel. The panel now says it has one size while its content paints outside that space.



Use auto when the user should scroll

For a message list, the extra content is still important. You should not hide it because the user needs a way to reach it.

That is a good job for `overflow: auto`.

Change the list to use automatic scrolling:

CSS

```
.message-list {
  max-height: 220px;
  padding-right: 8px;
  overflow: auto;
}
```

`overflow: auto` lets the browser add scrolling only when the content needs it.

If the messages fit, there may be no scrollbar. If the messages do not fit, the list becomes scrollable.

That is why `auto` is usually better than forcing scrollbars everywhere: it lets the content decide.

Use this for panels, menus, sidebars, log viewers, notification lists, and other bounded areas where the user should still reach all the content.

 **Prefer auto for real content** - If the content matters and the box has a size limit, start with `overflow: auto`. It preserves access to the content while keeping the layout under control.

Use hidden only when clipping is intentional

Sometimes overflow should be clipped. For example, a decorative image inside a rounded card might need to stay inside the rounded corners, a badge glow might need to stay inside a compact component, or a preview thumbnail might crop extra visual detail.

That is when `overflow: hidden` can make sense.

Add a small clipped preview to the card:

html

```
<div class="preview-box">
  <div class="preview-shape"></div>
</div>
```

Place it inside `.release-card`, above the code sample, then add this CSS:

CSS

```
.preview-box {
  height: 120px;
  margin-bottom: 16px;
  background-color: #dbeafe;
  border-radius: 12px;
  overflow: hidden;
}

.preview-shape {
  width: 180px;
  height: 180px;
  margin-left: auto;
  background-color: #60a5fa;
  border-radius: 999px;
}
```

The circle is taller than the preview box. `overflow: hidden` clips the part that sticks out.

This works because the extra part is decorative. The user is not losing an important message or control.

That distinction matters because `overflow: hidden` does not delete content from the HTML. It only makes the overflowing part invisible and unreachable visually.

 **Do not hide important content** - Avoid using `overflow: hidden` to make real text, form fields, buttons, or warnings disappear. If users need the content, use a layout that fits it or use scrolling.

Control one direction when needed

Sometimes only one direction is the problem.

The code sample is a good example. The command is one long line. You do not want the whole page to become wider just because one command is wide.

The fix is horizontal scrolling on the code box:

CSS

```
.code-sample {
  padding: 16px;
  color: #e5e7eb;
  background-color: #111827;
  border-radius: 8px;
  overflow-x: auto;
}
```

`overflow-x` controls horizontal overflow, and `auto` means the browser only adds horizontal scrolling when the content is wider than the box.

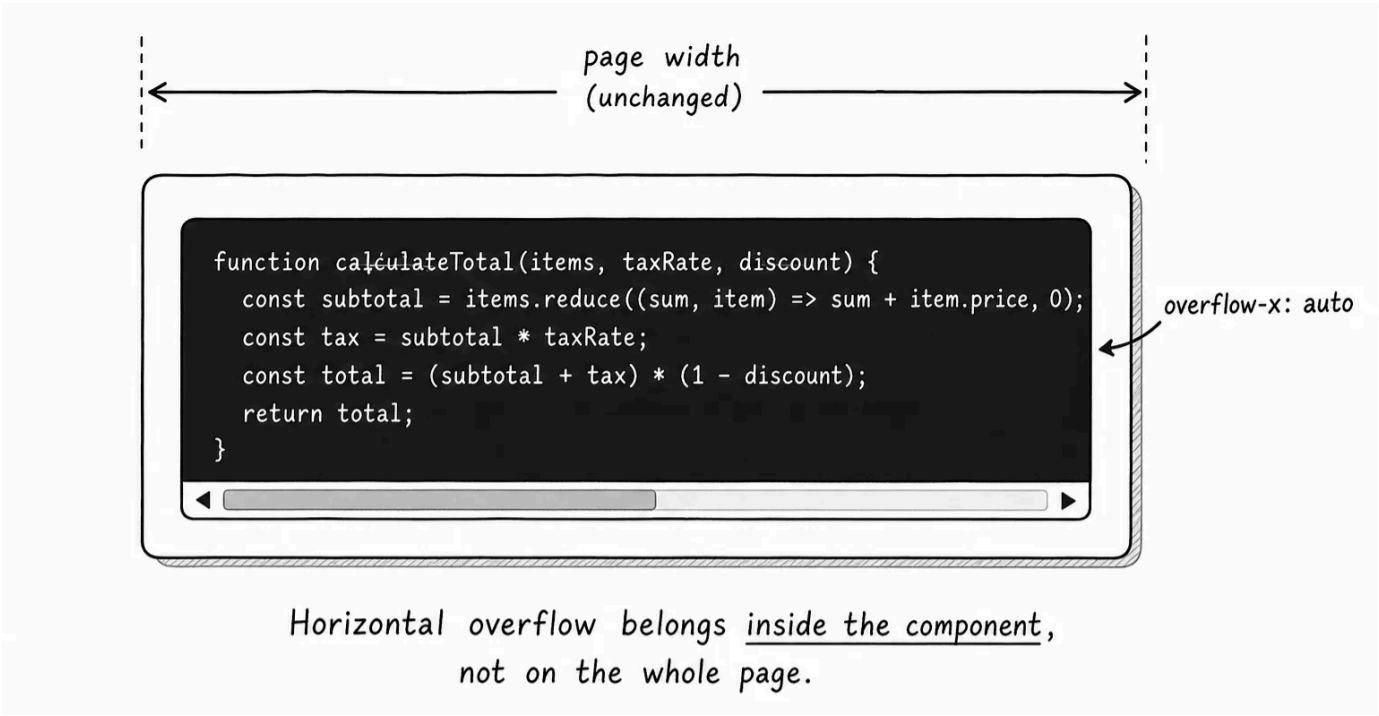
There is also **overflow-y** for vertical overflow. The messages list could be written like this:

CSS

```
.message-list {  
  max-height: 220px;  
  padding-right: 8px;  
  overflow-y: auto;  
}
```

overflow-y: auto focuses the scroll behavior on the vertical direction.

Use directional overflow when the problem is clearly one direction: wide tables, long code lines, tag rows, or tall panels.



Keep the page from getting wider by accident

Horizontal page overflow is one of the most common CSS bugs.

You see it when the whole page gets a sideways scrollbar. Usually, one child element is wider than the viewport or wider than its parent.

The release card is a flex item because **.layout** uses Flexbox. The card is supposed to take the available space:

CSS

```
.release-card {  
  flex: 1;  
}
```

flex: 1 lets the card take the available space in the row.

But Flexbox has a detail that can surprise you: flex items have an automatic minimum size based on their content. A long code line can make the card refuse to shrink.

Add this:

CSS

```
.release-card {  
  flex: 1;  
  min-width: 0;  
}
```

min-width: 0 allows the card to shrink inside the layout, so the code block can handle its own horizontal scrolling.

This is not an overflow property, but it often fixes overflow bugs in Flexbox layouts.

If a page has a mystery horizontal scrollbar, inspect the wide child first. Then check whether a flex item needs **min-width: 0**.

Truncate one-line text carefully

The long title is not normal paragraph text. It is a compact card heading. In some interfaces, you may want it to stay on one line and show that it was shortened.

That needs three declarations working together:

CSS

```
.long-title {  
  padding: 12px;  
  background-color: #eff6ff;  
  border-radius: 8px;  
  white-space: nowrap;  
  overflow: hidden;  
  text-overflow: ellipsis;  
}
```

white-space: nowrap keeps the title on one line.

overflow: hidden clips the part that does not fit.

text-overflow: ellipsis shows three dots at the clipped edge, so the user can see that the text continues.

The ellipsis only works when the text is forced onto one line and the overflow is hidden.

Use this carefully. It is fine for compact labels, table cells, filenames, and cards where the full text is available somewhere else. It is a poor choice for important instructions or body text.

Inspect overflow in DevTools

DevTools helps you find overflow instead of guessing.

Inspect the code sample and look at its size. Toggle `overflow-x: auto` off. If the page gets wider or the code spills out, you have found the reason for the horizontal overflow.

Inspect the messages list and toggle `overflow: auto` off. The extra messages should spill outside the constrained box again.

If you see a horizontal scrollbar on the whole page, use the element picker and hover wide elements. Look for something that is wider than its parent or wider than the viewport.

Overflow bugs usually come from one of these:

- A fixed width that is too large for the screen.
- A long unbroken line of text or code.
- A flex item that needs `min-width: 0`.
- An image or media element that is not allowed to shrink.
- A positioned or fixed element that reaches past the viewport edge.

The fix depends on the cause. Do not reach for `overflow: hidden` on the `body` as the first move. That can hide the symptom while leaving the broken layout in place.

Overflow values in one place

Here is the practical map:

Value	What it does
<code>visible</code>	Lets content paint outside the box. This is the default.
<code>auto</code>	Adds scrolling only when the content needs it.
<code>scroll</code>	Shows scrollbars even when they may not be needed.
<code>hidden</code>	Clips overflowing content and does not show scrollbars.

You can apply the idea to both directions with `overflow-x` and `overflow-y`.

The useful question is: should the user still be able to reach the content?

If yes, use scrolling or let the layout grow. If no, clipping may be acceptable.

Try it

Use the same page and test each overflow decision:

- Change `.message-list` from `overflow: auto` to `overflow: visible` . Notice how the messages spill outside the constrained height.
- Change `.message-list` to `overflow: hidden` . Notice that the extra messages are clipped and no scrollbar appears.
- Add two more messages inside `.message-list` . Check whether `overflow: auto` adds scrolling only when needed.
- Remove `overflow-x: auto` from `.code-sample` . Watch whether the code line pushes outside its box or makes the page wider.
- Remove `min-width: 0` from `.release-card` . If the page gets wider, inspect the card and code sample to see what is refusing to shrink.
- Remove `white-space: nowrap` from `.long-title` . Notice why the ellipsis no longer behaves the same way.
- Use DevTools to toggle each overflow declaration and watch what changes before saving the final version.

The habit is simple: first decide whether the content should stay reachable. Then choose growing, scrolling, clipping, or truncating based on that decision.

< 0 / 10 >

✔ Knew 0 Items

✳️ Learnt 0 Items

⏮ Skipped 0 Items

🔄 ResetProgress

Test Your Understanding

What is overflow?

[Click to Reveal the Answer](#)

✔ Already Know that

✳️ Didn't Know that

⏮ Skip Question

✓

LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →

Styling forms

Forms are where CSS stops being only visual polish. A form is something a user has to complete. Styling should help them understand what to fill in, which field they are on, what went wrong, and what action they can take next.

That gives this lesson a practical goal: Can the user read the form, fill it in, notice focus, spot errors, and submit it without guessing?

Work with a form that has real states

Use a support request form. It has text inputs, a select menu, a textarea, a checkbox, a submit button, a disabled button, a hint, and an error state. That mix is useful because form styling is not only about one `<input>` . Real forms have groups, labels, messages, choices, and actions.

Use this HTML:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Form styling practice</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <main class="page">
      <section class="intro">
        <p class="eyebrow">CSS practice</p>
        <h1>Style a form people can actually use</h1>
        <p class="summary">The form should feel calm, readable, and clear in its normal,
      </section>

      <form class="support-form" action="/support" method="post">
        <div class="field">
          <label for="full-name">Full name</label>
          <input id="full-name" name="full-name" type="text" autocomplete="name" placeholder="Full name" />
        </div>

        <div class="field has-error">
          <label for="email">Email address</label>
          <input id="email" name="email" type="email" autocomplete="email" aria-invalid="true" />
          <p class="field-hint" id="email-hint">Use the email address where we can reach you</p>
          <p class="field-error" id="email-error">Enter a valid email address.</p>
        </div>

        <div class="field">
          <label for="topic">Topic</label>
          <select id="topic" name="topic">
            <option value="">Choose a topic</option>
            <option value="layout">Layout problem</option>
            <option value="forms">Form styling</option>
            <option value="accessibility">Accessibility question</option>
          </select>
        </div>
      </form>
    </main>
  </body>
</html>
```



```
<div class="field">
  <label for="message">Message</label>
  <textarea id="message" name="message" rows="5" placeholder="Tell us what happened" />
</div>

<label class="check-field">
  <input type="checkbox" name="updates" />
  <span>Send me updates about this request.</span>
</label>

<div class="form-actions">
  <button type="submit">Send request</button>
  <button class="secondary-button" type="button" disabled>Save draft</button>
</div>
</form>
</main>
</body>
</html>
```

The HTML already does important form work:

- Each main control has a `<label>` .
- `autocomplete` helps the browser suggest the right saved values for name and email fields.

- `aria-invalid` and `aria-describedby` help connect the email field to its error and hint text for assistive technology.
- The checkbox stays a real checkbox.
- The disabled button uses the real `disabled` attribute.

CSS should improve that structure, not replace it.

Give the form a readable shape

The first form problem is usually scanning. A user should be able to see where the form starts, where each field belongs, and where the action buttons are.

Add the page and form layout styles:

CSS

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: system-ui, sans-serif;
  color: #111827;
  background-color: #f3f4f6;
}

.page {
  max-width: 720px;
  margin: 40px auto;
  padding: 0 16px;
}

.intro,
.support-form {
```

```
background-color: white;
border: 1px solid #e5e7eb;
border-radius: 8px;
}

.intro {
  margin-bottom: 24px;
  padding: 24px;
}

.support-form {
  padding: 24px;
  display: grid;
  gap: 18px;
}

.field {
  display: grid;
  gap: 6px;
}

.form-actions {
  display: flex;
  flex-wrap: wrap;
  gap: 12px;
}
```

.support-form becomes a one-column grid so the fields have consistent space between them.

.field also uses a small grid. That keeps each label, control, hint, and error message grouped together.

.form-actions uses Flexbox because the buttons are a small row of related actions. **flex-wrap: wrap** lets them move to another line if the available width gets tight.

None of this changes what the form means. It only makes the structure easier to scan.

Keep labels visible

Labels tell users what each field is for.

Placeholders can help with examples, but they are not a replacement for labels. Placeholder text disappears when the user types, and it is easy to forget what the field was asking for.

Style the labels and helper text:

CSS

```
.eyebrow {
  margin-top: 0;
  color: #2563eb;
  font-size: 14px;
  font-weight: 700;
}

h1,
p {
  margin-top: 0;
```

```
}

.summary,
.field-hint {
  color: #4b5563;
  line-height: 1.6;
}

label {
  color: #374151;
  font-weight: 700;
}

.field-hint,
.field-error {
  margin: 0;
  font-size: 14px;
}
```

font-weight: 700 makes labels easy to find while scanning.

The hint and error text use smaller type because they support the field. They should be readable, but they should not compete with the label.

 **Do not use placeholder text as the only label** - If the only label is inside the input, the user loses that label once they type. Keep a real visible label for normal form fields.

Make controls match the page

Browsers give form controls their own default styling. That is useful because the controls work immediately, but the defaults may not match your page font, spacing, or borders.

Start with the main text controls:

CSS

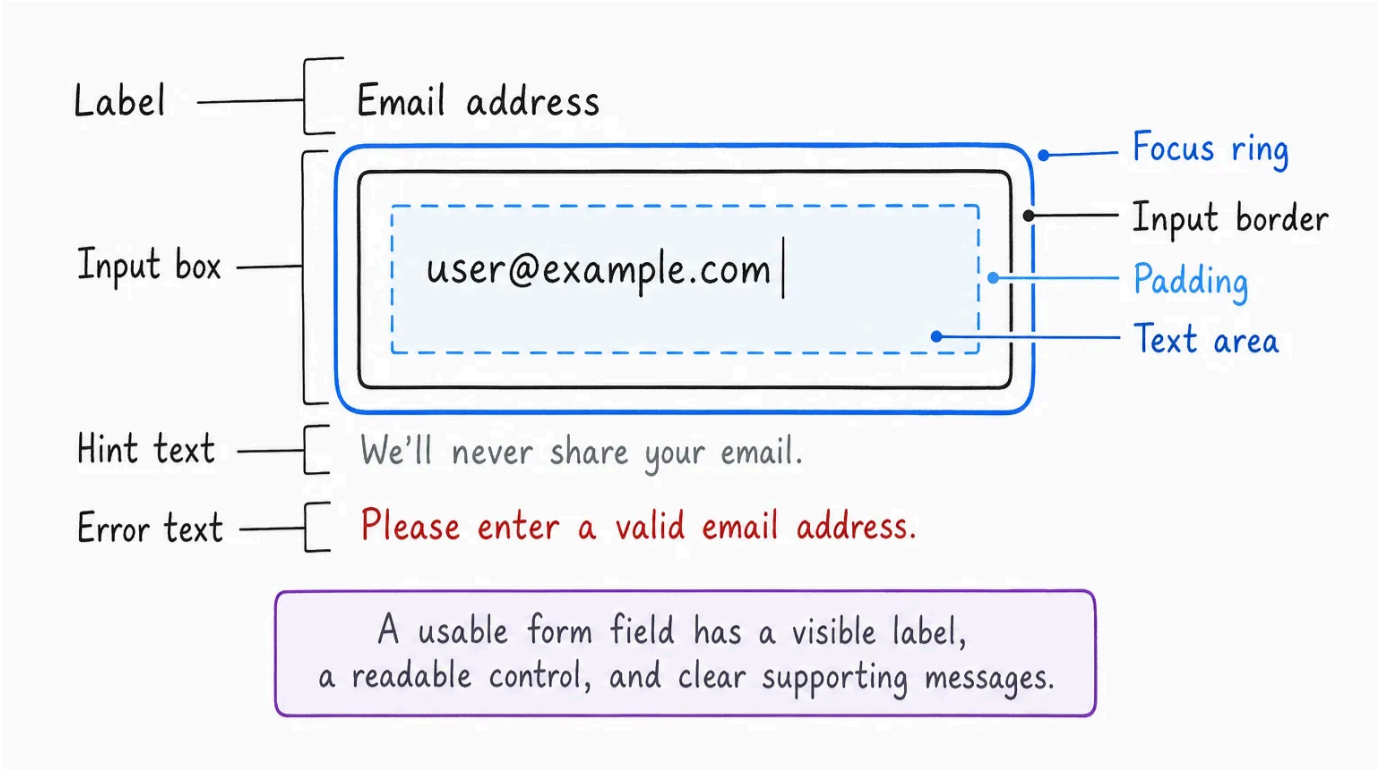
```
.field input,
.field select,
.field textarea {
  width: 100%;
  font: inherit;
  color: #111827;
  background-color: white;
  border: 1px solid #d1d5db;
  border-radius: 8px;
  padding: 10px 12px;
  line-height: 1.5;
}
```

.field input , **.field select** , and **.field textarea** target the controls that sit inside normal form fields. The checkbox is not inside **.field** , so it will not accidentally become **width: 100%** .

width: 100% makes each control fill the form column. That creates a clean edge for the user to scan.

font: inherit makes the controls use the same font as the page. Form controls often have browser-specific font defaults, so this line keeps the form visually consistent.

color controls the typed text, **background-color** controls the inside of the control, and **border** makes the editable area visible. **border-radius** softens the corners, **padding** gives text room inside the control, and **line-height** keeps typed text comfortable to read.



Let textareas grow in the right direction

A textarea is meant for longer text. It needs more room than a single-line input.

Add textarea-specific behavior:

```
css

.field textarea {
  min-height: 140px;
  resize: vertical;
}
```

min-height: 140px gives the message box a useful starting height.

resize: vertical lets the user make the textarea taller if they need more room. It also prevents sideways resizing, which can break the form layout.

That is a good example of form styling: you are not only changing appearance. You are preserving a useful behavior and guiding it so it fits the layout.

Keep placeholder text quiet

Placeholder text is an example, not the main instruction.

Style it so it feels secondary:

CSS

```
.field input::placeholder,
.field textarea::placeholder {
  color: #9ca3af;
}
```

`::placeholder` targets placeholder text inside inputs and textareas.

The color is lighter than normal text, so the user can tell it is not a real value.

Do not make placeholder text so faint that it becomes hard to read. Quiet still needs to be readable.

Make focus impossible to miss

Focus tells the user which control is active. This matters for everyone, but it is especially important for people using a keyboard. When they press Tab through the form, the focus style is their map.

Add a clear focus style:

CSS

```
.field input:focus-visible,
.field select:focus-visible,
.field textarea:focus-visible,
.check-field input:focus-visible,
button:focus-visible {
  outline: 3px solid #bfdbfe;
  outline-offset: 2px;
}

.field input:focus-visible,
.field select:focus-visible,
.field textarea:focus-visible {
  border-color: #2563eb;
}
```

`:focus-visible` matches when the browser decides a focus indicator should be shown, especially during keyboard navigation.

`outline` draws a visible ring outside the control.

`outline-offset` creates a little space between the control and the ring.

The second rule changes the border color only on controls that have the visible input border. That gives text fields, selects, and textareas a second focus cue without adding a meaningless border-color rule to buttons.

The checkbox and buttons are included because they can receive focus too.

⚠️ **Do not remove focus without replacing it** - Avoid `outline: none` unless you provide another clear focus style. A form without visible focus is much harder to use with a keyboard.

Show error states near the field

The email field in the HTML has `class="field has-error"` . In a real app, a validation system might add that class after checking the user's input. The class gives CSS a clear hook: style this field as a problem.

Add the error styles:

CSS

```
.field-error {
  color: #b91c1c;
  font-weight: 700;
}

.field.has-error input {
  border-color: #dc2626;
  background-color: #fef2f2;
}
```

`.field-error` makes the error text stand out.

`.field.has-error input` styles the input inside a field that has an error.

The border and background both change, so the error is not communicated by text alone and not communicated by color alone. The user sees the message and the field that needs attention.

CSS also has pseudo-classes like `:invalid` , but they can match before the user has had a fair chance to type. A class-based error state is often easier to control when you are learning or building app forms.

Style checkboxes without rebuilding them

Checkboxes and radio buttons are special controls. They have a native shape and behavior that users already understand.

For now, keep that native control and adjust the spacing and accent color:

CSS

```
.check-field {
  display: flex;
  align-items: flex-start;
  gap: 10px;
  color: #374151;
  font-weight: 500;
}

.check-field input{
  width: 18px;
```

```
height: 18px;
margin-top: 2px;
flex: none;
accent-color: #2563eb;
}
```

`.check-field` uses Flexbox because the checkbox and its text sit side by side.

`align-items: flex-start` keeps the checkbox aligned with the first line of text.

`gap: 10px` creates space between the box and the label text.

The checkbox gets a clear `width` and `height` so it is easy to click.

`flex: none` keeps the checkbox from stretching or shrinking strangely inside the row.

`accent-color` changes the native checked color without replacing the real checkbox with custom HTML.

That is a good beginner default: style the native control before trying to rebuild it.

Buttons need states too

The submit button is the main action. The disabled "Save draft" button shows that an action can exist but not be available.

Add the button styles:

CSS

```
button {
  font: inherit;
  border: 0;
  border-radius: 8px;
  padding: 10px 14px;
  font-weight: 700;
  cursor: pointer;
}

button[type="submit"] {
  color: white;
  background-color: #2563eb;
}

.secondary-button {
  color: #374151;
  background-color: #e5e7eb;
}

button:disabled {
  opacity: 0.55;
  cursor: not-allowed;
}
```


`font: inherit` keeps button text consistent with the page.

`border: 0` removes the browser's default button border because this design uses background color and shape instead.

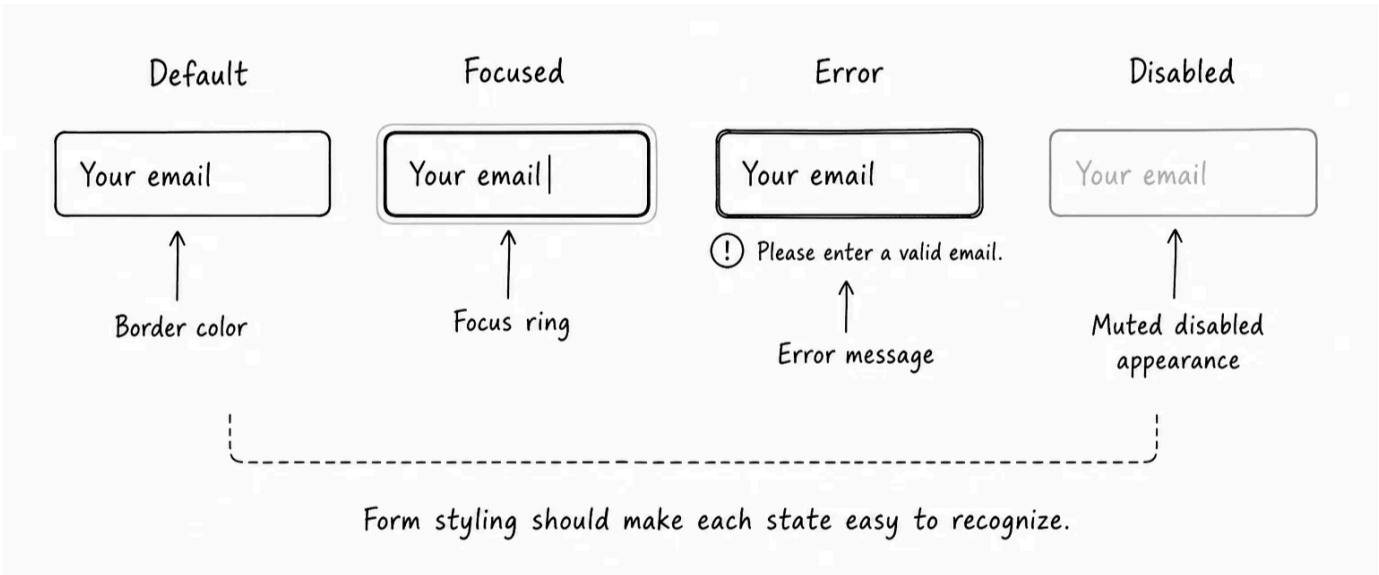
`border-radius` and `padding` make the button feel like a clickable control.

`cursor: pointer` gives mouse users a familiar cue on enabled buttons.

`button[type="submit"]` targets the main submit action.

`.secondary-button` targets the lower-priority button.

`button:disabled` styles real disabled buttons. The behavior comes from the HTML `disabled` attribute, and CSS only changes how that state looks.



Native form controls will not all look identical

Different browsers and operating systems can draw form controls slightly differently, especially select menus, date inputs, file inputs, checkboxes, and radio buttons, and that is normal.

Your first job is not to force every control to look identical everywhere. Your first job is to make the form readable, aligned, focused, and understandable while keeping the native behavior intact.

You will see CSS like `appearance: none` in custom form components. That property removes some native browser styling, but then you are responsible for rebuilding the missing visual states, so save that for later.

Inspect form states in DevTools

DevTools helps you test form CSS without guessing. Inspect an input and look at the Styles panel. Toggle padding, border, and `font: inherit` to see what each one changes.

Use the browser's force-state tools to test `:focus-visible` if your DevTools supports it. You can also press Tab through the form and watch the focus ring

move.

Inspect the email field and compare `.field.has-error input` with the normal field styles. The more specific error rule should override the normal border and background.

Inspect the disabled button and look for the `:disabled` rule. The button is disabled because of HTML; CSS only shows that state visually.

Form styling map

Here is the practical map:

Form part	CSS job
Form wrapper	Create a readable card or section around the form.
Field group	Keep label, control, hint, and error text together.
Label	Stay visible and easy to scan.
Input, select, textarea	Match the page font, spacing, border, and width.
Placeholder	Give an example without replacing the label.
Focus state	Show exactly which control is active.
Error state	Show what needs fixing near the field.
Checkbox or radio	Keep native behavior, adjust spacing and accent color.
Button	Show action priority and disabled state clearly.

The question is not "How do I make every control custom?" The better question is: "Can the user understand and complete this form?"

Try it

Use the same support form and test the states on purpose:

- Press Tab through the form. Make sure every input, select, textarea, checkbox, and button has a visible focus style.
- Remove `font: inherit` from the controls. Notice whether the form controls still match the page text.
- Remove the visible `<label>` for one field and rely only on the placeholder. Type into the field and notice why the label was important.

- Remove **has-error** from the email field. The error styling should disappear.
- Add **disabled** to the email input and inspect how the browser and your CSS treat it.
- Change **accent-color** on the checkbox and notice that the native checkbox still works.
- Increase the textarea content and resize it vertically. Check that it grows without breaking the form width.

Form styling is not about hiding the browser's controls. It is about making the controls easier to understand while preserving the behavior users depend on.

< 0 / 10 >

✔ Knew 0 Items

✧ Learnt 0 Items

⏮ Skipped 0 Items

🔄 ResetProgress

Test Your Understanding

What is the main goal of form styling?

[Click to Reveal the Answer](#)

✔ Already Know that

✧ Didn't Know that

⏮ Skip Question

✓

LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →

Modern selectors

Back in the selectors lesson, you learned the basics: element selectors, class selectors, ID selectors, groups, and descendants.

Modern CSS gives you more precise ways to ask questions about the HTML:

- Does this element have a certain attribute?
- Is this the current navigation link?
- Is this card marked as advanced?
- Is this item the last one in a list?
- Does this card contain a checked task?

That is the point of modern selectors. They help you use information that is already in the HTML, instead of adding a new class for every small styling case.

Use HTML that already has useful clues

In this lesson, you will work with a small lesson dashboard. The HTML has navigation links, lesson cards, difficulty levels, task checkboxes, and one external reference link. Put this markup in `index.html` :

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Modern selector practice</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <main class="page">
      <section class="intro">
        <p class="eyebrow">CSS practice</p>
        <h1>Use selectors that match real HTML clues</h1>
        <p class="summary">Modern selectors can match attributes, states, structure, and
      </section>

      <nav class="lesson-nav" aria-label="CSS lessons">
        <a href="/css/selectors">Selectors</a>
        <a href="/css/modern-selectors" aria-current="page">Modern selectors</a>
        <a href="https://developer.mozilla.org/">MDN reference</a>
      </nav>

      <section class="lesson-board" aria-label="Lesson cards">
        <article class="lesson-card" data-level="beginner">
          <p class="card-label">Beginner</p>
          <h2>Selectors recap</h2>
          <p>Review class selectors, group selectors, and descendant selectors.</p>
          <ul class="task-list">
```



```
<li><label><input type="checkbox" checked> Read the lesson</label></li>
<li><label><input type="checkbox"> Try the exercises</label></li>
</ul>
</article>

<article class="lesson-card" data-level="advanced">
  <p class="card-label">Advanced</p>
  <h2>Modern selectors</h2>
  <p>Use attributes, structure, and parent-aware selectors carefully.</p>
  <p class="card-note">Best after you understand specificity.</p>
  <ul class="task-list">
    <li><label><input type="checkbox"> Read the lesson</label></li>
    <li><label><input type="checkbox"> Inspect the matches</label></li>
  </ul>
</article>

<article class="lesson-card" data-level="beginner">
  <p class="card-label">Beginner</p>
  <h2>Practice project</h2>
  <p>Apply selectors to a small lesson dashboard.</p>
  <ul class="task-list">
    <li><label><input type="checkbox" checked> Build the layout</label></li>
    <li><label><input type="checkbox" checked> Style the cards</label></li>
  </ul>
</article>
</section>
</main>
</body>
</html>
```

Notice the useful clues already in the HTML:

- The current nav link has `aria-current="page"` .
- The external link starts with `https://` .
- Cards have `data-level="beginner"` or `data-level="advanced"` .

Modern selectors let CSS use those clues directly.

Add a simple base first

Start with the base styling so the page is readable before the selector-specific rules appear:

CSS

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: system-ui, sans-serif;
  color: #111827;
  background-color: #f3f4f6;
}

.page {
  max-width: 1040px;
  margin: 40px auto;
  padding: 0 16px;
}

.intro,
.lesson-nav,
```

```
.lesson-card {
  background-color: white;
  border: 1px solid #e5e7eb;
  border-radius: 8px;
}

.intro,
.lesson-nav,
.lesson-card {
  padding: 24px;
}

.intro,
.lesson-nav {
  margin-bottom: 24px;
}

.eyebrow,
.card-label {
  margin-top: 0;
  color: #2563eb;
  font-size: 14px;
  font-weight: 700;
}

h1,
h2,
p {
  margin-top: 0;
}

p,
.task-list {
  color: #4b5563;
  line-height: 1.6;
}

.lesson-nav {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  gap: 12px;
}

.lesson-nav a {
  color: #2563eb;
  font-weight: 700;
  text-decoration: underline 3px;
}

.lesson-board {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 16px;
}

.task-list {
  padding-left: 20px;
  margin-bottom: 0;
}
```

This CSS does not use modern selectors yet. It only gives the page a normal layout and readable card styles.

The next sections will use more precise selectors to style the clues already present in the HTML.

Select attributes when the HTML already says something useful

An attribute selector matches an element by one of its HTML attributes.

The current nav link already says `aria-current="page"` . You do not need to add a class like `.active-link` just to style it. Add this CSS to make the active link visible:

CSS

```
.lesson-nav a[aria-current="page"] {
  color: white;
  background-color: #2563eb;
  border-radius: 999px;
  padding: 6px 10px;
  text-decoration: none;
}
```

`a[aria-current="page"]` means "match an `<a>` element whose `aria-current` attribute is exactly `page` ."

The selector is scoped under `.lesson-nav` so it only styles the current link in this navigation area.

The CSS changes the current link's color, background, shape, spacing, and underline. The meaning still comes from the HTML attribute. CSS is only making that state visible.

Now style the advanced card using its data attribute:

CSS

```
.lesson-card[data-level="advanced"] {
  border-color: #f59e0b;
  background-color: #fffbeb;
}
```

`[data-level="advanced"]` matches a card with that exact custom data attribute.

Data attributes are useful when the HTML carries app-specific information that CSS can safely style. Here, `data-level` tells CSS whether a card is beginner or advanced.

💡 **Prefer meaningful existing hooks** - If the HTML already has a meaningful attribute like `aria-current` , `disabled` , `required` , or a clear `data-*` value, an attribute selector can be better than adding another styling-only class.

Match part of an attribute value

Some attribute selectors can match part of a value. The MDN link starts with `https://` , which tells you it goes to a full external URL.

Add a selector for links that start with **https://** :

CSS

```
.lesson-nav a[href^="https://"] {  
  color: #7c3aed;  
}
```

[href^="https://"] means "match links whose **href** value starts with **https://**."

The **^=** operator means "starts with." This kind of selector is useful for styling external links, file links, email links, or download links when the attribute value already reveals the type.

Keep the styling subtle here. The link should still look like a normal link, and the color only adds a small distinction.

Select structure instead of adding edge-case classes

Sometimes the useful clue is not an attribute. It is the element's position in a group.

The task list has several list items. If you want space between items, you could add a class to every item after the first one, but the structure already tells CSS what to do.

Add spacing between list items with the adjacent sibling selector:

CSS

```
.task-list li + li {  
  margin-top: 8px;  
}
```

li + li means "match a list item that comes immediately after another list item." The first item does not match, because it does not come after a previous item. The later items match, so they get top margin.

This is useful for spacing repeated items because it only creates space between items, not before the first item.

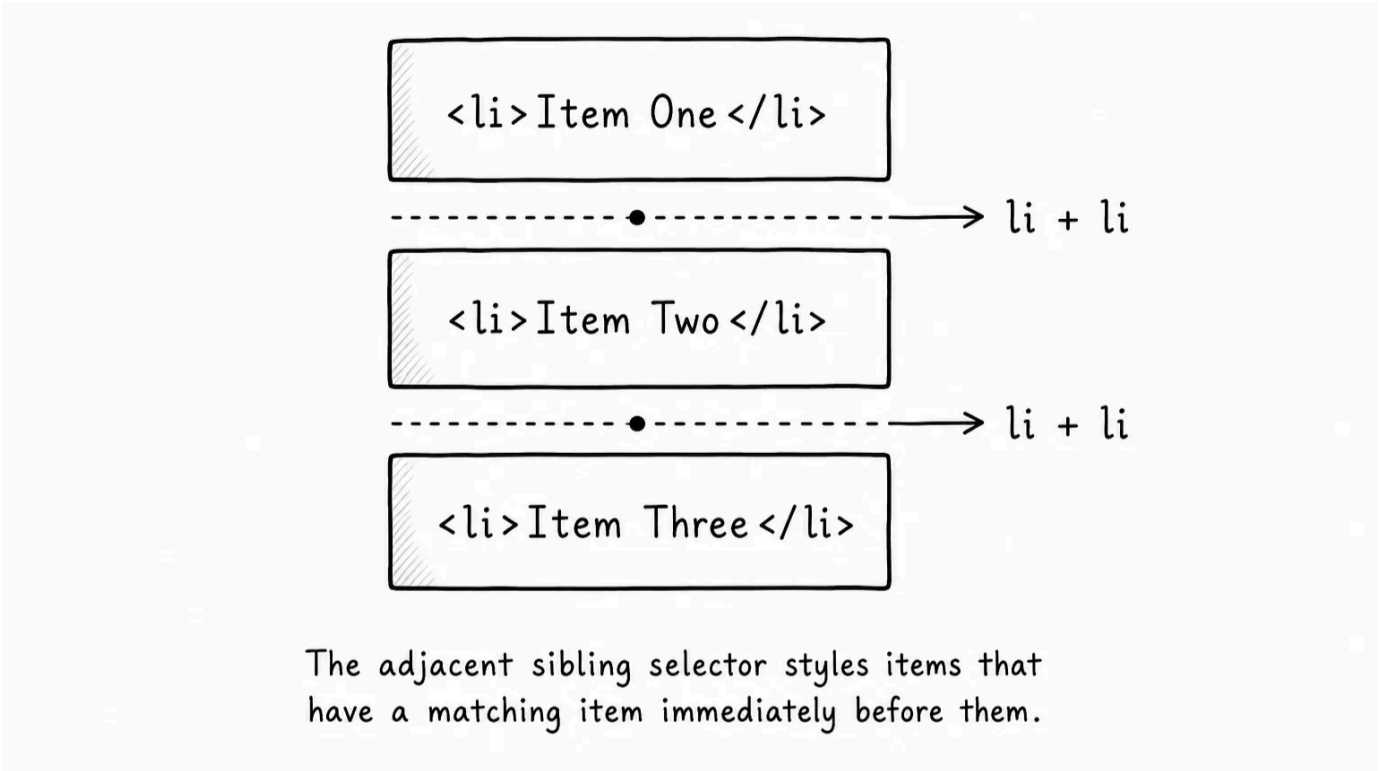
You can also select the last item in a group:

CSS

```
.task-list li:last-child {  
  font-weight: 700;  
}
```


:last-child matches an element when it is the last child inside its parent.

This example makes the last task heavier so you can see the match. In real projects, structural selectors are often used for spacing, borders, dividers, and small layout adjustments.



Use **:not()** to exclude a case

Sometimes the useful question is negative: "match cards except this kind." For example, you may want a quieter border on every card that is not advanced:

CSS

```
.lesson-card:not([data-level="advanced"]) {  
  border-color: #dbeafe;  
}
```

:not(...) means "match this element only if it does not match the selector inside the parentheses."

Here, **.lesson-card:not([data-level="advanced"])** matches lesson cards that are not advanced.

Use this when the exception is clear. Do not build long negative selectors that become hard to read. A simple class is often better when the logic gets complicated.

Use **:is()** to group selector options

Sometimes several selectors should receive the same style. You already know comma grouping:

CSS

```
.lesson-card h2,  
.lesson-card p,  
.lesson-card ul {  
  margin-bottom: 12px;  
}
```

```
}
```

`:is()` gives you another way to write that group:

CSS



```
.lesson-card :is(h2, p, ul) {  
  margin-bottom: 12px;  
}
```

`:is(h2, p, ul)` means "match any of these options."

The full selector means "inside `.lesson-card`, match any `h2`, `p`, or `ul`."

This is useful when the shared part of the selector is long. Instead of repeating `.lesson-card` three times, you write it once.

Use `:is()` for readability, not to make selectors look clever.

Use `:where()` for low-specificity defaults

`:where()` looks similar to `:is()`, but it has a different specificity behavior.

Specificity is selector strength, and you learned earlier that stronger selectors can override weaker selectors.

`:where()` matches elements like `:is()`, but the part inside `:where()` adds no specificity. That makes it useful for broad defaults you want to keep easy to override:

CSS



```
.lesson-card :where(h2, p, ul) {  
  margin-top: 0;  
}
```

This sets simple spacing defaults inside cards without making the selector stronger than it needs to be.

Later, a normal class like `.card-note` can override those defaults easily:

CSS



```
.card-note {  
  margin-top: 16px;  
  color: #92400e;  
  font-weight: 700;  
}
```

`.card-note` can set its own margin because the default came from `:where()` instead of a stronger selector.

The practical habit is simple: use `:where()` when you are writing gentle defaults. Use normal classes when you are styling a specific component part.

 **Do not fight yourself with overly strong selectors** - Modern selectors can become very specific if you chain too much together. If you need three ancestors, two attributes, and a state to style something, ask whether a clear class would be easier.

Use `:has()` when the parent depends on its contents

Most selectors match an element based on itself or its ancestors.

`:has()` lets you match a parent based on something inside it.

In the lesson cards, some tasks are already checked. You can style a card differently when it contains at least one checked task:

CSS

```
.lesson-card:has(.task-list input:checked) {  
  box-shadow: 0 12px 24px rgba(15, 23, 42, 0.08);  
}
```

`:has(.task-list input:checked)` means "match this card if it has a checked input inside `.task-list`."

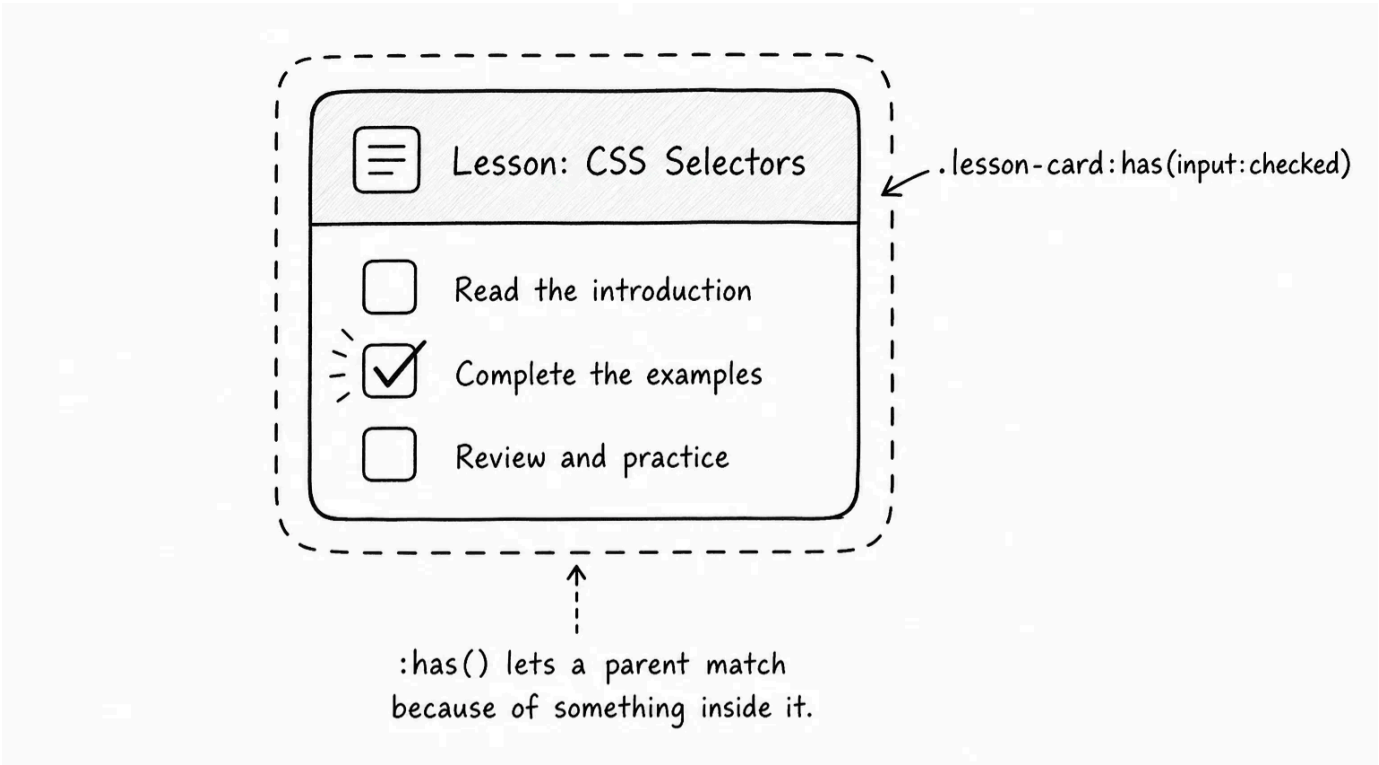
The selector reads like a condition:

text

Select a lesson card if it has a checked task.

That is useful for parent styling. The card can react to something inside it without adding a separate `.has-progress` class.

Use `:has()` when the parent truly depends on a child or descendant. If the parent already has a meaningful class or data attribute, that simpler hook may still be better.



Inspect which selectors match

DevTools is still the best way to check selector behavior.

Inspect the current nav link. You should see the `.lesson-nav a[aria-current="page"]` rule in the Styles panel.

Inspect the MDN link. You should see the `a[href^="https://"]` rule.

Inspect the advanced card. You should see the `[data-level="advanced"]` rule.

Inspect a card with checked tasks. You should see the `:has()` rule if the card contains a checked input.

If a selector does not work, check the exact HTML:

- Does the attribute exist?
- Does the attribute value match exactly?
- Is the element in the structure your selector expects?
- Is the checked state actually present?
- Is another rule winning in the cascade?

Modern selectors are powerful, but they are still selectors. They only match what the browser can actually see in the DOM.

Modern selector map

Here is the practical map:

Selector	Question it asks
<code>[aria-current="page"]</code>	Does this element have this exact attribute value?

Selector	Question it asks
<code>[href^="https://"]</code>	Does this attribute value start with this text?
<code>li + li</code>	Does this element come immediately after another matching
<code>:last-child</code>	Is this element the last child of its parent?
<code>:not(...)</code>	Does this element not match this condition?
<code>:is(...)</code>	Does this element match any option in this group?
<code>:where(...)</code>	Does this element match any option, without adding selector
<code>:has(...)</code>	Does this element contain something that matches?

The useful habit is not to use the newest selector everywhere. The useful habit is to use what is clearer and makes sense.

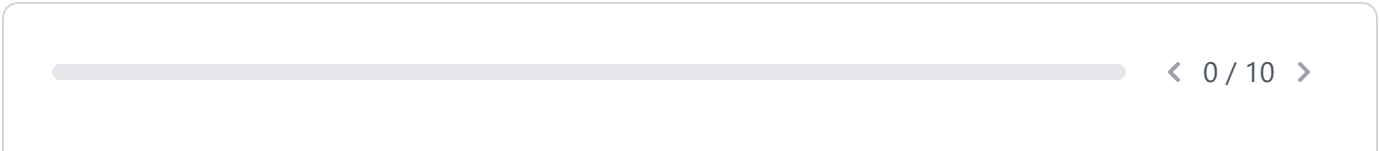


Try it

Use the same dashboard and test the selectors directly:

- Change `aria-current="page"` to another nav link. The current-link style should move.
- Change the MDN link from `https://developer.mozilla.org/` to `/docs`. The external-link color should stop applying.
- Change a card from `data-level="advanced"` to `data-level="beginner"`. Notice which card styles change.
- Add a new task to the bottom of a task list. Check how `li + li` and `:last-child` behave.
- Remove every checked checkbox from one card. The `:has()` shadow should disappear from that card.
- Replace the `:is()` rule with the comma-grouped version and confirm the result is the same.
- Try overriding the `:where()` margin rule with a simple class. Notice that the class can win easily.

Modern selectors are best when they make your CSS match the real meaning or structure already in the HTML. If a selector becomes hard to explain out loud, it is probably too clever.





Knew

0 Items



Learnt

0 Items



Skipped

0 Items



ResetProgress

Test Your Understanding

What problem do modern selectors solve?

[Click to Reveal the Answer](#)



Already Know that



Didn't Know that



Skip Question



LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →

CSS variables and organization

Once a stylesheet grows, one of the hard parts is keeping repeated decisions consistent:

- The same blue appears in links, buttons, labels, and borders.
- The same spacing values appear in cards, sections, and rows.
- The same radius appears on every panel.
- A component variant needs a different accent color without rewriting the whole component.

CSS variables help with that by giving a value a name, so you can reuse it and change it from one place.

Start with repeated values

Use a small dashboard where the same design choices appear in several places. The page works, but the CSS repeats the same colors, spacing, and radius values many times.

Start with this markup:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>CSS variables practice</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <main class="page">
      <section class="intro">
        <p class="eyebrow">CSS practice</p>
        <h1>Keep repeated design values under control</h1>
        <p class="summary">Colors, spacing, and component variants become easier to manage</p>
      </section>

      <section class="course-grid" aria-label="Course cards">
        <article class="course-card">
          <p class="card-label">Core lesson</p>
          <h2>CSS selectors</h2>
          <p class="card-description">Review how selectors match elements and how the browser interprets them</p>
          <a class="card-action" href="/css/selectors">Open lesson</a>
        </article>

        <article class="course-card is-success">
          <p class="card-label">Project ready</p>
          <h2>Responsive page</h2>
          <p class="card-description">Use layout, media, and accessibility habits together</p>
        </article>
      </section>
    </main>
  </body>
</html>
```



```
    <a class="card-action" href="/css/project">Open project</a>
  </article>
</section>

<aside class="notice">
  <strong>Reminder:</strong> Keep class names meaningful and keep repeated design
</aside>
</main>
</body>
</html>
```

Now add this deliberately repetitive CSS:

CSS

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: system-ui, sans-serif;
  color: #111827;
  background-color: #f3f4f6;
```

```
.page {
  max-width: 960px;
  margin: 40px auto;
  padding: 0 16px;
}

.intro,
.course-card,
.notice {
  background-color: white;
  border: 1px solid #e5e7eb;
  border-radius: 8px;
  padding: 24px;
}

.intro,
.course-grid {
  margin-bottom: 24px;
}

.eyebrow,
.card-label {
  margin-top: 0;
  color: #2563eb;
  font-size: 14px;
  font-weight: 700;
}

h1,
h2,
p {
  margin-top: 0;
}

.summary,
.card-description {
  color: #4b5563;
  line-height: 1.6;
}

.course-grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 16px;
}

.course-card {
  border-top: 4px solid #2563eb;
```



```
}

.course-card.is-success {
  border-top-color: #059669;
}

.course-card.is-success .card-label {
  color: #059669;
}

.card-action {
  display: inline-block;
  margin-top: 16px;
  padding: 10px 14px;
  color: white;
  background-color: #2563eb;
  border-radius: 8px;
  font-weight: 700;
  text-decoration: none;
}

.notice {
  color: #1e3a8a;
  background-color: #dbeafe;
  border-color: #93c5fd;
}
```

Open the page. It should look like a small dashboard with two cards and a notice. Now look at the CSS as a maintenance problem.

The blue `#2563eb`, the radius `8px`, and the spacing values `16px` and `24px` all appear in several places.

If the brand color changes, you have to hunt through the stylesheet and replace the same value in multiple rules.

That is the problem CSS variables solve.

Give repeated values names

A CSS variable is written as a custom property. Custom property names start with two dashes:

CSS

```
:root {
  --color-brand: #2563eb;
}
```

`:root` selects the top-level element of the page. For normal HTML pages, you can think of it as the highest place to put values that should be available everywhere.

In this rule, `--color-brand` is the custom property name, and `#2563eb` is the value stored inside that name.

The name is yours, but it must start with `--`.

Now add a small set of variables at the top of the stylesheet:

CSS

```
:root {
  --color-text: #111827;
  --color-muted: #4b5563;
  --color-page: #f3f4f6;
  --color-surface: #ffffff;
  --color-border: #e5e7eb;
  --color-brand: #2563eb;
  --color-brand-soft: #dbeafe;
  --color-brand-border: #93c5fd;
  --color-success: #059669;
  --space-sm: 8px;
  --space-md: 16px;
  --space-lg: 24px;
  --radius-card: 8px;
}
```

These names describe design decisions:

- `--color-brand` is the main brand color.
- `--color-muted` is for quieter text.
- `--space-md` is a reusable spacing value.
- `--radius-card` is the radius used on card-like surfaces.

The point is not to make a variable for every single value. The point is to name values that repeat or carry meaning.

💡 Name values by job - Prefer names like `--color-brand` , `--color-border` , and `--space-lg` over names like `--blue` or `--twenty-four` . A job-based name can survive a design change.

Use variables with `var()`

Defining a variable does not change the page by itself. To use the value, call it with `var(...)` .

Replace the first few repeated values:

CSS

```
body {
  margin: 0;
  font-family: system-ui, sans-serif;
  color: var(--color-text);
  background-color: var(--color-page);
}

.intro,
.course-card,
.notice {
  background-color: var(--color-surface);
  border: 1px solid var(--color-border);
  border-radius: var(--radius-card);
  padding: var(--space-lg);
}
```

`var(--color-text)` means "use the value stored in `--color-text`."

`var(--color-page)` means "use the value stored in `--color-page`."

The browser replaces each `var(...)` with the custom property value when it computes the final CSS.

Now the page should look the same, but the stylesheet is easier to change.

If you change `--color-page` in `:root`, every place using `var(--color-page)` updates.

Replace the repeated brand values

Now replace more repeated values with variables:

CSS

```
.intro,
.course-grid {
  margin-bottom: var(--space-lg);
}

.eyebrow,
.card-label {
  margin-top: 0;
  color: var(--color-brand);
  font-size: 14px;
  font-weight: 700;
}

.summary,
.card-description {
  color: var(--color-muted);
  line-height: 1.6;
}

.course-grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: var(--space-md);
}

.course-card {
  border-top: 4px solid var(--color-brand);
}

.card-action {
  display: inline-block;
  margin-top: var(--space-md);
  padding: 10px 14px;
  color: white;
  background-color: var(--color-brand);
  border-radius: var(--radius-card);
  font-weight: 700;
  text-decoration: none;
}

.notice {
  color: #1e3a8a;
  background-color: var(--color-brand-soft);
  border-color: var(--color-brand-border);
}
```

The result should still look the same. That is an important point: variables do not have to change the design immediately. They make the design easier to maintain.

Try changing `--color-brand` from `#2563eb` to `#7c3aed`. The label, card border, and button should change together. That is the practical payoff.

Variables follow the cascade

CSS variables are still CSS, which means they follow the cascade and can be scoped to part of the page.

Right now, the success card needs a different accent color. The old CSS did this by styling each piece separately:

CSS

```
.course-card.is-success {
  border-top-color: #059669;
}

.course-card.is-success .card-label {
  color: #059669;
}
```

That works, but it repeats the idea: this card's accent should be green. Instead, replace those separate success-card rules with a local accent variable on each card:

CSS

```
.course-card {
  --card-accent: var(--color-brand);
  border-top: 4px solid var(--card-accent);
}

.course-card .card-label {
  color: var(--card-accent);
}

.course-card.is-success {
  --card-accent: var(--color-success);
}
```

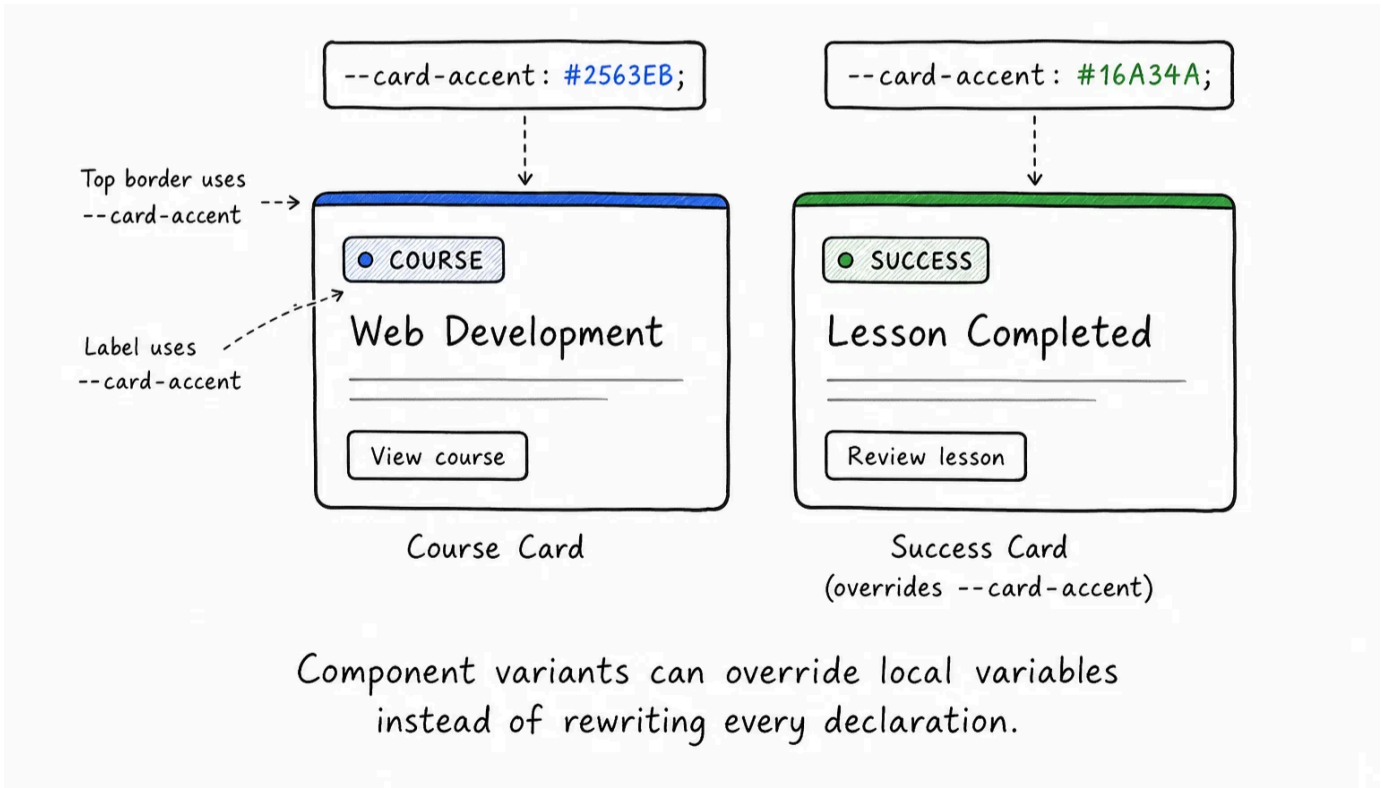
The normal card sets `--card-accent` to the brand color.

The card border and card label both use `var(--card-accent)`.

The success card overrides only `--card-accent`.

Because custom properties inherit, the `.card-label` inside the success card can use the green value from its parent card.

This is one of the best uses of CSS variables: define a component once, then change a small set of local values for variants.



Use fallbacks when a value might be missing

`var()` can include a fallback value. The fallback is used if the custom property is missing or invalid.

```
CSS

.card-action {
  background-color: var(--color-action, #2563eb);
}
```

This means: use `--color-action` if it exists. If it does not exist, use `#2563eb`.

Fallbacks are useful when:

- A component may be used on pages that do not define every variable.
- A theme may provide optional values.
- You want a safe default while the stylesheet is being refactored.

For this lesson, most values live in `:root`, so fallbacks are not required everywhere. Use them where a missing value would make the component hard to read or use.

Organize the stylesheet by job

Variables help values stay consistent, and organization helps people find the right rule later.

CSS comments use this syntax:

```
CSS

/* This is a CSS comment. */
```

The browser ignores comments because they are for people reading the file.

A small stylesheet can be organized like this:

CSS

```
/* 1. Design values */
:root {
  --color-text: #111827;
  --color-muted: #4b5563;
  --color-page: #f3f4f6;
  --color-surface: #ffffff;
  --color-border: #e5e7eb;
  --color-brand: #2563eb;
  --color-brand-soft: #dbeafe;
  --color-brand-border: #93c5fd;
  --color-success: #059669;
  --space-sm: 8px;
  --space-md: 16px;
  --space-lg: 24px;
  --radius-card: 8px;
}

/* 2. Base styles */
body {
  margin: 0;
  font-family: system-ui, sans-serif;
  color: var(--color-text);
  background-color: var(--color-page);
}

/* 3. Layout */
.page {
  max-width: 960px;
  margin: 40px auto;
  padding: 0 var(--space-md);
}

.course-grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: var(--space-md);
}

/* 4. Components */
.course-card {
  --card-accent: var(--color-brand);
  background-color: var(--color-surface);
  border: 1px solid var(--color-border);
  border-top: 4px solid var(--card-accent);
  border-radius: var(--radius-card);
  padding: var(--space-lg);
}

/* 5. Variants */
.course-card.is-success {
  --card-accent: var(--color-success);
}
```

The exact section names can change by project. The useful habit is to group rules by job:

- Design values first.
- Base element styles next.
- Layout rules near other layout rules.
- Component rules near the component.
- Variants and state rules close enough that they are easy to find.

Do not over-organize a tiny file. But once a stylesheet grows, sections help you avoid hunting.

Do not turn every value into a variable

CSS variables are useful, but using too many can make CSS harder to read. This is probably not worth a variable:

CSS

```
.card-action {  
  padding: 10px 14px;  
}
```

The two values are specific to this button shape. If they are not reused elsewhere, leaving them directly in the rule is fine.

This is more likely worth a variable:

CSS

```
:root {  
  --radius-card: 8px;  
}
```

The same radius appears on cards, notices, and buttons. If the design changes from `8px` to `10px`, one variable can update the shared shape.

Use variables for repeated values, meaningful design decisions, theme values, and component variants.

Leave one-off details alone until they actually need a name.

 **Variables do not fix unclear design decisions** - A variable name should make the CSS easier to understand. If `--thing`, `--blue-2`, or `--random-space` makes you stop and guess, the variable is not helping yet.

Inspect variables in DevTools

DevTools can show how variables resolve. Inspect the normal card and look at the `border-top` declaration; you should see the browser resolving `var(--card-accent)` to the brand color.

Then inspect the success card. The same `border-top` declaration should resolve to the success color because `.course-card.is-success` overrides `--card-accent`.

Try changing `--color-brand` in DevTools, and every declaration using `var(--color-brand)` should update.

If a variable does not work, check these things:

- Does the name start with `--` when you define it?
- Does the `var(...)` call use the exact same name?
- Is the variable defined on this element or inherited from an ancestor?
- Is another rule overriding the variable later in the cascade?
- Does `var(...)` need a fallback?

Variables are easier to debug when you remember that they still follow normal CSS rules.

CSS variable map

Here is the practical map:

Pattern	What it does
<code>--color-brand: #2563eb;</code>	Defines a custom property.
<code>var(--color-brand)</code>	Uses the custom property value.
<code>var(--color-brand, #2563eb)</code>	Uses a fallback if the custom property is missi
<code>:root { ... }</code>	Stores values that should be available across t
<code>.course-card { --card-accent: ... }</code>	Creates a local component value.
<code>.is-success { --card-accent: ... }</code>	Overrides the local value for a variant.

Try it

Use the same dashboard and test the variables directly:

- Change `--color-brand` in `:root`. Notice which parts of the page update together.
- Change `--radius-card` from `8px` to `16px`. Notice how shared rounded corners change.
- Remove `--card-accent` from `.course-card`. Check what happens to the card border and card label.
- Change `.course-card.is-success` to set `--card-accent: #dc2626;`. Notice that the success card variant changes without rewriting the border

and label rules.

- Add a fallback to one `var()` call, then temporarily misspell the variable name. Confirm that the fallback is used.
- Add a second `.course-card.is-success` rule later in the stylesheet with a different `--card-accent`. Notice that the later matching variable declaration wins.
- Add one new variable only if a value repeats at least twice. Leave one-off values alone.

CSS variables are most useful when they make change safer. If a value has a clear job and appears in several places, give it a name. If it is a one-time detail, keep the CSS direct.

< 0 / 10 >

✔ Knew 0 Items

✧ Learnt 0 Items

⏮ Skipped 0 Items

🔄 ResetProgress

Test Your Understanding

What is a CSS variable?

[Click to Reveal the Answer](#)

✔ Already Know that

✧ Didn't Know that

⏮ Skip Question

✓

LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →